

INFORMATIQUE

2025 - 2026

Spécification et Preuve de Programmes

SuP_Fc

Spécification et preuve de programmes

Support de cours

Enseignement à distance

Université Marie et Louis Pasteur

Françoise Bellegarde	Jean-François Couchot	Alain Giorgetti
Christophe Guyeux	Jacques Julliand	

Version du 5 janvier 2026

Avant-propos

Ce document est le support du cours sur la spécification et la preuve de programmes qui est dispensé en télé-enseignement en master d’informatique à l’Université Marie et Louis Pasteur, à partir de l’année universitaire 2025-2026.

Le contenu de ce support de cours est issu de documents antérieurs, rédigés par plusieurs enseignants, pour diverses unités d’enseignement, comme détaillé ci-après. Un premier cours à distance sur la preuve de programmes a été créé par Jacques Julliand en 1998. Son contenu initial était une partie de celui de l’ouvrage de J. Julliand intitulé “**Cours et exercices corrigés d’algorithmique - Vérifier, tester et concevoir des programmes en les modélisant**”, édité par Vuibert [Jul10]. À partir de février 2019, Alain Giorgetti a remplacé Jacques Julliand comme responsable et tuteur de cet enseignement à distance, et a revu ce contenu, sous la forme du chapitre 6 et d’une partie du chapitre 1. Il a ajouté le chapitre 7 sur l’utilisation de l’outil Why3, principalement à partir d’un fragment du langage C en entrée, et d’un langage de spécification original dédié à ce fragment, appelé micro-C, créé par Jean-Christophe Filliâtre, co-auteur de Why3.

Jusqu’en août 2026, cette unité d’enseignement représentait 3 crédits ECTS, sur un demi-semester de la licence d’informatique à distance de l’Université de Franche-Comté, transformée en Université Marie et Louis Pasteur en janvier 2026. En septembre 2026, cette unité d’enseignement a été augmentée à 6 crédits ECTS et étendue sur un semestre du master. Ce changement de niveau et cette extension de durée ont permis d’une part de remplacer dans le chapitre 7 le langage confidentiel micro-C par le langage natif WhyML de Why3, plus intéressant d’un point de vue pédagogique et formateur, et d’autre part d’ajouter deux compléments. Ce chapitre central est précédé par les chapitres 2 à 5, qui introduisent très progressivement une partie de la logique de Why3. Enfin, les chapitres 8 à 10 introduisent progressivement la programmation et la preuve de propriétés de programmes récursifs.

Les chapitres 2 et 3 et l’annexe A ont été adaptés par Alain Giorgetti en 2007, pour un module de troisième année de licence à distance sur les méthodes formelles, à partir d’un support de cours de l’I.U.T. de Belfort-Montbéliard co-écrit par Christophe Guyeux et Jean-François Couchot. Les chapitres 4, 8, 9, 10 et l’annexe B sont issus de deux chapitres de ce module sur les méthodes formelles, co-écrits avec Françoise Bellegarde. Le chapitre 5 est une création.

Avertissement

Ce document est soumis aux règles de code de la propriété intellectuelle de ses auteurs. Ceci inclut une obligation de citation et de référencement lors de toute utilisation de ce document. De plus, toute contrefaçon, plagiat, reproduction ou représentation illicite encourt une poursuite pénale.

Code de la Propriété Intellectuelle, article L122-4 : Toute représentation ou reproduction intégrale ou partielle faite sans le consentement de l’auteur ou de ses ayants droit ou ayants cause est illicite. Il en est de même pour la traduction, l’adaptation ou la transformation, l’arrangement ou la reproduction par un art ou un procédé quelconque.

Table des matières

1	Introduction	9
2	Logique des propositions, avec Why3	19
3	Logique du premier ordre, avec Why3	37
4	Types inductifs	55
5	Théories du premier ordre	65
6	Logique de Hoare	73
7	Spécifications et programmes WhyML	89
8	Calculs inductifs	97
9	Preuves de propriétés par récurrence	109
10	Preuve inductive, vérification de programmes récursifs	117
11	Références bibliographiques	125
12	Solutions des exercices du cours 2 sur la logique des propositions	131
13	Solutions des exercices du cours 3 sur la logique des prédicats	133
14	Solutions des exercices du cours 4 sur les types inductifs	137
15	Solutions des exercices du cours 6 sur la logique de Hoare	139
16	Solutions des exercices du cours 8 sur les calculs inductifs	141
17	Solutions des exercices du cours 9 sur la preuve par récurrence	145
18	Solutions des exercices du cours 10 sur les preuves inductives	147

A	Algèbre de Boole	151
B	Vérification et inférence de types	155
C	Solutions des exercices de l'annexe B sur la vérification et l'inférence de types	169

Chapitre 1

Introduction

Ce cours porte sur des méthodes et outils permettant d’effectuer des raisonnements rigoureux pour analyser la cohérence des algorithmes et des programmes qui les implémentent.

Les parties 1.1 à 1.5 de cette introduction définissent un peu de vocabulaire. Ces définitions s’appuient sur d’autres notions (soulignées) qui sont définies ensuite. Les propriétés que nous voulons vérifier sont précisées dans la partie 1.3. Les méthodes de vérification adaptées sont décrites dans la partie 1.4. Le tout, et plus particulièrement ce qui est écrit en **gras**, fixe le cadre de ce cours.

1.1 Spécifier

1.1.1 Spécifier, qu’est-ce que c’est ?

Le verbe “spécifier” et le substantif “spécification” ne sont pas des mots du langage le plus courant. Ils désignent une activité technique essentielle dans divers domaines, en particulier en informatique, qui doit être bien distinguée de celle de développement de logiciel, et associée à elle.

Dans le cadre de l’informatique, on désigne par “spécification” la description initiale d’un problème informatique. Une spécification guide le développement d’un logiciel en mettant en avant ses principales caractéristiques attendues, sans entrer dans les détails. Une spécification est dite *fonctionnelle* si elle décrit les fonctionnalités du logiciel. D’autres spécifications, dites non fonctionnelles, peuvent par exemple décrire les performances ou l’architecture du logiciel. Une spécification est dite *formelle* si elle est exprimée dans un langage exploitable par une machine. Ce langage est appelé *langage de spécification formelle*. Un tel langage sera présenté dans le chapitre 6.

Sous le nom de “spécification”, ce cours ne traite que de spécifications fonctionnelles et formelles.

1.1.2 Comment spécifier ?

Spécifier est une activité essentielle pour améliorer la qualité du logiciel. On peut voir la spécification comme l'expression d'un problème informatique, qu'on souhaite résoudre. Résoudre un problème informatique, c'est en donner une solution algorithmique.

Par nature, la spécification sera déclarative (logique), donc formalisée dans une logique mathématique. Par contraste, les algorithmes seront opérationnels, c'est-à-dire exécutables. Ils sont écrits dans un langage de programmation.

1.2 Vérifier

À partir d'un programme et d'une spécification, la question qui se pose naturellement est celle de la correction du programme vis-à-vis de la spécification. On parle aussi de "cohérence" ou de "conformité" entre ces deux formalisations. Nous décrivons dans cette partie les enjeux (le pourquoi) et les principes (le comment) de la vérification du logiciel, brièvement et de manière générale.

1.2.1 Pourquoi vérifier ?

La vérification nécessite un effort et a un coût qui ne sont souvent justifiés que pour des systèmes dits "critiques", qui mettent en jeu des vies humaines ou de lourds enjeux économiques. Mais une technique de vérification conçue sur des programmes quelconques peut être adaptée à des systèmes critiques. De ce point de vue, on peut apprendre et pratiquer des techniques de vérification dans le cadre général des programmes quelconques.

Un fort besoin de vérification se fait sentir lorsque la description d'un programme, même non critique, est devenue trop complexe pour convaincre seule que le programme satisfait ses propriétés attendues, celles pour lesquelles il est en cours de réalisation. La vérification intervient alors pour rétablir la confiance. Elle opère souvent par confrontation de points de vue sur ce programme. En pratique, la vérification s'inscrit dans la partie "évaluation" d'un processus de développement composé d'étapes de production et d'étapes d'évaluation qui contrôlent la qualité de cette production. C'est avant tout la complexité des programmes étudiés qui motive les travaux de spécification formelle et de vérification présentés dans ce cours.

1.2.2 Comment vérifier ?

On trouve dans la littérature deux grandes classifications des techniques de vérification. La première classification distingue les techniques de test, de model-checking et de preuve [Dad06, chapitre 2]. La seconde classification distingue les analyses statiques (sans exécution du programme) et les analyses dynamiques (par exécution du programme) [Pet15, partie 1.1]. Certaines techniques se situent à la frontière entre ces catégories, ou sont des combinaisons de techniques [Pet15, partie 2.1]. Plutôt que de dissenter longuement sur ces catégories et la pertinence

de ces classifications, nous inscrivons clairement ce cours parmi les analyses statiques, sous le nom de “**vérification déductive**”. Cette expression sera définie et justifiée dans la partie 1.4.

1.2.3 Mécanisation/automaticité

Pour réduire le coût de vérification et rendre les méthodes de vérification accessibles au plus grand nombre, nous étudions essentiellement des solutions assistées par des logiciels souvent capables d’automatiser complètement la vérification. Nous ne nous interdisons néanmoins pas d’utiliser des méthodes interactives, à l’aide d’outils appelés “assistants de preuve”, dans certains contextes où les outils de vérification automatique échouent.

Afin d’utiliser ces outils à bon escient, il est nécessaire de comprendre les fondements théoriques de la vérification déductive, mais aussi sa mise en œuvre en pratique, dans ces outils. C’est pourquoi ce cours couvre tout le spectre qui va des concepts fondamentaux requis, de la logique des propositions à la logique de Hoare, jusqu’à l’utilisation pratique d’un logiciel de vérification sur diverses applications.

1.3 Propriétés d’états

La vérification du logiciel est fondée sur un large éventail de méthodes formelles. Chacune d’elles est dédiée à un certain paradigme de programmation (programmation impérative, programmation fonctionnelle, systèmes concurrents, etc.) et se focalise sur certains aspects de la correction (terminaison, correction partielle, raffinement, etc.).

Ce cours ne porte que sur des programmes impératifs, surtout sur leur correction partielle et un peu sur leur terminaison.

La correction de ces programmes se réduit à la question suivante : Comment un programme donné modifie la valeur de vérité de propriétés portant sur son état ? Sous le nom de “*propositions*”, Robert W. Floyd [Flo67] a proposé d’utiliser ces propriétés d’états pour spécifier les ensembles d’états atteints lors de l’exécution d’un organigramme (“*flowchart*” en anglais). Plus tard, ces propriétés ont été insérées dans le code source des programmes impératifs, pour former des **programmes annotés**. Ainsi, la “cohérence d’un programme impératif avec sa spécification” est devenue la “correction du programme annoté à partir de cette spécification”. Lorsque les programmes sont ainsi annotés, leur vérification s’inscrit dans le cadre de leur analyse statique, en prolongement de leur analyse syntaxique.

1.4 Vérification déductive

Les expressions “preuve de programmes” et “vérification de programmes” sont pratiques, mais elles sont abusives et imprécises. On ne vérifie pas des programmes. On vérifie qu’ils satisfont certaines propriétés attendues. De plus, la notion de programme n’est pas claire : Vérifie-t-on un code source ou un code compilé ? un algorithme ou une implémentation ? Quelles sont les méthodes de vérification utilisées ? Comme ces questions – et bien d’autres – se posent de manière

récurrente dans ce domaine, cette partie fixe un vocabulaire précis.

Après avoir spécifié des propriétés d'un programme impératif, nous souhaitons que la cohérence entre le programme et sa spécification soit vérifiée de manière générique (une fois et une seule pour toutes les valeurs des paramètres d'entrée du programme) et (si possible) automatique. Dans ce but, nous suivons la démarche la plus répandue dans ce domaine, qui décompose la vérification en deux étapes : La première étape est une traduction automatique des conditions de correction en formules dans une logique adaptée. La seconde étape est l'application d'une procédure de décision (voir partie 1.5) de la validité de toutes les formules d'un fragment logique incluant tous les formules issues de la première étape. Dans toute la suite, nous désignons cette démarche en deux étapes par **vérification déductive**, ou **preuve de programmes**.

Dans toute sa généralité, la vérification déductive est un problème indécidable, c'est-à-dire qu'il n'existe pas d'algorithme pour résoudre ce problème dans tous les cas. En pratique, nous appliquons la seconde étape à l'aide d'outils appelés "prouveurs", même lorsque qu'aucune procédure de décision n'est connue pour la formule issue de la première étape. En effet, ces prouveurs implémentent beaucoup d'heuristiques qui font réussir la validation automatique de ces formules au-delà des limites des résultats théoriques connus.

Ainsi, la preuve de programmes devient de plus en plus accessible, car elle est progressivement assistée par des outils de plus en plus puissants et intégrée dans des environnements de vérification de plus en plus répandus et faciles à utiliser.

Nous étudions dans ce cours des spécifications et des programmes impératifs auxquels il devient raisonnable d'appliquer la méthode de vérification déductive, compte tenu de l'avancée actuelle des outils de vérification.

1.4.1 Précondition, postcondition, contrat

La vérification déductive permet de vérifier des algorithmes en se posant la question suivante :

Est-ce que le fragment d'algorithme A réalise la tâche T ?

Le fragment d'algorithme A est défini par une composition d'opérations élémentaires transformant des variables d'état. La tâche T est définie par deux propriétés sur les variables :

- l'une, appelée *précondition*, définit l'état des variables avant l'exécution de A ,
- l'autre, appelée *postcondition*, définit l'état des variables après l'exécution de A .

La méthode de *vérification déductive* consiste à produire une *démonstration* (ou *preuve*) que l'algorithme A effectue la tâche T , pour toutes les données qui satisfont sa précondition. Lorsque cette preuve est faite, le concepteur a la garantie que l'algorithme qu'il propose exécute correctement la tâche T . Par analogie avec un contrat commercial entre un client et un fournisseur, le couple (précondition, postcondition) de la tâche est appelé "contrat"¹, signé entre un utilisateur, qui s'engage à fournir des données respectant la précondition et demande un algorithme qui respecte la postcondition, et un concepteur d'algorithmes, qui, symétriquement, exige cette précondition et s'engage à ce que son algorithme respecte cette postcondition.

1. Voir https://fr.wikipedia.org/wiki/Programmation_par_contrat.

1.5 Procédure de décision

Un *problème de décision* est une question qui admet une réponse par “oui” ou par “non”. Une *procédure de décision* est un algorithme qui termine toujours et établit toujours la réponse correcte, pour tous les problèmes de décision d’une même famille. Nous ne considérons ici que des procédures de décision de la validité ou de la satisfaisabilité des formules d’un fragment d’une logique formelle.

1.6 Langages et outils

Des environnements de vérification actuels, comme Frama-C² et KeY³, implémentent la méthode de vérification déductive pour des langages de programmation très répandus, respectivement C et Java. Ils proposent d’utiliser la preuve de programmes pour aider à concevoir des programmes corrects. Dans ce but, ces langages de programmation ont été complétés par des langages de spécification formelle, comme ACSL (ANSI/ISO C Specification Language⁴) pour le langage C, JML (Java Modeling Language⁵) pour le langage Java et Spec#⁶ pour le langage C#.

Néanmoins, ces langages de programmation et de spécification répandus et ces outils puissants pour leur vérification déductive incluent un grand nombre de concepts et de subtilités qui complexifient significativement la compréhension des principes de la vérification. **C’est pourquoi ce cours d’initiation à la preuve de programmes outillée s’appuie sur un environnement de vérification conçu pour faciliter la preuve de programmes, nommé Why3, et sur un fragment limité de son langage de programmation et de spécification, nommé WhyML.**

1.7 Systèmes cyber-physiques

La fin du cours étend la vérification du logiciel avec un exemple de méthode et d’outil de vérification de systèmes dits “cyber-physiques”⁷ dont une partie seulement est réalisée par du logiciel. Par exemple, nous pourrions décrire des systèmes de manutention robotisée, ou de commande d’essuyage et de dégivrage de voiture, qui sont partiellement réalisés par du logiciel (commande du bras manipulateur, commande du moteur d’essuie-glace) et partiellement par du matériel (manette de commande des essuie-glaces, moteur, tapis roulants, etc). Nous pourrions décrire ces systèmes à condition qu’ils puissent être modélisés par des systèmes à événements discrets. Un **système à événements discrets** (SED) effectue des pas élémentaires d’évolution, pendant lesquels le système ne perçoit pas son environnement, il est totalement sourd aux changements de son environnement. Les changements qui surviennent pendant un pas élémentaire ne

2. <http://frama-c.com/>.

3. <http://www.key-project.org/>.

4. Voir <http://frama-c.com/acsl.html>.

5. Voir <http://www.cs.iastate.edu/leavens/JML>.

6. Voir <http://research.microsoft.com/specsharp/>.

7. https://en.wikipedia.org/wiki/Cyber-physical_system.

sont pris en compte qu'au début du pas suivant. Par exemple, les microprocesseurs constituent un bon exemple de systèmes modélisables par des SED. Au niveau physique, l'étape élémentaire est l'exécution d'une instruction machine (Load, Mov, Add, etc). Sur le microprocesseur réel, les interruptions (frappe clavier, clic souris, etc) qui arrivent pendant l'exécution d'une instruction machine sont prises en compte avant l'instruction suivante. Par conséquent, modéliser un microprocesseur comme un SED dont les événements discrets sont des modélisations des instructions machine et des événements d'interruption est un modèle correct par rapport au fonctionnement réel des microprocesseurs. Ces systèmes sont différents des systèmes continus où les événements externes sont pris en compte à l'instant exact de leur arrivée. Un système physique continu peut néanmoins être modélisé par un système à événements discrets si la durée de n'importe quel événement est inférieure ou égale au délai qui sépare deux informations successives en provenance de l'environnement. En général, l'observation de l'environnement est échantillonnée. Si la période d'échantillonnage est de durée supérieure à la durée d'un pas élémentaire alors le système peut être modélisé par un système à événements discrets.

1.7.1 Vérifier selon la classe de systèmes

Pour être efficace, une méthode de vérification doit être implémentée dans des outils qui l'automatise. La conception de ces outils pose immédiatement des problèmes de complexité algorithmique et d'indécidabilité. Pour contourner ces problèmes, il faut réduire les ambitions et effectuer quelques compromis judicieux, afin que les outils envisagés soient réalisables et applicables. Le compromis consiste, en général, à limiter l'expressivité des langages de modélisation afin d'être dans un cadre décidable et de complexité algorithmique acceptable. Ainsi les langages de modélisation et les outils associés sont adaptés à certaines classes de problèmes et pas à d'autres.

De ce point de vue, le principal critère de classification est le nombre d'états du système. Nous présentons dans la partie 1.7.2 une classification qui distingue les systèmes à nombre fini d'états de ceux qui en ont un nombre infini. Puis, dans la partie 1.7.3, nous distinguons les systèmes transformationnels des systèmes réactifs. Enfin dans la partie 1.7.4, nous présentons deux vues possibles d'un système réactif : la vue ouverte et la vue fermée.

1.7.2 Systèmes finis et infinis

Pour les systèmes à nombre fini d'états, la question de la vérification est décidable puisque réalisable par parcours du graphe des états, représentable en mémoire, puisque de taille finie.

Pour les systèmes à nombre infini d'états, la question de la vérification est, en général, indécidable. Les outils traitant des systèmes infinis doivent manipuler des représentations symboliques d'ensembles infinis d'états et certains algorithmes peuvent boucler indéfiniment.

Même pour un système à nombre d'états fini, on n'est pas assuré que les méthodes algorithmiques de vérification soient applicables. Leur complexité peut les rendre inapplicables en pratique, sur des systèmes ayant un grand nombre d'états.

Certains systèmes à nombre infini d'états admettent une représentation finie, ce qui permet de ramener le problème de leur vérification à celle d'un système à nombre fini d'états. Sinon, on

représente symboliquement des ensembles infinis d'état par des formules logiques, par exemple dans la logique des prédicats du premier ordre, et on utilise des outils de preuve pour raisonner sur ces formules. Ces outils de preuve nécessitent parfois l'intervention d'un opérateur humain, pour guider la preuve sur les questions indécidables.

Cette distinction permet de séparer deux classes de problèmes, ceux qui sont vérifiables automatiquement par exploration (appelée *model-checking* en anglais) et ceux qui nécessitent des techniques de preuve.

1.7.3 Systèmes transformationnels et systèmes réactifs

Manna et Pnueli [MP92, MP95] classent les systèmes en deux catégories, schématisées dans la figure 1.1 :

- Les systèmes transformationnels retournent des résultats à partir de données. Un système transformationnel est un SED composé d'une seule action élémentaire.
- Les systèmes réactifs interagissent constamment avec leur environnement en recevant des stimuli et en retournant des commandes. Un système réactif est un système qui évolue dans un environnement dont il doit prendre en compte les évolutions pendant son fonctionnement.

Données $\rightarrow$ Système transformationnel $\rightarrow$ Résultats

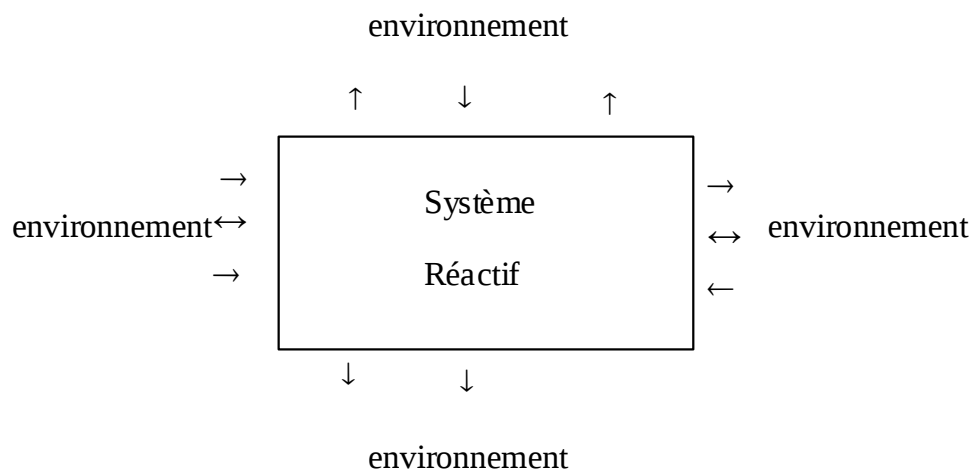


FIGURE 1.1 – Systèmes transformationnels et réactifs

Par exemple, un compilateur, un solveur de systèmes d'équations linéaires, ou une simulation de crash automobile sont des systèmes transformationnels. Un système d'exploitation, une IHM, un protocole de communication, une commande de procédés industriels, le pilotage de système de commande d'avion, une application de retrait bancaire par carte sont des systèmes réactifs.

Les principales différences sont les suivantes : Dans un système transformationnel, il y a beaucoup de calculs et peu de contrôle des échanges d'informations. Dans un système réactif, il y a peu de calculs et beaucoup de contrôle des interactions.

Les principales propriétés à assurer sont différentes : Pour un système transformationnel, on veut surtout vérifier l'exactitude des résultats fournis et la terminaison du calcul. Pour un système réactif, on veut vérifier la correction des comportements (toute commande soumise à un système d'exploitation doit être fatalement exécutée, l'échange des messages doit être en alternat, etc).

La conséquence est que la spécification est de nature très différente : Un système transformationnel a une spécification "boîte noire", composée d'*précondition* et une *postcondition* décrivant les états initiaux et finaux du système. On ignore totalement le comportement interne du système. On ignore toutes les évolutions de l'environnement après l'entrée des données. Elles existent dans le monde réel, mais n'ont pas d'impact sur le système. Elles sont donc ignorées dans le modèle. Un système réactif a une spécification "boîte de verre" des actions internes du système, et de son environnement s'il est spécifié de manière fermée (partie suivante). On spécifie une collection d'actions et des propriétés sur leurs enchaînements.

1.7.4 Systèmes ouverts et systèmes fermés

Il existe deux manières de spécifier un système réactif :

- le spécifier indépendamment de son environnement ; on parle de spécification de systèmes ouverts
 - le spécifier dans un environnement donné ; on parle de spécification de systèmes fermés ;
- En général les systèmes réactifs sont spécifiés comme des systèmes fermés.

Par exemple, pour développer le protocole T=1 pour un lecteur de cartes bancaire, la spécification décrit un système fermé en décrivant également le protocole côté carte. Dans le cas d'un ascenseur, fermer le système consiste à spécifier les usagers aux étages et dans la cabine. Dans le cas d'un robot de transport de pièces développé, bien que la spécification ait pour but de développer le logiciel de pilotage du bras manipulateur, celle-ci décrit un système fermé en décrivant le fonctionnement des dispositifs d'arrivée et d'évacuation de pièces qui seront réalisés par du hardware.

Spécifier un système fermé change le point de vue de la spécification dans la mesure où le système peut fonctionner seul. Les fonctions d'un système fermé sont appelées *actions* car elles se déclenchent spontanément. Dans un système ouvert on parle d'*opérations* car elles nécessitent d'être appelées depuis l'environnement.

L'intérêt de la fermeture d'un système est double : elle permet la vérification par model-checking puisqu'il n'y a plus besoin d'intervention externe pour faire fonctionner le système, et la spécification de l'environnement représente des hypothèses sur l'environnement dans lequel sera placé le système d'intérêt. Une conséquence est que les hypothèses sur l'environnement devront être satisfaites par l'environnement réel pour que les vérifications aient un sens vis-à-vis du système réel.

L'inconvénient de la fermeture est double : elle exige de spécifier l'environnement, alors que cette partie de la spécification sera totalement inutile pour l'implémentation, et on risque de mélanger les spécifications de l'environnement et du système.

1.8 Synthèse et compléments

L'activité traditionnelle de conception de programmes consiste, étant donné un problème, à proposer un programme et le tester pour s'assurer qu'il résout bien le problème sur les quelques cas testés. La méthode étudiée ici consiste, étant donné un problème, à le définir précisément par des spécifications, à proposer un programme solution, puis à vérifier que le programme proposé automatise les réponses au problème défini par les spécifications. Remarquons que cette méthode est applicable de manière incrémentale : un problème peut être décomposé en plusieurs sous-problèmes, chacun étant traité par cette méthode. Remarquons également que cette méthode doit être complétée par une phase de test, qui conserve son intérêt malgré la certitude que le fragment d'algorithme résout le problème défini par les spécifications, car on n'est jamais sûr que les spécifications expriment exactement le problème que l'utilisateur de départ a posé informellement.

Cette formation à la preuve de programmes contribue aussi à la formation en algorithmique et en développement de programmes. En effet, l'appropriation de la méthode permet sans aucun doute d'aborder la conception de programmes avec un autre point de vue. Cet autre point de vue résulte d'une plus grande maîtrise de la sémantique des divers fragments d'un programme. D'une part, le concepteur est amené à exprimer par des annotations synthétiques et précises ce que doit faire chaque fragment de programme (le "quoi ?"). D'autre part il doit justifier pourquoi le fragment de programme proposé, aussi compliqué soit-il, réalise bien la tâche exprimée par ses annotations. Il s'assure ainsi que le "quoi faire ?" décrit par les annotations et le "comment faire ?" décrit par le fragment de programme sont cohérents

1.9 Plan du cours

Pour comprendre la preuve de programmes, il faut déjà maîtriser quelques concepts de logique formelle. Dans ce but, les chapitres 2 et 3 offrent une initiation ou une mise à niveau complète sur la logique des propositions et de la logique des prédicats. Dès ces chapitres, l'outil Why3 et son langage WhyML peuvent être utilisés pour déclarer des formules logiques, en vérifier la syntaxe ou prouver leur validité.

Ensuite, le chapitre 4 introduit la notion de type et de typage de chaque donnée, permettant d'une part de clarifier la nature de chaque donnée et de chaque fonction de calcul sur ces données, et d'autre part de comprendre et d'utiliser une première forme de vérification, appelée "vérification de type". L'automaticité des preuves est accrue si les types de données et les opérations de base définies sur elles sont regroupés en petites "théories", avec des axiomes bien choisis. Ces notions sont définies dans le chapitre 5.

Les principes fondamentaux de la preuve formelle de programmes impératifs sont exposés dans le chapitre 6 sur la logique de Hoare. Enfin, la mise en œuvre de ces principes avec l'outil de vérification déductive Why3 est présentée dans le chapitre 7, sur un fragment de son langage de spécification et de programmation que nous utiliserons dans ce module.

Enfin, le chapitre 8 aborde la question de la spécification et de la preuve de programmes récursifs.

À l'exception du présent chapitre d'introduction et du chapitre 6 sur la logique de Hoare, tous les chapitres de cours incluent une pratique avec l'outil Why3.

Chapitre 2

Logique des propositions, avec Why3

2.1 Introduction

L'objet de la logique est d'analyser le raisonnement humain. On n'étudie pas ici la logique en tant que discipline philosophique, mais en tant que formalisation du raisonnement. Cette formalisation a de nombreux buts, mais on s'intéresse surtout à deux d'entre eux. Le premier but est de disposer d'un langage commun pour définir toutes les théories scientifiques. C'est le but de la *logique mathématique*, qui tente d'unifier les raisonnements utilisés dans les sciences, et plus particulièrement en mathématiques. Le second but, qui nous intéresse encore davantage, est d'automatiser des raisonnements, à l'aide d'un ordinateur. On parle de *mécanisation de la logique*, et de *logique formelle*, lorsque les énoncés logiques sont exprimés dans un langage formel manipulé par certains logiciels.

2.1.1 Raisonnement automatique

Chacun sait que, si un ordinateur calcule vite et bien, il est incapable de produire de lui-même quelque raisonnement que ce soit. Mais si l'on parvient à décrire les raisonnements logiques comme des calculs, alors l'ordinateur peut devenir, de ce point de vue aussi, plus rapide et plus efficace que l'esprit humain. Nous étudions les principes et des outils du raisonnement automatique, et ses limites.

2.1.2 Pour une machine, rien n'est évident

Avant de chercher à programmer le raisonnement, il convient de savoir précisément ce qu'est un raisonnement. Il faut apprendre à disséquer nos démarches intellectuelles, et, avant tout, ne pas se laisser paralyser par l'évidence : a priori, rien n'est évident pour un ordinateur. Le plus difficile est peut-être de pourchasser, pour les expliciter clairement, tous les sous-entendus qui nous permettent de produire des raisonnements intelligibles. Finalement, il convient toujours d'adapter son discours au niveau de son auditoire : le niveau de l'ordinateur, de ce point de vue, peut être considéré comme égal à zéro.

2.1.3 La plateforme de vérification Why3

Why3 [BFM⁺] est une plateforme de vérification déductive conçue pour faciliter la programmation, la spécification et la preuve formelle des programmes, avec des prouveurs automatiques ou des assistants de preuve. Ces notions seront étudiées à partir du chapitre 6. Auparavant, nous utiliserons Why3 pour illustrer et assimiler la plupart des concepts des logiques formelles fondamentales présentées dans les chapitres 2 et 3.

Cette utilisation quasi-systématique d'un outil logiciel pour comprendre les concepts de logique est une originalité de ce cours, par rapport à la plupart des autres cours de logique, qui ne proposent que des exercices sur papier.

2.1.4 Utilisation élémentaire de Why3

Pour le sujet des chapitres 2 et 3, nous recommandons d'utiliser Why3 sans installation, à l'aide de l'interface en ligne accessible à l'adresse <http://www.why3.org/try/>.

Cette interface en ligne de Why3 inclut un éditeur de texte qui permet d'écrire votre code Why3. Pensez à sauvegarder votre code dans des fichiers, en cliquant sur l'icône qui représente une disquette. Ensuite, vous pourrez ouvrir ces fichiers, ou d'autres créés ou modifiés par n'importe quel éditeur de texte, en cliquant sur l'icône qui représente un dossier entrouvert.

Le suffixe des fichiers qui n'utilisent que la partie logique de Why3 peuvent avoir le suffixe `.why`, mais le suffixe `.mlw` est préférable et devra être utilisé dans toute la suite.

2.1.5 Contenu du chapitre

La partie 2.2 définit la “logique des propositions”, également appelée “logique propositionnelle”, “calcul des propositions” ou “calcul propositionnel”. C'est la logique la plus élémentaire. Elle sert de base pour définir beaucoup d'autres logiques plus expressives. Elle est présentée sous sa forme “classique”. Il est essentiel d'assimiler toutes les notions et tout le vocabulaire définis ici.

Ensuite, la partie 2.3 établit un lien fondamental entre les formules propositionnelles et les expressions booléennes, tout en conservant une distinction entre ces deux notions, importante pour la suite.

Enfin, la partie 2.5 montre comment utiliser Why3 pour manipuler ces notions de logique propositionnelle, et notamment vérifier automatiquement que des formules propositionnelles sont vraies.

2.2 Les fondements de la logique des propositions

Dans “Les métamorphoses du calcul”, G. Dowek écrit : *Qu'est-ce donc qu'un raisonnement ? Si l'on sait que tous les écureuils sont des rongeurs, que tous les rongeurs sont des mammifères, que tous les mammifères sont des vertébrés et que tous les vertébrés sont des animaux, on peut en déduire que tous les écureuils sont des animaux. [...]*

Ce raisonnement est simple à l'extrême, mais sa structure ne diffère pas fondamentalement de celle d'un raisonnement mathématique. Dans les deux cas, le raisonnement est formé d'une suite de propositions dans laquelle chacune découle logiquement des précédentes [...]. Dans ce cas, on applique la même règle trois fois. Cette règle permet, si l'on sait déjà que tous les Y sont des X et que tous les Z sont des Y , de déduire que tous les Z sont des X .

La partie 2.2.1 définit la notion de proposition. La partie 2.2.2 montre comment les propositions peuvent être connectées entre elles pour former des propositions plus complexes. La partie 2.2.3 explique comment ces propositions sont représentées syntaxiquement.

2.2.1 Les propositions

L'homme exprime son raisonnement par un discours, et ce discours utilise une langue (une langue naturelle, français, anglais, ...). D'une manière générale, ce discours est articulé en phrases, d'un niveau de complexité variable, et c'est l'étude de ces « énoncés » que la logique se propose de faire.

DÉFINITION 2.1 (PROPOSITION). *Parmi tous les énoncés possibles qui peuvent être formulés dans une langue, on distingue ceux auxquels il est possible d'attribuer une « valeur de vérité » : « vrai » ou « faux ». Ces énoncés porteront le nom de propositions.*

EXEMPLE 2.1. Ainsi, « Henri IV est mort assassiné en 1610 », « Napoléon Bonaparte a été guillotiné en 1852 » sont des propositions, puisqu'on peut leur attribuer une valeur de vérité (« vrai » pour la première, « faux » pour la seconde).

2.2.1.1 Des énoncés qui ne sont pas des propositions

Un énoncé « hypothétique » comme « Dans six mois, il y aura une exceptionnelle période de beau temps » est exclu du domaine des propositions et donc de notre étude. Il en est de même pour ce qu'on appelle les « paradoxes » de la logique comme, par exemple, celui du menteur. En effet, il est impossible d'attribuer une valeur de vérité à ces énoncés : nous les rejetons en dehors du cadre des propositions.

2.2.1.2 Principes fondateurs du calcul des propositions

Le calcul que l'on étudie considère toujours comme acquises les vérités suivantes, élevées au rang d'axiomes.

Principe de non-contradiction : Une proposition ne peut pas être simultanément vraie et fausse.

Principe du tiers-exclu : Toute proposition est soit vraie soit fausse (il n'y a pas d'autre possibilité).

2.2.1.3 D'autres logiques

Il existe d'autres logiques, fondées sur d'autres axiomes, qui admettent d'autres « valeurs de vérité » (par exemple le « possible » et le « nécessaire »), qui attribuent des « coefficients de vraisemblance » aux énoncés, etc. Ces logiques sortent du cadre de ce cours.

2.2.2 Les connecteurs logiques

L'analyse logique d'une phrase (reconnue comme proposition) fait apparaître des sous-phrases qui sont elles-mêmes des propositions. Ces « membres de phrases » sont reliés entre eux par des « connecteurs logiques ».

Par exemple, considérons l'énoncé : « J'ai obtenu une mauvaise note à cet examen parce que je n'ai pas assez travaillé ou parce que le cours est trop difficile ». Puisqu'il est possible d'attribuer une valeur de vérité à cet énoncé « global », c'est une proposition.

On peut alors mener ce qu'en grammaire française on appelle l'analyse logique de cette phrase, de manière à en extraire les propositions (au sens grammatical du terme) : « J'ai obtenu une mauvaise note à cet examen », « je n'ai pas assez travaillé », « le cours est trop difficile », qui sont appelées *propositions élémentaires*. Ces propositions, au sens grammatical du terme, sont reliées entre elles par « parce que » et par « ou », qui sont – grammaticalement parlant – des conjonctions (respectivement de subordination et de coordination). La conjonction « parce que » introduit habituellement un lien de « cause à effet ». La conjonction « ou » se contente de juxtaposer les propositions (au même niveau).

2.2.2.1 Vers une formalisation

Globalement, cette proposition exprime que « ma mauvaise note » est conséquence de l'une (au moins) des deux causes suivantes :

- « mon manque de travail »,
- « un cours trop difficile ».

Autrement posé (il s'agit d'un début de formalisation) :

(« mon manque de travail » ou « cours trop difficile ») entraîne « ma mauvaise note ».

REMARQUE 2.1. Il ne faut pas sous-estimer la difficulté de ce travail d'analyse :

- le langage courant est souvent imprécis ou ambigu,
- il faut souvent se livrer à une véritable interprétation pour parvenir à formaliser une phrase.

REMARQUE 2.2. L'analyse en logique des propositions s'arrête au niveau des connecteurs logiques (qui vont être présentés un par un dans la suite de ce chapitre). Elle ne permet pas de prendre en compte certaines nuances, la concordance des temps, ou d'autres liens qui peuvent exister entre des propositions.

D'une manière générale, le calcul propositionnel ne se préoccupe que des valeurs de vérité, et pas du tout des liens sémantiques qui peuvent exister entre des propositions. Ces dernières sont reliées entre elles syntaxiquement par des connecteurs comme « ou » ou « entraîne ».

Les connecteurs logiques sont des symboles qui permettent de produire des propositions « plus complexes » à partir d'autres propositions « plus simples ». Ils sont définis axiomatiquement par leur “table de vérité”.

2.2.2.2 Négation logique

À partir d'une proposition P , le connecteur $\neg$, qui se lit « non », permet de construire la nouvelle proposition (non P), notée $(\neg P)$, dont la valeur de vérité est définie en fonction de celle de P par la table de vérité suivante :

P	$\neg P$
F	V
V	F

2.2.2.3 Disjonction logique

À partir de deux propositions P et Q , le connecteur $\vee$, qui se lit « ou », permet de construire la nouvelle proposition (P ou Q), notée $(P \vee Q)$.

La proposition $P \vee Q$ est fausse si et seulement si les deux propositions P et Q sont fausses. Autrement dit, la valeur de vérité de $P \vee Q$ est définie en fonction de celles de P et de Q par la table de vérité suivante, où “F” se lit “faux” et “V” se lit “vrai”.

P	Q	$P \vee Q$
F	F	F
F	V	V
V	F	V
V	V	V

REMARQUE 2.3. Dans le langage courant, le mot « ou » est souvent employé de deux façons distinctes. Il est parfois utilisé avec le sens « les deux cas peuvent se produire » (comme ici) et parfois avec le sens « p ou q , mais pas les deux » (e.g. « il ira à Paris ou à Marseille »). Sauf indication contraire, le « ou » sera toujours employé avec cette première signification.

2.2.2.4 Conjonction logique

À partir de deux propositions P et Q , le connecteur $\wedge$, qui se lit « et », permet de construire la nouvelle proposition (P et Q), notée $(P \wedge Q)$, dont la valeur de vérité est définie en fonction de celles de P et de Q par la table de vérité suivante :

P	Q	$P \wedge Q$
F	F	F
F	V	F
V	F	F
V	V	V

Autrement dit, la proposition $P \wedge Q$ est vraie si et seulement si les deux propositions P et Q sont vraies.

2.2.2.5 Implication logique

À partir de deux propositions P et Q , le connecteur $\Rightarrow$, qui se lit « si ... alors ... », permet de construire la proposition (si P , alors Q), notée $(P \Rightarrow Q)$, dont la valeur de vérité est définie en fonction de celles de P et de Q par la table de vérité suivante :

P	Q	$P \Rightarrow Q$
F	F	V
F	V	V
V	F	F
V	V	V

Autrement dit, la proposition $P \Rightarrow Q$ est vraie si P est fausse ou si Q est vraie.

REMARQUE 2.4. Lorsque la proposition P est fausse, la proposition « si P alors Q » est vraie, quelle que soit la valeur de vérité de la proposition Q . Il s'agit simplement d'une proposition, qui forme un tout, et qui a la valeur de vérité « vrai » lorsque, notamment, P et Q ont toutes les deux la valeur de vérité « faux ».

Par exemple, la proposition : « Si le pôle Nord est l'endroit le plus chaud de la planète, alors les poules ont des dents » doit être considérée comme ayant la valeur de vérité « vrai ». Le connecteur logique $\Rightarrow$ ne s'occupe nullement (on l'a dit) des liens sémantiques qui peuvent exister entre la température qui règne au pôle Nord et la dentition des poules.

2.2.2.6 Équivalence logique

À partir de deux propositions P et Q , le connecteur $\Leftrightarrow$, qui se lit « si et seulement si », permet de construire la nouvelle proposition (P si et seulement si Q), notée $(P \Leftrightarrow Q)$, dont la valeur de vérité est donnée par la table de vérité suivante :

P	Q	$P \Leftrightarrow Q$
F	F	V
F	V	F
V	F	F
V	V	V

REMARQUE 2.5. Même remarque que pour l'implication logique : l'équivalence logique de deux propositions fausses est une proposition vraie.

2.2.2.7 Exercices corrigés

Exercice (corrigé) 2.2. En notant M et C les affirmations suivantes :

- M = « Jean est fort en Maths »,
- C = « Jean est fort en Chimie »,

représenter les affirmations qui suivent sous forme symbolique, à l'aide des lettres M et C et des connecteurs usuels.

1. « Jean est fort en Maths mais faible en Chimie »
2. « Jean n'est fort ni en Maths ni en Chimie »
3. « Jean est fort en Maths ou il est à la fois fort en Chimie et faible en Maths »
4. « Jean est fort en Maths s'il est fort en Chimie »
5. « Jean est fort en Chimie et en Maths ou il est fort en Chimie et faible en Maths »

Réponses :

1. $M \wedge (\neg C)$; 2. $(\neg M) \wedge (\neg C)$; 3. $M \vee (\neg M \wedge C)$; 4. $C \Rightarrow M$; 5. $(M \wedge C) \vee (\neg M \wedge C)$.

Exercice (corrigé) 2.3. Énoncer la négation des affirmations suivantes en évitant d'employer l'expression : « il est faux que »

1. « S'il pleut ou s'il fait froid je ne sors pas. »
2. « Le nombre 522 n'est pas divisible par 3 mais il est divisible par 7. »
3. « Ce quadrilatère n'est ni un rectangle ni un losange. »
4. « Si Paul ne va pas travailler ce matin il va perdre son emploi. »
5. « Tout nombre entier impair peut être divisible par 3 ou par 5 mais jamais par 2. »
6. « Tout triangle équilatéral a ses trois angles égaux à 60° . »

Réponses :

1. Il pleut ou il fait froid et pourtant, je sors.
2. Le nombre 522 est divisible par 3 ou il n'est pas divisible par 7.

3. *Ce quadrilatère est un rectangle ou un losange.*
4. *Paul n'ira pas travailler ce matin mais il ne perdra pas son emploi.*
5. *Il existe un nombre entier impair, qui n'est divisible ni par 3 ni par 5, ou qui est divisible par 2.*
6. *Il existe un triangle équilatéral dont (au moins) un angle n'est pas égal à 60° .*

2.2.3 Variables et formules propositionnelles

Comme le calcul propositionnel ne s'occupe que des valeurs de vérité, il est possible, dans une proposition, de remplacer chacune de ses propositions élémentaires par un symbole (en général, une lettre de l'alphabet), appelé *variable propositionnelle*, et d'étudier ensuite les valeurs de vérité de l'expression en fonction des valeurs de vérité de ces symboles.

Une variable propositionnelle est parfois appelée *proposition atomique*.

Par exemple, dans l'exercice 2.2, les lettres M et C sont des variables propositionnelles.

Nous pouvons à présent définir la notion de formule propositionnelle, de deux manières différentes (d'abord informelle, puis formelle).

DÉFINITION 2.2 (FORMULE PROPOSITIONNELLE, DÉFINITION INFORMELLE). *Toute expression obtenue en remplaçant dans une proposition les propositions les plus simples par des variables propositionnelles est appelée formule propositionnelle.*

Cette définition est dite « informelle » car elle donne de l'information sur ce qu'est une formule propositionnelle. Mais on peut aussi dire qu'elle est « in-formelle » parce qu'elle n'est pas formelle, contrairement à la seconde définition.

DÉFINITION 2.3 (FORMULE PROPOSITIONNELLE, DÉFINITION FORMELLE). *Les règles (de syntaxe) qui permettent de former des formules propositionnelles correctes sont les suivantes :*

1. *Toute variable propositionnelle est une formule propositionnelle.*
2. *Si F et G sont des formules propositionnelles, alors $(\neg F)$, $(F \vee G)$, $(F \wedge G)$, $(F \Rightarrow G)$ et $(F \Leftrightarrow G)$ sont des formules propositionnelles.*
3. *Il n'existe pas d'autre formule propositionnelle que celles qui sont formées selon les règles 1 et 2.*

2.2.4 Règles de priorité et parenthèses

Les conventions suivantes permettent d'éviter d'écrire certaines paires de parenthèses dans une formule.

DÉFINITION 2.4 (RÈGLES DE PRIORITÉ DES CONNECTEURS LOGIQUES). *Les conventions de priorité des connecteurs logiques sont les suivantes (par ordre de priorité décroissante) :*

- la négation,
- la conjonction,
- la disjonction,
- l'implication, et
- l'équivalence.

EXEMPLE 2.4. Ainsi, la formule $\neg A \wedge B \Rightarrow C \Leftrightarrow E \wedge F$ allège l'écriture de la formule complètement parenthésée $((\neg A) \wedge B) \Rightarrow C \Leftrightarrow (E \wedge F)$. De même pour la formule $A \vee B \wedge C$, qui abrège $A \vee (B \wedge C)$. Dans la formule $(A \vee B) \wedge C$, les parenthèses sont requises.

Lorsque plusieurs connecteurs de conjonction se suivent, comme dans $A \wedge B \wedge (\neg C) \wedge D$, les parenthèses ne sont pas requises. De même pour une succession de disjonctions, comme dans la formule $(\neg A) \vee B \vee (\neg C)$. La pertinence de ces omissions de parenthèses peut être justifiée à l'aide de l'interprétation booléenne de ces connecteurs (partie 2.3), qui les associe à des opérateurs booléens associatifs. Par contre, le connecteur d'implication n'est pas interprété par un opérateur booléen associatif. On convient que $A \Rightarrow B \Rightarrow C$ abrège $A \Rightarrow (B \Rightarrow C)$. Dans la formule $(A \Rightarrow B) \Rightarrow C$, les parenthèses sont obligatoires.

REMARQUE 2.6. En cas de doute, ne pas hésiter à mettre plus de parenthèses que strictement nécessaire selon ces règles.

Exercice (corrigé) 2.5. *Quelles sont les différentes formules propositionnelles qu'on peut obtenir en ajoutant des parenthèses indispensables dans $\neg p \vee q \wedge \neg r$? Déterminer la table de vérité de chacune des formules obtenues.*

Réponse : En ajoutant des parenthèses indispensables dans $\neg p \vee q \wedge \neg r$, on peut obtenir les formules propositionnelles suivantes :

1. $\neg p \vee (q \wedge \neg r)$
2. $(\neg p \vee q) \wedge \neg r$
3. $(\neg(p \vee q)) \wedge \neg r$
4. $\neg(p \vee (q \wedge \neg r))$
5. $\neg((p \vee q) \wedge \neg r)$

Tables de vérité :

p	q	r	1	2	3	4	5
V	V	V	F	F	F	F	V
V	V	F	V	V	F	F	F
V	F	V	F	F	F	F	V
V	F	F	F	F	F	F	F
F	V	V	V	F	F	V	V
F	V	F	V	V	F	F	F
F	F	V	V	F	F	V	V
F	F	F	V	V	V	V	V

Exercice (corrigé) 2.6. Construire les tables de vérité des formules propositionnelles suivantes :

1. $(\neg p) \wedge q$
2. $(\neg p) \Rightarrow (p \vee q)$
3. $\neg((\neg p) \wedge (\neg q))$
4. $(p \wedge q) \Rightarrow (\neg q)$
5. $(p \Rightarrow q) \vee (q \Rightarrow p)$
6. $(p \Rightarrow (\neg q)) \vee (q \Rightarrow (\neg p))$
7. $(p \vee (\neg q)) \wedge ((\neg p) \vee q)$
8. $p \Rightarrow ((\neg p) \Rightarrow p)$

Réponse :

p	q	1	2	3	4	5	6	7	8
V	V	F	V	V	F	V	F	V	V
V	F	F	V	V	V	V	V	F	V
F	V	V	V	V	V	V	V	F	V
F	F	F	F	F	V	V	V	V	V

2.3 Interprétation booléenne de la logique des propositions

Cette partie donne une définition précise pour la notion de « valeur de vérité » et un sens, une *sémantique*, à toute formule propositionnelle. Cette sémantique est appelée « interprétation booléenne de la logique propositionnelle ». Elle est fondée sur la notion d'algèbre de Boole et sur ses notations. Le lecteur qui ne connaît pas cette notion et ces notations doit d'abord les assimiler par l'étude de l'annexe A.

Puisqu'une formule propositionnelle représente un énoncé qui est soit vrai, soit faux, la signification ultime d'une formule propositionnelle est l'une des deux *valeurs de vérité*, « vrai » ou « faux ». On code respectivement ces deux valeurs par les constantes booléennes 1 et 0. Cependant, la valeur d'une formule propositionnelle dépend de celle de ses variables propositionnelles, elle varie *en fonction de* ces valeurs. Par conséquent, on définit sa sémantique en deux étapes. La première étape transforme la formule propositionnelle en une *expression booléenne*. La seconde étape considère cette expression booléenne comme une *fonction booléenne* des variables qu'elle contient. Pour l'instant, on ne formalise que la première étape, par la définition suivante.

DÉFINITION 2.5 (EXPRESSION BOOLÉENNE ASSOCIÉE À UNE FORMULE PROPOSITIONNELLE). Soit F une formule propositionnelle quelconque. L'expression booléenne associée à F , notée $\text{expr-bool}(F)$, est obtenue de la manière suivante :

1. Si P est une variable propositionnelle, alors $\text{expr-bool}(P) = P$.

2. Si G est une formule propositionnelle, alors

$$\text{expr-bool}(\neg G) = \overline{\text{expr-bool}(G)}.$$

3. Si F est de la forme $G \vee H$, où G et H sont des formules propositionnelles, alors

$$\text{expr-bool}(G \vee H) = \text{expr-bool}(G) + \text{expr-bool}(H).$$

4. Si F est de la forme $G \wedge H$, où G et H sont des formules propositionnelles, alors

$$\text{expr-bool}(G \wedge H) = \text{expr-bool}(G) \cdot \text{expr-bool}(H).$$

5. Si F est de la forme $G \Rightarrow H$, où G et H sont des formules propositionnelles, alors

$$\text{expr-bool}(G \Rightarrow H) = \overline{\text{expr-bool}(G)} + \text{expr-bool}(H).$$

6. Si F est de la forme $G \Leftrightarrow H$, où G et H sont des formules propositionnelles, alors

$$\text{expr-bool}(G \Leftrightarrow H) = \overline{\text{expr-bool}(G)} \cdot \overline{\text{expr-bool}(H)} + \text{expr-bool}(G) \cdot \text{expr-bool}(H).$$

EXEMPLE 2.7. Soit $F = A \vee \neg B \Leftrightarrow (B \Rightarrow C)$. On a alors :

$$\begin{aligned} & \text{expr-bool}(F) \\ = & \text{par la définition (2.5)} \\ & A + \overline{B} \cdot \overline{B} + C + (A + \overline{B}) \cdot (\overline{B} + C) \\ = & \text{par la première loi de De Morgan} \\ & \overline{A} \cdot B \cdot B \cdot \overline{C} + (A + \overline{B}) \cdot (\overline{B} + C) \\ = & \text{par commutativité de } + \\ & \overline{A} \cdot B \cdot B \cdot \overline{C} + (\overline{B} + A) \cdot (\overline{B} + C) \\ = & \text{par la deuxième loi de distributivité} \\ & \overline{A} \cdot B \cdot B \cdot \overline{C} + \overline{B} + A \cdot C \\ = & \text{par la règle d'itempotence, appliquée à } B \\ & \overline{A} \cdot B \cdot \overline{C} + \overline{B} + A \cdot C \\ = & \text{par la règle de double négation} \\ & \overline{A} \cdot \overline{\overline{B}} \cdot \overline{C} + \overline{B} + A \cdot C \\ = & \text{par la 3e règle de redondance } a + \overline{a} \cdot b = a + b \text{ en remplaçant } a \text{ par } \overline{B} \text{ et } b \text{ par } \overline{A} \cdot \overline{C} \\ & \overline{A} \cdot \overline{C} + \overline{B} + A \cdot C. \end{aligned}$$

REMARQUE 2.7. Les connecteurs logiques de négation, de disjonction et de conjonction sont simplement respectivement transformés en négation booléenne, somme booléenne et produit booléen. En revanche, il n'y a pas d'opération dans les algèbres de Boole qui corresponde aux connecteurs d'implication et d'équivalence logique. C'est pourquoi ils sont « éliminés » par la fonction *expr-bool*, qui les définit à partir des trois opérations des algèbres de Boole.

2.3.1 Fonctions de vérité

Cette partie présente la seconde étape de la définition de la sémantique booléenne de la logique des propositions. Elle est fondée sur la remarque suivante :

REMARQUE 2.8. La valeur de vérité d'une formule propositionnelle est une fonction des valeurs de vérité des variables propositionnelles qui y interviennent.

DÉFINITION 2.6 (FONCTION DE VÉRITÉ). Soit F une formule propositionnelle, dont les variables propositionnelles sont $P_1, P_2, P_3, \dots, P_n$ ($n \geq 0$). La fonction de vérité de F , notée φ_F , est la fonction booléenne qui associe aux n variables propositionnelles $P_1, P_2, P_3, \dots, P_n$ de F la valeur de l'expression booléenne $\text{expr-bool}(F)$.

On peut donner une par une toutes les valeurs d'une fonction de vérité en construisant sa « table de vérité ». Des exemples de tables de vérité ont déjà été donnés dans les parties 2.2.2 et 2.2.3.

2.3.2 Conséquences logiques

Soit $\mathcal{H} = \{H_1, \dots, H_n\}$ un ensemble de formules propositionnelles.

DÉFINITION 2.7 (CONSÉQUENCE LOGIQUE). On dit que la formule propositionnelle C est une conséquence logique des formules propositionnelles $H_1, \dots, H_n$ de l'ensemble $\mathcal{H} = \{H_1, \dots, H_n\}$ lorsque, chaque fois que les fonctions de vérité $\varphi_{H_1}, \dots, \varphi_{H_n}$ prennent simultanément la valeur « vrai » (ou 1), il en est de même pour la fonction de vérité de la formule C .

NOTATION : Cette relation est notée $\mathcal{H} \models C$, qui se lit : « C est conséquence logique de $\mathcal{H}$ ».

EXEMPLE 2.8. $\{P \Rightarrow Q, P\} \models Q$. En effet :

P	Q	$P \Rightarrow Q$
F	F	V
F	V	V
V	F	F
V	V	V

Il n'y a qu'un seul cas dans lequel $P \Rightarrow Q$ et P sont simultanément vrais. Dans ce cas, Q est vrai.

2.3.3 Tautologies

DÉFINITION 2.8 (TAUTOLOGIE). Une tautologie est une formule propositionnelle dont la fonction de vérité est la fonction booléenne constante qui vaut 1 quelle que soit la valeur de vérité de ses paramètres.

NOTATION : La notation utilisée pour signaler une tautologie F est $\models F$, qui se lit « F est une tautologie ».

EXEMPLE 2.9. Soit $F = A \Rightarrow A$. Comme $\text{expr-bool}(F) = \overline{A} + A = 1$, on a $\models F$.

EXEMPLE 2.10. Montrons que $F = (A \Rightarrow C) \Rightarrow ((B \Rightarrow C) \Rightarrow (A \vee B \Rightarrow C))$ est une tautologie.

$$\begin{aligned}
 & \text{expr-bool}((A \Rightarrow C) \Rightarrow ((B \Rightarrow C) \Rightarrow (A \vee B \Rightarrow C))) \\
 = & \quad \text{par la définition (2.5)} \\
 & \overline{\text{expr-bool}(A \Rightarrow C)} + \text{expr-bool}(B \Rightarrow C) + \text{expr-bool}(A \vee B \Rightarrow C) \\
 = & \quad \text{par la définition (2.5)} \\
 & \overline{\overline{A} + C} + \overline{\overline{B} + C} + \overline{A + B} + C \\
 = & \quad \text{par les lois de De Morgan et la règle d'involution} \\
 & \overline{A}C + \overline{B}C + \overline{A + B} + C \\
 = & \quad \text{par distributivité} \\
 & (A + B)\overline{C} + \overline{A + B} + C \\
 = & \quad \text{par commutativité} \\
 & \overline{A + B} + C + \overline{C}(A + B) \\
 = & \quad \text{par la 3e règle de redondance } a + \overline{a} \cdot b = a + b \text{ en remplaçant } a \text{ par } C \text{ et } b \text{ par } A + B \\
 & \overline{A + B} + C + A + B \\
 = & \quad \text{par commutativité} \\
 & \overline{A + B} + A + B + C \\
 = & \quad \text{par la règle de partition} \\
 & 1 + C \\
 = & \quad \text{par la règle d'absorption} \\
 & 1
 \end{aligned}$$

donc $\models F$.

Exercice 2.11. Les formules propositionnelles suivantes sont-elles des tautologies ?

1. $(p \wedge q) \Rightarrow p$
2. $(p \vee q) \Rightarrow (p \wedge q)$
3. $(p \wedge q) \Rightarrow (p \vee q)$
4. $p \Rightarrow (p \vee q)$

5. $p \Rightarrow ((\neg p) \Rightarrow p)$
6. $p \Rightarrow (p \Rightarrow q)$
7. $p \Rightarrow (p \Rightarrow p)$
8. $(p \Rightarrow q) \Rightarrow ((q \Rightarrow r) \Rightarrow (p \Rightarrow r))$

2.3.4 Théorème de substitution

PROPRIÉTÉ 2.1 (THÉORÈME DE SUBSTITUTION) : Soit F une formule propositionnelle dans laquelle interviennent les variables propositionnelles $P_1, P_2, P_3, \dots, P_n$. Supposons que l'on remplace ces variables par des formules propositionnelles $G_1, G_2, G_3, \dots, G_n$. La nouvelle formule propositionnelle obtenue est notée F^* .

Dans ces conditions, si $\models F$, alors $\models F^*$.

PREUVE : F étant une tautologie, sa fonction de vérité ne dépend pas des valeurs de vérité des variables booléennes, qui peuvent donc être remplacées par n'importe quelle fonction booléenne. ■

Attention, la réciproque n'est pas vraie ...

EXEMPLE 2.12. Soit $F = A \Rightarrow B$ et $F^* = P \wedge \neg P \Rightarrow Q$, obtenue à partir de F en remplaçant A par $P \wedge \neg P$ et B par Q . Comme $\text{expr-bool}(F^*) = \overline{P \cdot \overline{P}} + Q = \overline{0} + Q = 1 + Q = 1$, alors F^* est une tautologie. Cependant de $\text{expr-bool}(F) = \overline{A} + B$, on ne peut pas dire que F est une tautologie.

Voici un exemple d'utilisation de ce théorème :

EXEMPLE 2.13. La formule propositionnelle

$$G = ((P \Rightarrow Q \wedge \neg R) \vee (\neg S \Leftrightarrow T)) \Rightarrow ((P \Rightarrow Q \wedge \neg R) \vee (\neg S \Leftrightarrow T))$$

est compliquée puisqu'elle contient 5 variables propositionnelles. Il y a donc 32 lignes à calculer dans sa table de vérité. Cependant, il suffit de remarquer que G est obtenue par substitution à partir de $F = A \Rightarrow A$, en remplaçant A par $((P \Rightarrow Q \wedge \neg R) \vee (\neg S \Leftrightarrow T))$. Puisque F est une tautologie, F^* en est une aussi.

Ce résultat peut évidemment être appliqué aussi à des parties de formules propositionnelles, pour accélérer le calcul de leurs fonctions de vérité : si une partie d'une formule propositionnelle constitue à elle seule une tautologie, la partie correspondante de la fonction de vérité peut être avantageusement remplacée par 1.

2.4 Solveurs de satisfaisabilité

Le calcul sur les fonctions de vérité paraît satisfaisant, pour calculer des valeurs de vérité ou examiner des conséquences logiques. Il est vrai qu'il est simple, qu'il nécessite un minimum de réflexion (très important dans le cas des ordinateurs !) et qu'il est très facile à programmer.

Cependant, pour une formule propositionnelle qui comporte 10 variables propositionnelles (ce qui n'est pas beaucoup pour les problèmes que l'on cherche à résoudre !), la table (des valeurs de sa fonction) de vérité comporte $2^{10} = 1024$ lignes. Celui qui opère à la main a déjà démissionné. L'ordinateur démissionne un peu plus loin, certes, mais il finit aussi par avouer son incapacité :

- Sur les machines modernes, il n'est plus impossible d'envisager d'écrire et d'exécuter une « boucle » qui porte sur toutes les valeurs entières représentables sur 32 bits, donc de 0 à $2^{32} - 1$. Le temps d'exécution est récemment devenu raisonnable.
- Il ne faut cependant pas exiger que ce temps demeure raisonnable dès qu'il s'agit d'exécuter un algorithme un peu compliqué. Et 32 variables constituent un nombre ridiculement petit pour un système expert, dans lequel les expressions offrent souvent une complexité qui n'a aucune commune mesure avec ce que l'on peut imaginer de plus compliqué ...

Les « raccourcis » qui viennent d'être étudiés et qui permettent d'accélérer, voire de supprimer totalement, le calcul d'une fonction de vérité, sont plus utiles lorsqu'on opère « à la main » que pour la programmation d'algorithmes de logique.

Il faut donc garder en réserve la méthode des fonctions de vérité : elle peut être très utile dans certains cas, essentiellement lorsque le problème peut être résolu « à la main », mais il faut trouver une autre méthode pour songer à aborder des problèmes plus complexes.

Cette partie présente l'approche d'automatisation du raisonnement propositionnel actuellement reconnue comme étant la plus efficace, appelée « SAT solving » en anglais. Elle traite le problème de satisfaisabilité (booléenne), défini comme suit.

DÉFINITION 2.9 (SATISFAISABILITÉ). *Une formule propositionnelle est satisfaisable (satisfiable en anglais) si elle est vraie pour au moins une des valeurs de vérité de ses variables propositionnelles. Le problème de satisfaisabilité (booléenne/propositionnelle) est le problème de décider, en répondant par “oui” ou par “non”, si une formule propositionnelle est satisfaisable ou pas.*

Ce problème est fondamental en informatique théorique et a des applications pratiques dans de nombreux domaines, dont la vérification formelle de programmes, la synthèse de circuits électroniques, l'optimisation de contraintes ou la résolution de problèmes combinatoires.

Il est facile de se convaincre qu'une formule propositionnelle F est satisfaisable si et seulement si sa négation $\neg F$ est une *antilogie* (une formule toujours fausse) et qu'une formule propositionnelle F est non-satisfaisable si et seulement si sa négation $\neg F$ est une tautologie. Par conséquent, savoir résoudre (efficacement) le problème de satisfaisabilité permet aussi de décider si une formule est une tautologie, ou si c'est une antilogie, etc.

Le problème de satisfaisabilité propositionnelle est évidemment *décidable* (c'est-à-dire qu'il existe des algorithmes et des programmes capables de le résoudre quel que soit la formule propo-

sitionnelle donnée), puisque la construction de la table de vérité d’une formule propositionnelle peut s’effectuer automatiquement en un temps fini. Cependant, cette méthode est très inefficace et il existe des méthodes beaucoup plus puissantes, implémentées dans des logiciels appelés “solveurs SAT” (*SAT solvers*, en anglais).

Ces outils sont appelés “solveurs” au lieu de “prouveurs” car ils ne se contentent pas de répondre “oui” ou “non” à la question de satisfaisabilité. Si la formule est satisfaisable, ils en fournissent de plus un *modèle*, c’est-à-dire une valeur de vérité pour chacune de ses variables propositionnelles qui rend la formule vraie. Ce modèle est un “témoin” de satisfaisabilité.

Les solveurs SAT solving implémentent des algorithmes d’exploration de l’espace des affectations des variables booléennes de la formule, par exemple les *algorithmes de type DPLL* (Davis-Putnam-Logemann-Loveland) qui cherchent des solutions par division de l’espace en branches et élimination de branches inutiles, les *techniques de propagation de contraintes* qui éliminent des variables et simplifient la formule, et diverses heuristiques astucieuses.

Plusieurs solveurs SAT performants sont disponibles :

- **Glucose** : solveur basé sur l’algorithme CDCL (*Conflict-Driven Clause Learning*)
- **MiniSAT** : solveur *open-source* très utilisé comme base pour d’autres outils
- **Z3** : solveur SMT (*Satisfiability Modulo Theories*) qui inclut le SAT solving
- **CVC5** : autre solveur SMT avec des capacités avancées
- **Alt-Ergo** : solveur spécialisé pour la vérification de programmes

Dans ce cours, nous utiliserons Why3 pour résoudre des problèmes de logique propositionnelle, car cette plateforme offre un langage d’entrée unique facile à apprendre qu’elle traduit automatiquement dans le langage d’entrée des solveurs SMT comme Alt-Ergo, CVC5 et Z3. De plus l’interface en ligne de Why3 donne accès au solveur Alt-Ergo sans nécessiter aucune installation.

2.5 La logique propositionnelle avec Why3

Les connecteurs propositionnels de la négation ($\neg$), du “ou” ($\vee$), du “et” ($\wedge$), de l’implication ($\Rightarrow$) et de l’équivalence ($\Leftrightarrow$) se notent respectivement `not`, `\ /`, `/ \`, `->` et `<->` en Why3. Dans nos listings, ils apparaissent respectivement sous la forme plus lisible **not**, **$\vee$** , **$\wedge$** , **$\rightarrow$** et **$\leftrightarrow$** .

L’exercice suivant, guidé et commenté, vous permet de découvrir la syntaxe de la logique propositionnelle en Why3, mais aussi l’utilisation de l’interface en ligne de Why3.

Exercice 2.14 (Logique propositionnelle avec Why3). *Suivez pas à pas les étapes suivantes :*

1. Recopier le code du listing 2.1 dans l’interface en ligne de Why3.

*Après avoir déclaré deux variables propositionnelles **p** et **q**, ce code déclare un lemme nommé **p11**.*

2. Demander à Why3 de démontrer cette formule propositionnelle, en tapant **Alt+R** ou en cliquant dans l’interface en ligne sur l’icône  qui représente une baguette magique. Observer et interpréter le résultat dans l’onglet “Task list”.

La démonstration est automatique. Elle réussit si et seulement si la formule est une tautologie (une formule toujours vraie).

```

predicate p
predicate q

lemma p11: p ∧ q → q ∧ p

```

Listing 2.1 – Logique propositionnelle avec Why3.

3. Sauvegarder ce code dans un fichier nommé `lp1.mlw`. Vérifier le contenu de ce fichier en l'ouvrant avec votre éditeur de texte favori.
4. Pour chacun des connecteurs propositionnels, ajouter au code une tautologie contenant ce symbole. Démontrer automatiquement chacun de ces nouveaux lemmes. Voir une solution dans le listing 2.2. Le lemme `p11` du listing 2.1 est déjà une tautologie contenant les symboles $\wedge$ et $\rightarrow$.

```

predicate p
predicate q

lemma exNegImp: not (not p) → p
lemma exOrImp: p → p ∨ q
lemma exEquiv: p ↔ p

```

Listing 2.2 – Exemples de tautologies propositionnelles.

La logique propositionnelle de Why3 contient aussi les formules `true` et `false` pour “vrai” et “faux”, dont quelques propriétés sont énoncées dans le listing 2.3.

```

lemma 11: true ↔ true
lemma 12: not true ↔ false
lemma 13: false ∨ true ↔ true
lemma 14: true ∧ false ↔ false

predicate p
lemma 15: p ∧ true ↔ p

```

Listing 2.3 – Exemples de tautologies avec “vrai” et “faux”.

Chapitre 3

Logique du premier ordre, avec Why3

Ce chapitre décrit la logique du premier ordre et son implémentation dans l'outil Why3, qui automatise certaines démonstrations de validité de formules de cette logique. Cet outil permet de raisonner sur diverses théories utiles en programmation. Nous utiliserons plus tard des formules de cette logique pour spécifier des propriétés de programmes Why3.

3.1 Introduction

3.1.1 Insuffisances de la formalisation en logique propositionnelle

La logique des propositions ne s'occupe, on le sait, que des liens entre propositions réalisés à l'aide des connecteurs logiques. Autrement dit, une fois qu'une proposition « complexe » a été décomposée selon ces connecteurs, il subsiste des « atomes » (non sécables en logique des propositions) et l'analyse ne peut pas être poussée plus profondément, alors peut encore exister des « relations » dans ces atomes ou entre ces atomes, relations que la logique des propositions ne permet pas de prendre en compte.

Par exemple, la proposition « Pierre est le père de Marc » ne comporte aucun connecteur logique, elle n'est plus décomposable en logique des propositions. On ne peut la formaliser que par une variable propositionnelle, par exemple A . Alors que la proposition « Jean est le père de Sylvie » entretient des rapports évidents avec la précédente, elle ne peut être formalisée en logique des propositions, elle aussi, que par une (autre) variable propositionnelle, par exemple B . Après cette formalisation, on se retrouve avec deux variables propositionnelles A et B sans aucun lien entre elles, alors qu'il est évident que ces deux propositions évoquent un même lien de parenté (la paternité), simplement entre des individus différents.

3.1.2 Notion de prédicat

Ce premier exemple suggère la nécessité, pour poursuivre l'analyse des propositions, de la création d'un langage qui permettrait de décrire des propriétés accordées (ou non) à des individus, ou les reliant. Cette préoccupation est aussi celle des mathématiciens, qui s'occupent, par

exemple, de décrire les propriétés des entiers naturels, des opérations sur ces entiers (comme l'addition) et des relations entre ces entiers (comme l'égalité et les comparaisons).

D'une manière générale, et avant de donner une définition précise de ces concepts, on dira qu'une « propriété » possédée (ou non) par des objets est un « prédicat ». Ces prédicats porteront sur des « variables », prenant leurs valeurs dans un certain domaine. Pour certaines « valeurs » de ces variables, certaines propriétés seront « vraies » ou « fausses ».

3.1.3 Notion d'univers du discours

Soit la proposition « 7 n'a pas de racine carrée ». En logique propositionnelle, elle ne peut être formalisée que par une variable propositionnelle. De plus, il semble même difficile de lui accorder une « valeur de vérité » ; en effet, si l'on se place dans l'ensemble des entiers naturels, cette proposition est « vraie », alors que si l'on se place dans l'ensemble des nombres réels, elle est évidemment fausse.

Dans cet exemple, comme dans celui de la partie précédente, la proposition évoque une propriété, accordée ou refusée, à un objet bien précis (ici, le nombre 7). Bien entendu, la valeur de vérité dépend de la valeur de cet objet, mais aussi de l'ensemble dans lequel ces valeurs peuvent être choisies.

On pourra envisager de formaliser une telle proposition par une expression de la forme « x a une racine carrée », dans laquelle la « variable » x peut être remplacée par un nombre particulier. La valeur de vérité de la proposition obtenue en donnant ainsi une valeur à x dépend non seulement de cette valeur, mais aussi de l'ensemble dans lequel cette valeur peut être prise.

Cet ensemble des valeurs reconnues comme possibles ($\mathbb{N}$ ou $\mathbb{R}$ dans notre exemple) est appelé « univers du discours ».

3.1.4 Notion de quantification

Considérons les propositions « Tous les étudiants sont sérieux » et « Certains étudiants sont sérieux ». La logique des propositions est incapable de faire apparaître le lien manifeste qui existe entre elles. Elle ne pourra les formaliser que par deux propositions distinctes A et B , parce qu'elles sont différentes et que l'une n'est pas la négation de l'autre.

La généralisation de la logique propositionnelle suggérée ici est encore plus grande que dans les exemples précédents, puisque la propriété évoquée (« être sérieux ») est accordée, par ces propositions, non pas seulement à un individu bien précis, mais à certains individus, considérés dans leur ensemble, ou à toute une catégorie d'individus, sans qu'aucun ne soit nommément désigné.

Ce troisième exemple suggère la notion de « quantification » d'une variable, permettant de formaliser des énoncés tels que « Tous les ... » ou « Certains ... ».

L'objet du « calcul des prédicats », également appelé « logique des prédicats » ou « logique du premier ordre », est de permettre de formaliser les notions qui viennent d'être brièvement évoquées, et de raisonner symboliquement sur les formules obtenues, par des manipulations syntaxiques.

3.1.5 Plan du chapitre

Les formules du calcul des prédicats, définies dans la partie 3.3, sont construites à partir de « termes », qui sont définis auparavant, dans la partie 3.2. Les significations possibles d'une formule, qui constituent leur « sémantique », sont explicitées dans la partie 3.4. Quelques transformations formelles classiques de ces formules sont présentées dans la partie 3.5. Grâce à la notion théorique de décidabilité, la partie 3.6 explique pourquoi il est vain d'espérer qu'une machine puisse vérifier n'importe quelle formule du premier ordre. Enfin, la logique du premier ordre dans Why3 est présentée dans la partie 3.7.

3.2 Termes

Dans la logique des prédicats, les objets sur lesquels portent les propriétés formalisées par des prédicats sont appelés « termes ». On considère qu'on peut décrire tous les termes à partir d'objets élémentaires et d'une seule règle de construction, qui consiste à appliquer un « symbole fonctionnel » à un certain nombre de termes, pour former un terme plus gros.

Nous définissons d'abord cette notion de symbole fonctionnel, avant de définir deux ensembles de termes : les termes sans variables, puis plus généralement les termes qui peuvent contenir des variables.

3.2.1 Symboles fonctionnels, signature fonctionnelle

Puisqu'un terme représente une expression, ses constructeurs sont souvent des symboles d'opérations. On les appelle *symboles fonctionnels* et on les écrit comme des fonctions, c'est-à-dire avant leurs paramètres, qui sont encadrés de parenthèses et séparés par des virgules.

On désigne par F un ensemble fini de *symboles fonctionnels* (ou opérateurs). Cet ensemble est parfois appelé *signature fonctionnelle*.

On associe à chaque symbole fonctionnel le nombre de ses opérandes. Ce nombre est appelé *arité* du symbole fonctionnel. Formellement, l'arité est une application a de F dans l'ensemble $\mathbb{N}$ des entiers naturels, ce que l'on note par $a : F \rightarrow \mathbb{N}$.

Pour tout entier naturel n , on désigne par F_n l'ensemble des symboles fonctionnels de F d'arité n . Formellement, on a $F_n = \{f \in F. a(f) = n\}$. Les symboles d'arité 0, éléments de F_0 , sont appelés *constantes*.

EXEMPLE 3.1. On peut avoir

$$F =_{\text{def}} \{+, -, *, /, \text{Moins}, \text{Abs}, 2, 3, 4\}$$

avec $F_0 =_{\text{def}} \{2, 3, 4\}$, $F_1 =_{\text{def}} \{\text{Abs}, \text{Moins}\}$, $F_2 =_{\text{def}} \{+, -, *, /\}$ et $F_i = \emptyset$ pour $i \geq 3$.

Convention de nommage : Les symboles fonctionnels qui ne sont pas des constantes sont souvent notés

$$f, g, h, k, f_1, g_1, h_1, k_1, \dots$$

et les constantes

$$a, b, c, d, a_1, a_2, d_1, \dots$$

Exercice 3.2. Décrire l'ensemble des symboles fonctionnels employés dans chacune des expressions suivantes :

1. $*(+(2, 3), 5), +(4, \text{Moins}(3)), 2)$
2. $f(g(a, h(c)), g(f(a, b, c), h(a)), a)$

3.2.2 Termes clos

L'ensemble des *termes clos* de signature fonctionnelle F , noté T_F , est le plus petit ensemble qui contient le terme $f(t_1, \dots, t_n)$ pour tout entier naturel n , tout symbole fonctionnel f de F_n et tous les termes $t_1, \dots, t_n$ dans cet ensemble T_F .

Cette définition peut sembler bizarre, car l'ensemble T_F est défini à partir de lui-même, mais on l'admet temporairement et on y reviendra dans un chapitre ultérieur sur les types inductifs.

Remarque importante : Dans le cas où $n = 0$, il n'y a pas de $t_1, \dots, t_n$ et la définition dit que T_F contient toutes les constantes de F . Si le symbole f est d'arité 0, alors le terme $f()$ est plus simplement noté f .

Exercice 3.3. Justifier que les expressions suivantes appartiennent à T_F , où F est un des ensembles de symboles fonctionnels trouvés dans l'exercice précédent.

1. $*(+(2, 3), 5), +(4, \text{Moins}(3)), 2)$
2. $f(g(a, h(c)), g(f(a, b, c), h(a)), a)$

3.2.3 Représentation arborescente des termes

Il est souvent utile de représenter graphiquement un terme par un arbre, en appliquant la règle suivante :

Un terme de la forme $f(t_1, \dots, t_n)$ est représenté par un arbre dont la racine est étiquetée par f et qui a n fils, qui sont les représentations arborescentes de $t_1, \dots, t_n$ dans cet ordre. Si n est nul, alors la racine n'a pas de fils.

Cette règle est illustrée dans la figure 3.1 par la représentation arborescente du terme

$$f(a, a, g(f(a, b, c), h(a))).$$

Dans l'arbre obtenu par application systématique de la règle précédente, tout nœud sans fils est appelé *feuille*. Les autres nœuds sont appelés *nœuds internes*.

Exercice 3.4. Dessiner sous forme arborescente les deux termes de l'exercice précédent.

Questions

Répondre aux questions suivantes, en justifiant la réponse :

1. Tout terme a-t-il une représentation arborescente ?
2. Toute arborescence est-elle un terme ?

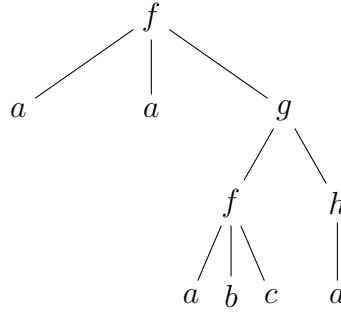


FIGURE 3.1 – Représentation arborescente d'un terme.

3.2.4 Notation infixée

La notation $f(t_1, \dots, t_n)$ est appelée *notation fonctionnelle*. Si le symbole fonctionnel f est binaire (c'est-à-dire d'arité 2), on lui préfère souvent la *notation infixée*. Par exemple, on écrira $(x + y)$ au lieu de la notation fonctionnelle $+(x, y)$.

Cette convention présente plusieurs avantages. D'une part, elle améliore la lisibilité des expressions en plaçant l'opérateur entre ses deux opérandes, ce qui correspond naturellement à la façon dont les humains perçoivent les opérations arithmétiques ou logiques. D'autre part, cette notation permet de simplifier la syntaxe dans de nombreux langages de programmation, en évitant des parenthèses superflues.

3.2.5 Termes avec variables

Dans une expression, une variable peut être remplacée (on dit *substituée*) par une expression quelconque. Par exemple, dans l'expression

$$e =_{\text{def}} (2 + x) * (y * ((4 * \text{Moins}(x)) + 2))$$

on peut substituer la variable x par $2 * x + 7$ et la variable y par $y + 2$, pour obtenir

$$e[x := 2 * x + 7, y := y + 2] =_{\text{def}} (2 + (2 * x + 7)) * ((y + 2) * ((4 * (\text{Moins}(2 * x + 7)) + 2))).$$

Les variables seront souvent désignées par $x, y, z, x_1, y_1, z_1, \dots$, sur la base des lettres de la fin de l'alphabet, à partir de x .

On désigne par X un ensemble quelconque (c'est-à-dire non nécessairement fini) de variables. L'ensemble des *termes avec variables* dans X , noté $T_F(X)$, est le plus petit ensemble qui contient X et qui contient le terme $f(t_1, \dots, t_n)$ pour tout entier naturel n , tout symbole fonctionnel f de F_n et tous les termes $t_1, \dots, t_n$ dans cet ensemble $T_F(X)$.

Les termes avec variables sont aussi appelés *termes ouverts*.

Exercice 3.5. Avec $X = \{x, y, z\}$, décrire un ensemble des symboles fonctionnels F tel que les expressions suivantes soient dans $T_F(X)$:

1. $*(+(x, x), *(x, +(4, Moins(x), 2)))$
2. $f(g(a, h(x)), y, g(f(a, z, z), h(y)))$

Syntaxiquement, les symboles fonctionnels sont des briques de base – des constructeurs de données – qu’on relie entre elles pour construire les termes. Les constantes sont situées aux feuilles de la représentation arborescente tandis que les symboles fonctionnels d’arité non nulle en sont les nœuds internes.

Exercice 3.6. *Questions sur les termes avec variables :*

1. *Peut-on représenter un terme ouvert de façon arborescente ?*
2. *Si oui, quelle est la position des variables ?*

3.3 Syntaxe des formules

Les formules du calcul des prédicats sont composées des connecteurs logiques de la logique des propositions, de formules quantifiées introduites dans les parties 3.3.2 à 3.3.4 et de formules atomiques (partie 3.3.1) s’appliquant à des termes, définis dans la partie 3.2.

3.3.1 Prédicats, signature relationnelle et formules atomiques

Il s’agit à présent d’introduire des relations entre des termes de l’univers du discours. Par exemple, « 22 est le double de 11 », « 46 est le double de 45 » sont des propositions (vraies ou fausses !) qui évoquent la relation « être le double de ».

DÉFINITION 3.1 (RELATION). *On appelle relation n -aire définie sur l’univers du discours $\mathcal{U}$ toute application de $\mathcal{U}^n$ dans l’ensemble $\mathbb{B} = \{0, 1\}$ des booléens.*

DÉFINITION 3.2 (SYMBOLE RELATIONNEL, PRÉDICAT). *On appelle symbole relationnel ou prédicat, à n places ou prédicat d’arité n , tout symbole p tel que l’expression $p(t_1, t_2, \dots, t_n)$, dans laquelle $t_1, t_2, \dots, t_n$ sont des termes, s’interprète comme l’application d’une relation n -aire entre les interprétations des termes $t_1, t_2, \dots, t_n$.*

Un ensemble fini de symboles relationnels est souvent nommé R et appelé *signature relationnelle*. Pour tout entier naturel n , on désigne par R_n l’ensemble des symboles relationnels de R d’arité n .

DÉFINITION 3.3 (FORMULE ATOMIQUE, ATOME). *On appelle formule atomique, ou atome, toute formule de la forme $p(t_1, \dots, t_n)$ où p est un prédicat d’arité n et $t_1, \dots, t_n$ sont des termes.*

Par exemple $mange(chat, souris)$ est un atome, tandis que $\neg mange(grenouille, \acute{e}l\acute{e}phant)$ n'est pas un atome.

Remarque importante : Ces d  finitions s'appliquent aussi dans le cas o   $n = 0$. Si p est un pr  dicat 0-aire (d'arit   0, sans « variables »), alors la formule atomique $p()$ est plus simplement not  e p et c'est une proposition atomique au sens de la logique des propositions, qui a donc une valeur de v  rit  .

3.3.2 Quantification universelle

Consid  rons la proposition « Tous les   tudiants travaillent les math  matiques » pour l'analyser du point de vue du calcul des pr  dicats. Aucun connecteur logique n'appara  t. Par contre, on peut remarquer l'intervention d'un pr  dicat (binaire, d'arit   2) qui peut   tre formalis   par $travaille(x, y)$, et qui signifie qu'un individu x travaille une certaine mati  re y .

Il faut pr  ciser l'univers du discours, il s'agit ici d'un ensemble produit cart  sien de deux ensembles :

- le premier est l'ensemble des   tudiants,
- le second est l'ensemble des noms des mati  res susceptibles d'  tre enseign  es.

L'expression « Tous les   tudiants travaillent les math  matiques » est bien une proposition, qui se formalise par $travaille(x, maths)$. Ici, $maths$ est une constante qui s'interpr  te par une mati  re susceptible d'  tre enseign  e. En revanche, x ne d  signe pas un   tudiant particulier : c'est une variable qui peut repr  senter (prendre pour valeur) n'importe quel   tudiant.

La construction qui permet d'obtenir des propositions    partir de pr  dicats sans qu'une variable soit distingu  e porte le nom de *quantification*. Une proposition de la forme “Pour tout . . .” se formalise par une *quantification universelle*, not  e avec un « $\forall$ » qui repr  sente un A    l'envers (pour *All*) et qui se lit “pour tout”.

Ainsi on repr  sente la proposition ci-dessus par la quantification

$$\forall x . travaille(x, maths).$$

D  FINITION 3.4 (QUANTIFICATEUR UNIVERSEL). $\forall$ est un symbole de quantification, appel   quantificateur universel. Dans une formule qui commence par $\forall x$ on dit que ce symbole quantifie (universellement) la variable x , et on dit que cette variable x est li  e (par ce quantificateur).

REMARQUE 3.1. La pr  sence du symbole de quantification est indispensable pour donner du sens aux formules avec variables.

- $\forall x . travaille(x, maths)$ est une proposition.
- $travaille(x, maths)$ n'est pas une proposition : on ne peut pas lui attribuer une valeur de v  rit  . Il est n  cessaire de substituer x par un terme sans variables pour que cela le devienne (i.e. $travaille(Jean, maths)$ ou bien $travaille(fils(Pierre), maths)$). Dans cette expression, on dit que la variable x est *libre*.

3.3.3 Quantification existentielle

Dans la proposition « Tous les étudiants travaillent au moins une matière » le même prédicat binaire *travaille* intervient. On remarque qu'aucun étudiant n'est explicitement nommé, pas plus que la matière qu'il travaille. Si le « Tous » induit une quantification universelle, le « au moins une matière » signifie qu'il existe une matière travaillée par cet étudiant. Pour formaliser ce genre de propositions, on utilise le quantificateur existentiel ($\exists$), qui est un E (première lettre de *Exists*) retourné.

Cet exemple se traduit alors par

$$\forall x. \exists y. \text{travaille}(x, y)$$

où la variable x est liée par le quantificateur universel $\forall x$ et la variable y est liée par le quantificateur existentiel $\exists y$.

3.3.4 Alternance de quantificateurs

Attention : *Il est interdit d'intervertir des quantificateurs de symboles différents* (si l'on désire que la formule conserve le même sens, bien entendu).

EXEMPLE 3.7. La formule $\forall x. \exists y. \text{travaille}(x, y)$ signifie que tout étudiant travaille une matière (au moins). Autrement dit, la matière travaillée dépend de cet étudiant, elle n'est éventuellement pas la même pour tous les étudiants.

La formule $\exists y. \forall x. \text{travaille}(x, y)$ signifie qu'il y a une matière que tous les étudiants travaillent. La différence fondamentale avec le cas précédent est que, dans cette dernière affirmation, on affirme que tous les étudiants travaillent la même matière.

EXEMPLE 3.8. En supposant que (a, b, c) prenne ses valeurs dans un univers du discours qui est une partie de $\mathbb{R}^3$ telle que $b^2 - 4ac > 0$, la formule :

- $\forall a. \forall b. \forall c. \exists x. ax^2 + bx + c = 0$ est une formule vraie (une équation du second degré dont le discriminant est positif admet des racines réelles, et les valeurs de ces racines dépendent de celles de a , de b et de c).
- $\exists x. \forall a. \forall b. \forall c. ax^2 + bx + c = 0$ est une formule fausse : il n'y a pas de réel qui soit solution de n'importe quelle équation du second degré !

3.3.5 Variables libres et variables liées

DÉFINITION 3.5 (VARIABLE LIÉE, VARIABLE LIBRE). *Dans une formule, une variable est liée si elle est sous la portée d'un quantificateur. Sinon, elle est dite libre.*

EXEMPLE 3.9. Dans l'expression $(\forall x. (\exists k. x = 2 * k + y))$, y est une *variable libre* (non liée). Dans l'expression $(\exists k. x = 2 * k + y)$, k est une *variable liée* (par le quantificateur existentiel), tandis que x et y sont des variables libres. Dans l'expression $x = 2 * k + y$, toutes les variables sont libres.

En général, on permet qu'un seul quantificateur lie plusieurs variables, comme dans l'expression

$$(\exists x, y, z. x = y^2 + z^2),$$

qui n'est qu'une abréviation pour la formule

$$(\exists x. (\exists y. (\exists z. x = y^2 + z^2))).$$

REMARQUE 3.2. Les variables liées peuvent être renommées, mais pas les variables libres.

EXEMPLE 3.10. Par exemple, $(\forall x. (\exists k. x = 2 * k + y))$ et $(\forall z. (\exists x. z = 2 * x + y))$ sont deux expressions équivalentes.

3.3.6 Formules du calcul des prédicats

DÉFINITION 3.6 (FORMULE). *La définition d'une formule (de la logique des prédicats) est :*

- *Un atome est une formule,*
- *si P est une formule, si Q est un symbole de quantificateur ($\exists$ ou $\forall$) et si x est un symbole de variable, alors $(Qx. P)$ est une formule,*
- *si P est une formule, alors $(\neg P)$ est une formule,*
- *si P et Q sont des formules, alors $(P \vee Q)$, $(P \wedge Q)$, $(P \Rightarrow Q)$, $(P \Leftrightarrow Q)$ sont des formules.*
- *Il n'existe pas d'autres manières de construire une formule qu'en appliquant les règles précédentes un nombre fini de fois.*

3.3.7 Portée d'un quantificateur

DÉFINITION 3.7 (PORTÉE D'UN QUANTIFICATEUR). *Dans une formule de la forme $Qx.P$, où Q est $\forall$ ou $\exists$, la portée du quantificateur Q est la partie de la formule P dans laquelle la variable x est effectivement liée par ce quantificateur.*

Pour les formules de la forme $Qx. (\dots)$ ou $(Qx. \dots)$, la portée est clairement imposée par les parenthèses. Par exemple, dans la formule $\forall x. (p(x) \rightarrow \exists y. r(x, y))$, la portée de $\forall x$ ne peut être que $p(x) \rightarrow \exists y. r(x, y)$. La question ne se pose pas non plus pour la sous-formule $\exists y. r(x, y)$, dans laquelle la portée de $\exists y$ est évidemment $r(x, y)$.

La question se pose quand il y a moins de parenthèses. Par convention, dans l'écriture d'une formule, un quantificateur est moins prioritaire que tout connecteur logique. Par exemple, $\forall x. p(x) \Rightarrow q(x)$ correspond à $(\forall x. p(x) \Rightarrow q(x))$ et non pas à $(\forall x. p(x)) \Rightarrow q(x)$. Ceci signifie que la portée d'un quantificateur va toujours le plus à droite possible, tout en respectant les parenthèses. En cas de doute sur la portée d'un quantificateur, on recommande d'ajouter une paire de parenthèses pour la rendre plus lisible.

La portée permet de comprendre quelles variables sont liées par chaque quantificateur et quelles occurrences de variables sont libres.

3.4 Sémantique

3.4.1 Valeurs de vérité

Le calcul des prédicats utilise, comme le calcul propositionnel, les connecteurs logiques, et produit des propositions. Il est possible d'étendre la notion de « valeur de vérité » au calcul des prédicats. Mais l'étude de la valeur de vérité d'une formule du calcul des prédicats est beaucoup plus compliquée.

Une formule propositionnelle typique est $p \Rightarrow q$. Les « atomes » sont ici des variables propositionnelles qui peuvent être remplacées par n'importe quelle proposition. Quelle que soit cette proposition, sa valeur de vérité ne peut être que « vrai » ou « faux ». Cela permet de lui associer la fonction booléenne $(p, q) \mapsto \bar{p} + q$ qui associe l'expression booléenne $\bar{p} + q$ aux variables booléennes p et q .

Une formule du calcul des prédicats analogue pourrait être $\forall x. p(x, y) \Rightarrow q(x, z)$. Les atomes sont ici des prédicats d'arité 2 qui, eux aussi, ne peuvent prendre que les valeurs « vrai » ou « faux », mais pas indépendamment des interprétations des variables libres y et z .

Il n'est donc pas possible de remplacer un atome par une simple variable booléenne. Seule une fonction booléenne (de variables non booléennes) peut convenir, pour faire intervenir les valeurs des variables qui sont les arguments du prédicat.

Si $f(x, y)$ est la fonction booléenne associée au prédicat $p(x, y)$ et si $g(x, z)$ est celle qui est associée à $q(x, z)$, alors la fonction de vérité de la formule $p(x, y) \Rightarrow q(x, z)$ est $\overline{f(x, y)} + g(x, z)$.

Attention, x , y et z ne sont pas ici des variables booléennes, mais elles prennent leurs valeurs dans l'univers du discours. Il n'est donc pas question de calculer avec x , y et z comme en algèbre de Boole. En théorie, si l'univers du discours est fini, on peut étudier une telle fonction de vérité en construisant le tableau de ses valeurs, en donnant à x , y et z successivement toutes les valeurs possibles dans l'univers du discours. En pratique, ce n'est possible que si le nombre de variables et la taille de l'univers du discours sont petits. Si ce dernier est infini, les difficultés commencent ...

3.4.2 Motivations

Cette partie définit rigoureusement la sémantique des formules de la logique des prédicats à l'aide des notions d'interprétation et d'environnement, qu'on peut introduire et motiver comme suit, avant d'en donner une définition précise.

Soit φ une formule d'un langage du premier ordre. Par exemple, φ peut être la formule

$$(\forall y. (\exists z. R(z)) \vee (\forall z. S(y, z) \wedge \neg S(h(x, z), a))).$$

Quand peut-on dire que la formule φ est vraie ? La vérité de la formule φ dépend

- de la signification des symboles fonctionnels (a et h dans l'exemple) et relationnels (R et S dans l'exemple). C'est la notion d'interprétation, définie plus loin.
- de la valeur des variables libres de φ (x dans l'exemple). C'est la notion d'environnement, définie plus loin.

Globalement, il s'agit de définir la *sémantique* (le sens, la signification) d'un langage du premier ordre.

3.4.3 Langages du premier ordre

Un *langage du premier ordre* est défini par un ensemble infini X de variables et deux ensembles finis de symboles :

- une signature fonctionnelle F de symboles fonctionnels d'arité fixée et
- une signature relationnelle R de symboles relationnels d'arité fixée.

Rappel : Les sous-ensembles de symboles fonctionnels et relationnels d'arité n sont notés F_n et R_n . Un *terme* est soit une variable (dans l'ensemble X), soit un symbole fonctionnel d'arité n ($n \geq 0$) appliqué à n termes. Un *atome* (ou *formule atomique*) est un symbole relationnel d'arité n ($n \geq 0$) appliqué à n termes. Une formule de la logique des prédicats est composée de quantifications et de connecteurs logiques appliqués à des formules atomiques.

3.4.4 Définition d'une interprétation

Une *interprétation* $\mathcal{I}$ d'un langage L du premier ordre consiste à choisir

- un ensemble non vide $\mathcal{U}_{\mathcal{I}}$, appelé *domaine d'interprétation*, ou *univers (du discours)*,
- une fonction $f_{\mathcal{I}}$ de $\mathcal{U}_{\mathcal{I}}^n$ dans $\mathcal{U}_{\mathcal{I}}$ pour chaque symbole fonctionnel f d'arité n de L , et
- une relation $R_{\mathcal{I}}$ sur $\mathcal{U}_{\mathcal{I}}^n$ pour chaque symbole relationnel R d'arité n de L .

Cette *interprétation* $\mathcal{I}$, composée de son ensemble $\mathcal{U}_{\mathcal{I}}$, de ses fonctions $f_{\mathcal{I}}$ et de ses relations $R_{\mathcal{I}}$, est aussi appelée *structure (interprétative)*.

EXEMPLE 3.11. Pour le langage de la formule φ précédente, on peut choisir l'interprétation $\mathcal{I} = (\mathbb{Z}, -1, \dots + \dots, \dots \leq 0, \dots > \dots)$ dans laquelle $\mathcal{U}_{\mathcal{I}} = \mathbb{Z}$ est l'ensemble des entiers relatifs, $a_{\mathcal{I}} = -1$, $h_{\mathcal{I}}(x, y) = x + y$, $R_{\mathcal{I}} = "\dots \leq 0"$ et $S_{\mathcal{I}} = "\dots > \dots"$.

Exercice 3.12. *Exercice de compréhension de cet exemple de structure interprétative $\mathcal{I}$.*

1. Traduire en français ce que signifie φ dans cette structure.
2. Pour quelles valeurs de x la formule φ est-elle vraie dans cette structure ?

3.4.5 Définition d'un environnement

Soit L un langage du premier ordre, soit $\mathcal{I} = (\mathcal{U}_{\mathcal{I}}, \dots)$ une interprétation de L et soit φ une formule de L . La vérité de la formule φ dans l'interprétation $\mathcal{I}$ ne dépend que de la valeur de ses variables libres dans le domaine d'interprétation. Pour la définir, on introduit la notion suivante.

Un *environnement* (ou *valuation*) du langage L est le choix d'une valeur dans le domaine d'interprétation pour chaque variable de l'ensemble X des variables de L . Formellement, un environnement est une fonction (totale) e de X dans $\mathcal{U}_{\mathcal{I}}$. La vérité de φ dans l'interprétation $\mathcal{I}$ ne dépend que de la valeur $e(x)$ donnée par l'environnement e à chaque variable x libre dans φ . Ainsi, dans l'environnement e , on peut définir la *valeur* de tout terme, puis la *valeur de vérité* de toute formule.

3.4.6 Notion de substitution

Si t est un terme et φ est une formule pouvant contenir la variable x , alors on désigne par $\varphi(t/x)$ le résultat du remplacement de toutes les occurrences libres de x par t dans φ . Le résultat de ce remplacement est une formule qui est une conséquence logique de la formule originale φ si aucune des variables libres de t ne devient liée suite à ce remplacement. On dit alors que t “est substituable à” x dans φ . Pour rendre t substituable à x dans φ , c’est-à-dire pour éviter que des variables libres de t deviennent liées, il suffit de remplacer les noms des variables liées de φ par des noms “frais”, c’est-à-dire qui n’apparaissent pas dans les variables libres de t . L’oubli de cette condition est une cause fréquente d’erreurs de raisonnement.

À titre d’exemple, considérez la formule φ définie par $\forall y. y \leq x$ et une interprétation dans laquelle le symbole $\leq$ est interprété comme étant une relation d’ordre.

- Si t est un terme sans la variable libre y , alors la formule $\varphi(t/x)$ signifie juste que t est l’élément maximal.
- Si t est la variable y , alors la formule $\varphi(y/x)$ est $\forall y. y \leq y$, qui ne dit plus que y est maximal, mais que la relation $\leq$ est réflexive, ce qui est très différent !

NOTATION : La substitution (t/x) est aussi notée $[x := t]$ ou $[x \leftarrow t]$.

Pour définir la sémantique d’une formule φ , il suffit de considérer les environnements qui ne modifient que les variables libres de φ , et laissent toutes les autres variables de X inchangées. Ces environnements qui ne modifient qu’un ensemble fini de variables sont dits à *support fini*. On peut les voir comme des substitutions, non pas par des termes (syntaxiques) mais par des valeurs (sémantiques) et utiliser la même notation. Ainsi, l’environnement qui remplace respectivement les variables $x_1, \dots, x_n$ par les valeurs $v_1, \dots, v_n$ est noté $(v_1/x_1, \dots, v_n/x_n)$, $[x_1 := v_1, \dots, x_n := v_n]$ ou $[x_1 \leftarrow v_1, \dots, x_n \leftarrow v_n]$.

3.4.7 Valeur d’un terme

Dans une interprétation $\mathcal{I}$ donnée, la valeur $val_{\mathcal{I}}(t, e)$ du terme t dans l’environnement e est définie comme suit :

- Si $x \in X$ est une variable, alors $val_{\mathcal{I}}(x, e) = e(x)$.
- Si f est un symbole fonctionnel d’arité n ($n \geq 0$) et si $t_1, \dots, t_n$ sont n termes, alors $val_{\mathcal{I}}(f(t_1, \dots, t_n), e) = f_{\mathcal{I}}(val_{\mathcal{I}}(t_1, e), \dots, val_{\mathcal{I}}(t_n, e))$.

C’est une définition récursive car les termes sont eux-mêmes définis de manière récursive (dans la partie 3.2).

3.4.8 Valeur de vérité d’une formule

Dans toute la suite, on désigne par $e[x \leftarrow u]$ l’environnement défini à partir de e par

$$e[x \leftarrow u](y) = \begin{cases} e(y) & \text{si } y \neq x \\ u & \text{si } y = x. \end{cases}$$

Soit L un langage du premier ordre, $\mathcal{I}$ une interprétation de L , φ une formule de L et e un environnement de L . La *valeur de vérité* de la formule φ dans l'environnement e est la constante booléenne 0 ou 1 notée $eval_{\mathcal{I}}(\varphi, e)$ qui est définie comme suit, pour tout symbole relationnel $p \in R$:

$$\begin{aligned} eval_{\mathcal{I}}(p(t_1, \dots, t_n), e) &= 1 \text{ si } (val_{\mathcal{I}}(t_1, e), \dots, val_{\mathcal{I}}(t_n, e)) \in p_{\mathcal{I}} \\ eval_{\mathcal{I}}(\forall x. F, e) &= 1 \text{ si } eval_{\mathcal{I}}(F, e[x \leftarrow u]) = 1 \text{ pour tout } u \in \mathcal{U}_{\mathcal{I}} \\ eval_{\mathcal{I}}(\exists x. F, e) &= 1 \text{ s'il existe } u \in \mathcal{U}_{\mathcal{I}} \text{ tel que } eval_{\mathcal{I}}(F, e[x \leftarrow u]) = 1 \\ eval_{\mathcal{I}}(\neg F, e) &= \overline{eval_{\mathcal{I}}(F, e)} \\ eval_{\mathcal{I}}(F \vee G, e) &= eval_{\mathcal{I}}(F, e) + eval_{\mathcal{I}}(G, e) \\ &\dots \end{aligned}$$

où les pointillés sont à remplacer par les cas des autres connecteurs logiques, faciles à deviner par analogie avec la définition 2.5 de la fonction *expr-bool*.

3.4.9 Relation de satisfaction

Une formule est dite *close* si elle n'a pas de variables libres. Soit φ une formule close. On dit que “la structure interprétative $\mathcal{S}$ satisfait φ ”, ou bien que “ $\mathcal{S}$ est un *modèle* de φ ”, et on note $\mathcal{S} \models \varphi$, si la formule φ est vraie dans la structure/interprétation $\mathcal{S}$.

Notez que l'environnement n'intervient pas dans cette définition, car la formule est close.

La relation $\models$ entre une structure interprétative et une formule est appelée *relation de satisfaction*.

3.4.10 Satisfaisabilité et validité

Une formule du premier ordre est *valide* si elle est vraie dans toutes les interprétations de ses symboles et pour tous les environnements pour ses variables libres. Une formule est *satisfaisable* s'il existe au moins une interprétation de ses symboles et un environnement de ses variables libres pour lesquels elle est vraie.

Pour éliminer la notion d'environnement dans ces deux définitions, on introduit les notions de clôture suivantes. Soit φ une formule et $x_1, \dots, x_n$ ses variables libres. La *clôture universelle* de φ est la formule

$$\forall x_1. \dots \forall x_n. \varphi. \quad (3.1)$$

La *clôture existentielle* de φ est la formule

$$\exists x_1. \dots \exists x_n. \varphi. \quad (3.2)$$

Ces deux clôtures sont des formules closes, c'est-à-dire sans variable libre. On peut donc aussi définir la validité et la satisfaisabilité comme suit :

Une formule est *valide* si sa clôture universelle est vraie pour toutes ses interprétations. Une formule est *satisfaisable* s'il existe une interprétation pour laquelle sa clôture existentielle est vraie.

3.4.11 Satisfaisabilité et validité

Les définitions de tautologie et de conséquence logique s'adaptent comme suit.

DÉFINITION 3.8 (TAUTOLOGIE, CONSÉQUENCE LOGIQUE, ÉQUIVALENCE). Soit P et Q deux formules du calcul des prédicats.

- P est une tautologie, noté $\models P$, si et seulement si, pour tous les univers du discours possibles, pour tous les prédicats qui peuvent intervenir dans P , et pour toutes les valeurs des variables dans chacun des univers, la valeur de vérité de P est « vrai ».
- Q est une conséquence logique de P , noté $\{P\} \models Q$, si et seulement si, dans les mêmes conditions que ci-dessus, chaque fois que P est vraie, Q l'est aussi.
- Les formules P et Q sont équivalentes, noté $P \approx Q$, si et seulement si $\{P\} \models Q$ et $\{Q\} \models P$.

Il reste à donner la définition de la fonction de vérité pour les nouveaux symboles introduits (les quantificateurs).

- La valeur de vérité de $\forall x. p(x)$ est obtenue en faisant la liste des valeurs de vérité de $p(x)$ pour toutes les valeurs possibles de x dans l'univers du discours (liste effective lorsque c'est possible, ou démonstration). Si, pour toute valeur de x , la valeur de vérité de $p(x)$ est vraie, alors la valeur de vérité de $\forall x. p(x)$ est vraie. S'il y a au moins une valeur de x pour laquelle la valeur de vérité de $p(x)$ est fautive, alors la valeur de vérité de $\forall x. p(x)$ est fautive.
- La valeur de vérité de $\exists x. p(x)$ est obtenue en faisant la liste des valeurs de vérité de $p(x)$ jusqu'à ce qu'on trouve vraie. Si on trouve vraie, la valeur de vérité de $\exists x. p(x)$ est vraie. Si, pour tout élément x de l'univers du discours, la valeur de vérité de $p(x)$ est fautive (liste effective ou démonstration), alors celle de $\exists x. p(x)$ est fautive.

Bien entendu, dans certains cas, il n'est pas nécessaire d'établir effectivement la table de vérité d'une formule du calcul des prédicats pour prouver qu'il s'agit d'une tautologie. Par exemple, il est bien clair que, pour tout prédicat ternaire p , on a

$$\models (\forall x, y, z. p(x, y, z)) \Rightarrow (\forall x, y, z. p(x, y, z)).$$

Donnons enfin deux exemples pour lesquels la construction d'une table de vérité n'est pas nécessaire :

EXEMPLE 3.13. Montrer que $\models (\forall x. p(x, x)) \Rightarrow (\forall x. (\exists y. p(x, y)))$.

Soit $\mathcal{U}$ un univers du discours, p une relation binaire sur $\mathcal{U}$.

- Premier cas : dans $\mathcal{U}$, $\forall x. p(x, x)$ est fautive ; alors l'implication est vraie.
- Deuxième cas : dans $\mathcal{U}$, $\forall x. p(x, x)$ est vraie ; donc, pour chaque valeur de x dans $\mathcal{U}$, $p(x, x)$ est vraie. Alors $\forall x. \exists y. p(x, y)$ est vraie car, pour toute valeur de x , il suffit de prendre pour y la même valeur que celle de x pour que $p(x, y)$ soit vraie.

EXEMPLE 3.14. Montrer que $(\forall x. (s(x) \Rightarrow r(x))) \wedge (\exists x. (\neg r(x) \wedge s(x)))$ n'est pas une tautologie.

Pour montrer que cette formule n'est pas une tautologie, il suffit d'exhiber un exemple dans lequel elle est fausse.

Dans l'univers du discours $\mathcal{U}$, considérons deux parties R et S telles que $S \subset R$. On choisit pour $r(x)$ la signification « $x \in R$ » et pour $s(x)$ la signification « $x \in S$ ». Comme $S \subset R$, tout élément extérieur à R ne peut pas être dans S , donc $\exists x. \neg r(x) \wedge s(x)$ est *faux*, et donc la formule est fausse.

3.5 Simplification de formules quantifiées

De la définition de la valeur de vérité d'une formule quantifiée, on peut déduire les équivalences suivantes :

$$\neg(\exists x. p) \approx (\forall x. \neg(p)) \quad (3.3)$$

et

$$\neg(\forall x. p) \approx (\exists x. \neg(p)) \quad (3.4)$$

Pour illustrer cette propriété, voici un exemple : Considérons la proposition (fausse) « l'ensemble des entiers naturels admet un élément maximum ». Pour formaliser cette proposition en calcul des prédicats, on introduit le prédicat $\leq$ de comparaison de deux entiers. La proposition “ x est inférieur ou égal à y ” se formalise en toute rigueur par $\leq(x, y)$, que nous notons $x \leq y$ selon la convention usuelle. L'univers du discours est évidemment $\mathbb{N}$, et, pour exprimer que $\mathbb{N}$ admet un élément maximum, on exprime que tout élément de $\mathbb{N}$ est inférieur ou égal à cet élément maximum, soit :

$$\exists M. \forall n. n \leq M.$$

Pour montrer que cette proposition est fausse, il suffit de démontrer que sa négation est vraie, c'est-à-dire

$$\neg(\exists M. \forall n. n \leq M).$$

D'après ce que nous venons de voir, cette négation a la même valeur de vérité que

$$\forall M. \exists n. \neg(n \leq M),$$

ou encore $\forall M. \exists n. M < n$. Ceci signifie que, pour tout entier M , on peut trouver un entier n qui est plus grand. En effet, il suffit de prendre n égal à $M + 1$.

Voici d'autres équivalences entre formules permettant de réduire la portée de quantificateurs.

PROPRIÉTÉ 3.1 (RÉDUCTION DE PORTÉE DE QUANTIFICATEURS) : Soit p et q des formules quelconques. Alors on a les deux équivalences suivantes :

$$(\forall x. p \wedge q) \approx (\forall x. p) \wedge (\forall x. q) \quad (3.5)$$

$$(\exists x. p \vee q) \approx (\exists x. p) \vee (\exists x. q) \quad (3.6)$$

3.6 Décidabilité

La logique du premier ordre n'est pas décidable, c'est-à-dire qu'il n'existe pas de programme informatique capable de répondre correctement par "oui" ou "non" à la question "Cette formule de la logique du premier ordre est-elle valide ?" pour toutes les formules du premier ordre.

Plus précisément, la logique du premier ordre est semi-décidable, car un tel programme existe pour les formules valides. En effet, le théorème de Church dit que l'ensemble des formules valides de la logique du premier ordre est récursivement énumérable. Autrement dit, il existe un programme qui énumère toutes ces formules les unes après les autres. On peut exécuter ce programme et comparer chaque formule générée avec une formule F donnée, et l'arrêter dès que la formule F est générée. Ce programme termine pour toute formule F valide, mais ne terminera pas si la formule F n'est pas valide, car il doit alors énumérer (en vain) une infinité de formules valides.

3.7 Logique du premier ordre avec Why3

Cette partie montre comment définir des formules de la logique du premier ordre dans le langage WhyML de l'outil Why3, puis déterminer si ces formules sont vraies ou pas. Par rapport à la présentation classique des parties précédentes, deux différences majeures sont à noter :

1. En WhyML, toute expression doit avoir un "type", tandis qu'il n'y a pas de notion de type dans la logique du premier ordre. Cette notion de type sera approfondie ultérieurement. En attendant, on admet facilement que si toutes les expressions WhyML ont le même type, disons $\mathbf{t}$, alors c'est comme s'il n'y avait pas de type, ce qui nous ramène à la logique des prédicats, sans types.
2. Pour $n \geq 1$, l'application du symbole fonctionnel f d'arité n aux expressions $e_1, \dots, e_n$ ne se note pas

$$f(e_1, \dots, e_n)$$

mais

$$(f \ e_1 \ \dots \ e_n)$$

avec au moins un espace avant chaque expression $e_1, \dots, e_n$. De même pour les symboles relationnels.

3.7.1 Fonctions, prédicats et quantificateurs en WhyML

Le listing 3.1 montre comment déclarer des symboles fonctionnels, des symboles relationnels et des formules du premier ordre avec des quantificateurs en WhyML.

On peut aussi utiliser le mot-clé `function` pour définir un symbole fonctionnel d'arité nulle, mais l'usage en WhyML est de le remplacer par le mot-clé `constant`, même si conceptuellement il n'est pas nécessaire de faire cette distinction.

Chaque variable liée par un quantificateur a un type, mais cet exemple montre qu'il n'est pas nécessaire d'explicitement ce type (cas de `y`), car chaque symbole et expression ne peut avoir qu'un type, que la machine peut déterminer à partir des déclarations données.

```
type t

constant c : t
function f t t : t
function g t : t

predicate r t t

lemma l1: exists x:t. forall y. r (f (g c) x) (g y) → r (f (g c) c) (g c)
```

Listing 3.1 – Syntaxe WhyML pour les symboles fonctionnels, relationnels et les quantificateurs.

3.7.2 Preuve de formules du premier ordre avec Why3

L'outil Why3 est capable de démontrer automatiquement la validité de certaines formules du premier ordre. La formule à démontrer doit d'abord être exprimée dans un *lemme* (avec le mot-clé `lemma`), comme dans l'exemple du listing 3.1, ou dans un *but* (avec le mot-clé `goal`). La différence entre un but et un lemme en WhyML est que, pour démontrer un lemme ou but donné, Why3 pourra se servir de tous les lemmes qui le précède, mais n'utilisera aucun des buts précédents.

En théorie, la logique du premier ordre étant semi-décidable, l'outil Why3 pourrait démontrer toutes les formules valides. En pratique, un programme comme celui décrit dans la partie 3.6, qui reconnaît toutes les formules valides par énumération, est très inefficace. L'outil Why3 implémente des règles de raisonnement plus efficaces. Même si Why3 implémentait ce programme, sa réponse pourrait prendre beaucoup trop de temps. C'est pourquoi cet outil ne cherche une preuve que pendant un temps limité prédéfini. Lorsque ce temps est écoulé et qu'aucune preuve n'est obtenue (*timeout* en anglais), l'utilisateur ne peut pas savoir si la formule à démontrer est fausse (non valide), ou bien si elle est valide mais trop longue à démontrer par cet outil, compte tenu des hypothèses dont il dispose et des règles de raisonnement qu'il applique.

3.7.3 Déclaration et définition d'un prédicat en WhyML

En WhyML, la ligne

```
predicate p T1 T2
```

est une **déclaration** de prédicat. Elle déclare un prédicat binaire dont le nom est **p** et dont le premier argument est de type **T1** et le second argument de type **T2**. On peut compléter cette syntaxe pour donner un nom à une formule du premier ordre, comme dans la première ligne du listing 3.2. On dit qu'on a **défini** le prédicat **phi**. Les deux lignes suivantes dans ce listing montrent comment on peut ensuite utiliser ce nom pour raccourcir l'expression de deux lemmes.

```
predicate phi (x : t) (y : t) = exists z. r x (f z y)
```

```
lemma 12: forall x y. phi x y
```

```
lemma 13: (forall x y. phi x y) → forall x y. phi x y
```

Listing 3.2 – Exemple de définition de prédicat, avec deux cas d'utilisation.

Plus généralement, une ligne de la forme

```
predicate p (x:T1) (y:T2) = ...
```

avec une formule à la place des pointillés, sans autres variables libres que **x** et **y** est une **définition** de prédicat.

3.8 Synthèse de ce chapitre

La logique du premier ordre, ou logique des prédicats, étend la logique propositionnelle avec des symboles fonctionnels, des symboles relationnels et les quantifications universelle ($\forall$, qui se lit “pour tout”) et existentielle ($\exists$, qui se lit “il existe”). Les formules de cette logique peuvent être exprimées dans le langage WhyML d'entrée du logiciel Why3, qui peut démontrer automatiquement la validité de certaines d'entre elles.

Chapitre 4

Types inductifs

4.1 Introduction

Un problème fondamental en informatique est de stocker et de manipuler des collections de données de même nature. Une *structure de données* est une manière de représenter des données de même nature dans la mémoire d'un ordinateur, pour rendre leur manipulation possible et efficace. Étudier ces structures, notamment celles à base de pointeurs, fait partie d'une branche de l'informatique appelée *algorithmique*. Raisonner sur des structures à base de pointeurs est difficile, en particulier parce que plusieurs pointeurs peuvent faire référence à la même partie de la mémoire. C'est pourquoi la notion de pointeur et le raisonnement sur les pointeurs sont exclus du programme de ce cours d'initiation.

4.1.1 Notions de type abstrait et de système de types

Dans ce cours, on apprend à raisonner sur les données informatiques indépendamment de la manière dont elles sont représentées dans la mémoire de l'ordinateur, grâce aux *types de données abstraits* (*abstract datatypes* en anglais). Dans ce chapitre, une syntaxe est proposée pour définir de tels types et associer un type unique à chaque expression. Dans l'annexe [B](#), des règles de raisonnement sont présentées pour vérifier ou inférer le type de toute expression. Le tout (syntaxe + règles) constitue un *système de types*.

Le système de types présenté est le *système de Hindley-Milner*, car il offre un bon compromis entre expressivité et automaticité des raisonnements. On le retrouve au sein du système de types du langage WhyML de Why3.

4.1.2 Types et ensembles

Un type désigne par un nom unique des données de même nature. En gros, un type peut être considéré comme un ensemble de valeurs. Comme c'est inexact en toute généralité (quoique correct dans le cadre limité de ce cours), on ne dit que par abus qu'un type a des « éléments » qui « appartiennent » au type (vocabulaire de la théorie des ensembles). On préfère toujours dire qu'un type a des « *habitants* » qui « *habitent* » ce type.

4.1.3 Plan du chapitre

Dans ce chapitre, la partie 4.2 définit certains types de données abstraits, dits « inductifs », adaptés à une forme simple de raisonnement formel sur les propriétés des données et programmes informatiques. La partie 4.3 ajoute la notion de type dans les formules de quantification de la logique du premier ordre. Enfin, la partie 4.4 présente la syntaxe WhyML pour déclarer ces types et les utiliser dans des formules de la logique de Why3.

4.2 Définir un type inductif

Si vous avez déjà programmé dans un langage typé, comme le langage C, vous connaissez les types de base, prédéfinis, comme `char`, `int` ou `float` en C, et des moyens de définir vos propres types, comme les structures (`struct`) et les tableaux en C.

Nous adoptons ici un style de programmation où les types sont définis par les diverses manières de construire leurs habitants. Nous présentons d’abord ces fonctions de construction, appelées « constructeurs », puis des exemples de types ainsi définis, et enfin la syntaxe générale des (noms des) types.

4.2.1 Constructeurs

Nous avons besoin de symboles qui permettent le regroupement des informations dans la structure. On les appelle les *constructeurs* des habitants du type. Commençons par quelques exemples, avant de généraliser dans la partie 4.2.2.

4.2.1.1 Constantes booléennes

Dans ce cours, nous désignons par $\mathbb{B}$ le *type énuméré* dont les deux habitants sont deux constantes, dites *booléennes*, notées 0 (ou F , pour « faux ») et 1 (ou V , pour « vrai »). Ces deux constantes sont les constructeurs du type $\mathbb{B}$.

Remarque importante : Les constantes booléennes ne sont notées 0 et 1 que quand il n’y a pas de risque de confusion avec des entiers naturels. Sinon, elles sont notées F et V .

4.2.1.2 Constructeurs des entiers naturels

Nous désignons par $\mathbb{N}$ le type (infini) des entiers naturels, défini formellement plus loin. Il est impossible de programmer des calculs sur les entiers naturels (comme l’addition) si l’on considère qu’il est habité par une infinité de constantes 0, 1, 2, 3, ...

En revanche, nous allons voir comment définir le type $\mathbb{N}$ des entiers naturels avec seulement deux constructeurs :

- une constante, notée Z , pour représenter le plus petit entier naturel 0, et
- un constructeur $S : \mathbb{N} \rightarrow \mathbb{N}$ qui construit chaque entier naturel à partir de son prédécesseur.

Avec ces constructeurs, les entiers naturels usuels 0, 1, 2 et 3 sont respectivement représentés par les expressions Z , $S(Z)$, $S(S(Z))$ et $S(S(S(Z)))$.

Lorsqu'il n'y a pas de risque de confusion, la constante Z est notée 0.

La notation $S : \mathbb{N} \rightarrow \mathbb{N}$ sera généralisée ultérieurement. Elle suggère que le constructeur S est une fonction qui associe un entier naturel à tout entier naturel. Effectivement, ce constructeur correspond à la fonction usuelle $n \mapsto n + 1$, mais, pour éviter des confusions, on s'interdit temporairement de remplacer $S(n)$ par $n + 1$.

4.2.1.3 Connecteurs logiques

Les connecteurs logiques $\wedge$, $\vee$, $\neg$, $\Rightarrow$ et $\Leftrightarrow$ présentés dans le cours 2 peuvent être considérés comme des constructeurs de formules propositionnelles. Dans ce cas, ils seront respectivement notés *Et*, *Ou*, *Non*, *Imp* et *Eqv*.

4.2.1.4 Constructeurs des listes

Nous introduisons à présent le type de données abstrait le plus simple et le plus fréquemment utilisé pour décrire des collections de données. C'est le type des *listes (homogènes)*. Une liste homogène est une collection ordonnée (une séquence) finie de données de même nature.

Pour définir formellement les listes homogènes, nous utilisons la notion de type "générique" ou "polymorphe", qui sera généralisée dans la partie 4.2.3. Le type des listes homogènes dont tous les éléments sont des booléens est noté $liste(\mathbb{B})$. Le type des listes homogènes dont tous les éléments sont des entiers naturels est noté $liste(\mathbb{N})$. Plus généralement, quel que soit le type α , le type des listes homogènes dont tous les éléments sont de type α est noté $liste(\alpha)$. Ce type est dit « générique » ou « polymorphe », car on peut remplacer α par toute expression qui désigne un type, comme $\mathbb{B}$, $\mathbb{N}$, $liste(\mathbb{B})$, $liste(liste(\mathbb{N}))$, etc.

Exercice 4.1. *Après avoir étudié tout le chapitre, donner un exemple d'élément appartenant à chacun de ces types.*

Pour définir le type $liste(\alpha)$, on introduit deux constructeurs :

- La constante *Nil*, qui désigne la liste vide (sans élément). Elle permet d'initialiser la construction.
- Le constructeur *Cons*, qui réclame deux arguments : un élément a de type α et une liste l de type $liste(\alpha)$. L'expression $Cons(a, l)$ représente la liste qui commence avec l'élément a et se poursuit avec les éléments de la liste l , dans le même ordre.

Ainsi l'expression $Cons(2, Cons(4, Cons(1, Nil)))$ représente la liste des trois entiers 2, 4 et 1 dans cet ordre.

Vu comme une fonction, le constructeur *Cons* ajoute l'élément a en tête (au début) de la liste l .

Exercice 4.2. *Quelles sont les différences majeures entre une liste et un ensemble ?*

4.2.1.5 Remarques et conventions pour les constructeurs

Du point de vue de la logique du premier ordre, un constructeur est un symbole fonctionnel. La nouveauté par rapport à la logique du premier ordre présentée dans le chapitre 3, qui est sans types, c'est qu'un constructeur est un symbole fonctionnel **associé à un type** t , en ce sens qu'il participe à la définition de ce type t . Pour le constructeur C , ceci est formalisé par la notation

$$C : t \tag{4.1}$$

si C est une constante ou

$$C : \dots \rightarrow t \tag{4.2}$$

si C est d'arité non nulle. Cette notation sera définie plus loin. Nous expliciterons et formaliserons aussi plus loin la manière dont un ensemble de constructeurs définit un type. Pour l'instant, il est important de retenir et de respecter les conventions suivantes, dans le cadre de ce cours :

1. Tous les constructeurs ont des noms différents.
2. Tous les noms de constructeurs commencent par une majuscule.
3. Tous les symboles fonctionnels qui ne sont pas des noms de constructeurs commencent par une minuscule.

Autrement dit, pour chaque constructeur, il y a une *déclaration de type* unique, de la forme (4.1) ou (4.2), qui fixe le nom du constructeur et son type associé, qu'il contribue à définir. Grâce aux deux dernières conventions, les constructeurs se distinguent facilement de tous les autres symboles fonctionnels, dans toute expression formelle. Nous retrouverons ces conventions dans la partie 4.4.

4.2.2 Habitants d'un type inductif

Informellement, les habitants d'un type inductif t sont tous les termes qu'on peut construire en appliquant un constructeur de ce type à des habitants du même type ou d'un autre type (en respectant des règles de typage qui seront détaillées plus loin). Le nombre total d'applications de constructeurs doit être fini. Ainsi, chaque habitant est une expression formelle composée de noms de constructeurs, de parenthèses et de virgules, comme illustré par les exemples précédents et suivants. A priori, chaque habitant est supposé représenter une donnée différente de ce type t (propriété dite de "liberté" du type), et il existe un habitant pour chaque donnée de ce type (propriété dite de "complétude" de ce type). Cette définition informelle et ces propriétés seront formalisées ultérieurement.

4.2.3 Expressions de type

- Dans les exemples précédents, nous avons vu trois sortes d'expressions pour définir un type :
- des types qui portent juste un nom, comme $\mathbb{B}$ ou $\mathbb{N}$, qui sont appelés "types de base";
 - des types avec paramètres, comme $liste(\mathbb{N})$ ou $liste(\alpha)$;

— des types de fonctions, de la forme $t \rightarrow t'$, où t et t' sont des types ;

Tous les types que nous considérerons seront de l’une de ces trois sortes. Cette partie donne des explications complémentaires sur les deux premières sortes.

4.2.3.1 Types de base

On appelle “type de base”, “type simple” ou “type élémentaire”, tout type défini uniquement par un identificateur. Parmi ces identificateurs, on utilisera dans la suite $\mathbb{N}$, $\mathbb{Z}$ et $\mathbb{B}$, pour désigner respectivement les types des entiers naturels, des entiers relatifs et des constantes booléennes.

Si l’on considère qu’une expression de type est un terme, alors ces identificateurs sont des constantes dans ces termes, d’où le nom de *type constant* parfois donné à cette première sorte de types.

Par exemple, l’expression $liste(\mathbb{N})$, qui désigne le type des listes dont tous les éléments sont des entiers naturels, est un terme construit à partir de la “constante de type” $\mathbb{N}$ et d’un symbole fonctionnel $liste$ d’arité 1, qui pourrait être appelé “constructeur de type” (à ne pas confondre avec les constructeurs présentés précédemment, qui sont des constructeurs de données).

4.2.3.2 Types polymorphes et variables de type

Le terme $liste(\mathbb{N})$ est clos, puisqu’il ne contient pas de variable. Or des fonctions comme la longueur d’une liste ne dépendent pas du type des éléments de cette liste. On souhaite définir ces fonctions une seule fois, sur un type unique, qui serait le type des listes homogènes dont tous les éléments sont de même type, ce type de ces éléments étant quelconque. Pour désigner ce type, dit “générique” ou “polymorphes”, on considère des variables substituables par un type quelconque, appelées *variables de type*. Pour distinguer ces variables de type des variables substituables par des termes ou par des expressions, qui sont notées $x, y, z, \dots$, nous les désignerons par des lettres grecques $\alpha, \beta, \gamma, \dots$.

Par exemple, on parlera du type générique $liste(\alpha)$ pour signifier que la variable α peut être substituée par n’importe quel type (générique ou pas). À partir de ce type générique, on obtient par exemple le type $liste(\mathbb{N})$ des listes d’entiers naturels, en remplaçant la variable α par la constante $\mathbb{N}$.

On définira plus loin et on utilisera aussi le type générique $arbre(\alpha)$ des arbres binaires à feuilles de type quelconque.

4.2.4 Exemples de types inductifs

De ce qui précède il résulte qu’un type inductif t est défini par une *signature fonctionnelle* composée d’un nombre fini de déclarations de constructeurs de la forme (4.1) ou (4.2), qui ont tous des noms deux à deux distincts. Ceci est illustré par les exemples suivants.

4.2.4.1 Exemple des booléens

DÉFINITION 4.1 (TYPE INDUCTIF DES BOOLÉENS). *Le type énuméré $\mathbb{B}$ des booléens est défini par les deux constructeurs $0 : \mathbb{B}$ et $1 : \mathbb{B}$.*

Ainsi, les seuls habitants de ce type sont 0 et 1, et ils sont distincts l'un de l'autre.

4.2.4.2 Exemple des entiers naturels

DÉFINITION 4.2 (TYPE INDUCTIF DES ENTIERS NATURELS). *Le type inductif $\mathbb{N}$ des entiers naturels est défini par les deux constructeurs $Z : \mathbb{N}$ et $S : \mathbb{N} \rightarrow \mathbb{N}$.*

L'entier naturel n est représenté par l'unique expression de la forme $S(S(\dots S(Z)\dots))$ contenant n fois le symbole S . En particulier, l'entier 0 est représenté par Z , l'entier 1 par $S(Z)$, l'entier 2 par $S(S(Z))$. Ainsi, tous les entiers naturels sont représentés, et chacun est représenté par une expression unique.

4.2.4.3 Exemple des formules propositionnelles

La définition suivante correspond à la définition formelle (2.3) des formules propositionnelles, lorsque l'ensemble des propositions atomiques est représenté par le type ap , supposé énuméré et non vide.

DÉFINITION 4.3 (TYPE INDUCTIF DES FORMULES PROPOSITIONNELLES). *Le type inductif $pf(ap)$ des formules propositionnelles sur le type ap des propositions atomiques est défini par les constructeurs suivants :*

$$Atom : ap \rightarrow pf(ap) \quad (4.3)$$

$$Non : pf(ap) \rightarrow pf(ap) \quad (4.4)$$

$$Ou : pf(ap) \times pf(ap) \rightarrow pf(ap) \quad (4.5)$$

$$Et : pf(ap) \times pf(ap) \rightarrow pf(ap) \quad (4.6)$$

$$Imp : pf(ap) \times pf(ap) \rightarrow pf(ap) \quad (4.7)$$

$$Eqv : pf(ap) \times pf(ap) \rightarrow pf(ap) \quad (4.8)$$

La notation $\times$, détaillée dans la partie B.2.6, correspond pour les types à la notion de produit cartésien de la théorie des ensembles. Le constructeur $Atom$ permet de “plonger” (d'injecter) le type ap dans le type $pf(ap)$ tout en respectant la convention que tout terme n'a qu'un seul type.

Par exemple, si $P : ap$ et $Q : ap$ sont deux constructeurs du type ap , alors $Et(P, Non(Q))$ est un habitant du type $pf(ap)$ des formules propositionnelles, qui représente la formule $P \wedge (\neg Q)$.

4.2.4.4 Exemple des listes

DÉFINITION 4.4 (TYPE INDUCTIF DES LISTES (HOMOGÈNES)). *Le type inductif $liste(\alpha)$ des listes (homogènes) dont les éléments sont de type α est défini par les constructeurs suivants :*

$$Nil : liste(\alpha) \quad (4.9)$$

$$Cons : \alpha \times liste(\alpha) \rightarrow liste(\alpha) \quad (4.10)$$

Les premières explications sur ce type sur dans la partie 4.2.1.4. La déclaration (4.9) affirme que la liste vide Nil est de type $liste(\alpha)$, pour toute expression de type α quelconque. Le terme $Cons(a, l)$ est une expression de type $liste(\alpha)$ construite à partir d’une expression a de type α et d’une expression l de liste, elle-même de type $liste(\alpha)$.

4.2.4.5 Exemple des arbres binaires

Un constructeur est dit *binnaire*, ou d’arité 2, s’il a deux arguments. Par exemple, les constructeurs Et , Ou , Imp et Eqv du type inductif des formules propositionnelles sont binaires. Un type est dit *arborescent* quand il contient au moins un constructeur d’arité supérieure ou égale à 2.

Le type des arbres binaires (homogènes) est le plus simple des types arborescents.

DÉFINITION 4.5 (TYPE INDUCTIF DES ARBRES BINAIRES). *Le type inductif $arbre(\alpha)$ des arbres binaires à feuilles de type α est défini par les deux constructeurs suivants :*

$$Leaf : \alpha \rightarrow arbre(\alpha) \quad (4.11)$$

$$Node : arbre(\alpha) \times arbre(\alpha) \rightarrow arbre(\alpha) \quad (4.12)$$

Un arbre binaire de la forme $Leaf(a)$ s’appelle une *feuille*. Cette feuille stocke la donnée a , de type α . Un arbre binaire qui n’est pas une feuille est de la forme $Node(t_1, t_2)$. Le mot “Node” désigne un *nœud* interne. Son sous-terme t_1 est appelé *fil gauche*, son sous-terme t_2 est appelé *fil droit*. Pour les arbres binaires, le constructeur $Leaf$ joue le même rôle que Nil pour les listes : il permet de construire le plus petit objet.

4.3 Quantification et types inductifs

Le calcul des prédicats présenté jusqu’ici est une logique universelle, car sa sémantique exige qu’une formule soit vraie pour tout univers du discours (sous-entendu, pour tout ensemble dont l’existence est admise en mathématiques). Cette universalité, utile en mathématiques, est moins utile en informatique, dont les univers du discours ne sont pas quelconques, mais sont des ensembles de données. Dans la suite de ce cours, nous considérons une quantification et une interprétation des formules limitées aux types inductifs.

Les formules quantifiées de ce calcul des prédicats dit *typé* sont de la forme

$$(Qx : t. F)$$

où Q est un quantificateur ($\forall$ ou $\exists$), x est une variable, t est un type et F est une formule quelconque de ce calcul des prédicats typé.

REMARQUE 4.1. Cette notation est utilisée en mathématiques, par exemple lorsqu'on écrit

$$\forall x : \mathbb{N}. x \geq 0.$$

4.4 Types inductifs en WhyML

Le langage WhyML de Why3 est fondé sur un système de types qui inclut le système de Hindley-Milner. Nous présentons ici une partie de ce système qui couvre tous les types abordés dans ce module. Certains types sont prédéfinis dans le langage WhyML, d'autres peuvent être ajoutés par l'utilisateur, comme détaillé ici.

Comme dans les conventions de la partie 4.2.1.5, tous les noms de constructeurs WhyML doivent être deux à deux distincts, les noms des constructeurs en WhyML doivent commencer par une majuscule, et les autres identificateurs en WhyML ne doivent pas commencer par une majuscule.

4.4.1 Types énumérés en WhyML

L'utilisateur peut définir en WhyML un type avec un (petit) nombre fini d'habitants, en lui donnant un nom et en énumérant ses constructeurs. Voici par exemple comment on peut déclarer un type pour les sept couleurs de l'arc-en-ciel, selon la syntaxe WhyML :

```
type rainbowColor = Red | Orange | Yellow | Green | Blue  
| Indigo | Violet
```

Dans une telle déclaration de type, des retours à la ligne peuvent être ajoutés ou supprimés.

4.4.2 Types de base en WhyML

Nous n'utiliserons que les types de base suivants du langage WhyML :

- Le type `unit`, qui n'a qu'un habitant, qui est la constante notée `()`.
- Le type `bool` des booléens, dont les deux constantes sont notées `True` et `False`.
- Le type `int` pour les entiers relatifs des mathématiques, que nous notons $\mathbb{Z}$.

Ces trois types sont prédéfinis, en ce sens qu'on peut les mentionner dans un fichier WhyML comme dans le code reproduit dans le listing 4.1, sans avoir à importer quoi que ce soit. Les premières lignes montrent comment on peut définir des constantes, des prédicats et des fonctions avec ces types. La ligne suivante montre qu'on peut exprimer un lemme avec ce prédicat, cette fonction et des constantes de ces trois types. Ce lemme est faux, mais ce n'est pas le sujet ici. Il est correct du point de vue de sa syntaxe et du respect des types.

```
constant u : unit
predicate p unit bool
function f unit bool int : bool
```

```
lemma l1: p () (f u False 042)
```

Listing 4.1 – Exemples de déclarations avec des types prédéfinis de WhyML.

Exercice 4.3. *Cet exercice porte sur le code reproduit dans le listing 4.1.*

1. *Utiliser l’interface en ligne de Why3 pour vérifier que ce code est syntaxiquement correct et bien typé.*
2. *Sans introduire d’autres déclarations, proposer un autre lemme qui serait syntaxiquement correct mais qui serait incorrect du point de vue des types.*
3. *Reproduire le message de l’interface en ligne de Why3 signalant cette erreur de typage.*

On peut remarquer que `()` est un nom de constructeur qui ne commence pas par une majuscule. De même, toutes les séquences (finies) de chiffres décimaux sont acceptées en tant que constantes de type `int`, qui n’est donc pas un type inductif au sens de ce cours. Nous reviendrons sur ce type dans la partie 5.4.

Enfin, on remarque qu’aucun type n’est prédéfini en WhyML pour notre type $\mathbb{N}$ des entiers naturels et le type inductif $pf(ap)$ des formules propositionnelles sur un type ap des propositions atomiques.

4.4.3 Un type pour les listes en WhyML

Au delà des types prédéfinis, Why3 inclut une *bibliothèque standard* qui définit de nombreux types, fonctions et prédicats en WhyML. Ces définitions sont réparties dans des modules, eux-mêmes répartis dans plusieurs fichiers. La notion de module WhyML sera présentée ultérieurement. Une documentation complète de la bibliothèque standard de Why3 au format HTML est consultable à l’adresse <https://www.why3.org/stdlib/>.

Cette bibliothèque contient un fichier `list.mlw`¹ dont le premier module contient la déclaration suivante du type inductif $liste(\alpha)$ des listes homogènes dont tous les éléments sont de type α .

```
type list 'a = Nil | Cons 'a (list 'a)
```

On retrouve ici les deux constructeurs *Nil* et *Cons* des listes, et la barre verticale pour séparer leurs déclarations en WhyML. La lettre grecque α est remplacé par `'a`, qui est la syntaxe WhyML pour une variable de type. Le nom du type n’est pas le mot français *liste*, mais sa traduction `list` en anglais. Enfin, dans l’expression de ce type, la notation fonctionnelle de la forme $l(\cdot)$, avec des parenthèses après l’identificateur de type, est remplacée par la notation `(l~..)` avec un espace entre l’identificateur de type et son argument, comme remarqué dans la partie 3.7 pour les symboles fonctionnels et relationnels.

1. Sa documentation est dans la page <https://www.why3.org/stdlib/list.html>.

4.4.4 Formules quantifiées en WhyML

Nous avons déjà vu dans le lemme du listing 3.1 un premier exemple de formule quantifiée typée en WhyML. Cette notation **(exists $x:t$. F)**, **(exists x . F)**, **(forall $x:t$. F)** ou **(forall x . F)** est la même pour tous les types et pour toute formule F en syntaxe WhyML.

Si la formule F est bien typée, alors les emplacements de présence de la variable x dans la formule F lui attribuent un type unique, disons t , que Why3 sait trouver et qu’il n’est jamais nécessaire de mentionner entre x et le point (.) dans la formule. On choisit donc d’écrire $x:t$ ou x seulement selon qu’on veut rendre la formule plus lisible ou plus courte. Ce choix n’a aucun impact sur la représentation interne de la formule et donc sur le fonctionnement de Why3 sur cette formule.

4.5 Synthèse

Ce chapitre a présenté la notion de type formel, à travers un exemple de système de types qui permet d’associer un type unique à chaque catégorie de données et de détecter des erreurs de mauvais typage. La plupart des types qui seront considérés dans ce cours ont été introduits, dans des exemples faciles à généraliser pour définir d’autres types. Certains types (entiers naturels, listes, arbres, etc) ont une infinité d’habitants, mais sont définis inductivement par un nombre fini de règles. Tous ces types peuvent être définis dans le langage WhyML de l’outil Why3, qui implémente aussi une vérification automatique du bon typage de toute expression.

Cependant, pour définir et vérifier des programmes, on a besoin de plusieurs ingrédients supplémentaires :

1. d’autres types, qui ne sont pas des types inductifs, comme les entiers relatifs ou les tableaux ;
2. des fonctions pour calculer sur ces types, par exemple l’addition de deux nombres ;
3. des prédicats pour exprimer des propriétés sur ces types, par exemple l’appartenance d’une donnée à une liste ;
4. des “axiomes” pour caractériser ces types par des propriétés admises, portant sur certaines de leurs fonctions et prédicats.

Ces notions seront introduites dans les chapitres suivants.

Chapitre 5

Théories du premier ordre

Ce chapitre introduit la notion générale de théorie (formelle) du premier ordre, en l’illustrant par des exemples utiles pour la vérification de programmes. L’exemple principal est la théorie de l’égalité, car il est extrêmement fréquent qu’on ait à exprimer ou vérifier que deux choses sont égales.

5.1 Introduction

Le calcul des prédicats (ou logique du premier ordre) considère des fonctions et des prédicats quelconques, sans signification particulière. Or chaque domaine particulier d’application de ce calcul peut avoir besoin d’introduire des symboles fonctionnels ou relationnels privilégiés. Par exemple, un symbole fonctionnel $\cup$ et un symbole relationnel $\in$ pourront représenter un opérateur d’union et une relation d’appartenance dans une théorie mathématique des ensembles.

Chaque spécialisation du calcul des prédicats à un domaine particulier est appelée “théorie du premier ordre”. Cette spécialisation porte sur les deux aspects du calcul des prédicats. D’un part, son langage est limité à des symboles fonctionnels et relationnels particuliers. D’autre part, la sémantique de ces symboles fonctionnels et relationnels est limitée par des axiomes, qui participent à leur définition.

La construction d’une théorie est une activité de *modélisation* de son domaine d’application. Elle présente donc toutes les caractéristiques d’une activité de modélisation. Ainsi, la théorie produite peut ne pas être totalement conforme à la réalité qu’elle tente de cerner. Cette validation est une activité humaine qui ne peut pas être mécanisée.

Le qualificatif “du premier ordre” est lié à la sémantique de ces théories. Il signifie que les variables des formules ne peuvent être interprétées que par des éléments du domaine d’interprétation, et pas par des structures composées, telles que des fonctions ou des ensembles de valeurs.

5.1.1 Définitions

Dans le calcul des prédicats, les symboles fonctionnels et relationnels sont illimités en nombre, et leur interprétation n’est pas fixée. Une *signature* fixe un sous-ensemble particulier fini de tels

symboles, par leur nom et leur type, si la logique est typée, ou seulement par leur nom et leur arité si la logique est sans types. Cette signature est l'union de la signature fonctionnelle et de la signature relationnelle, définies dans le chapitre 3. À toute signature correspond un *langage (des formules) du premier ordre*, qui est l'ensemble des formules du calcul des prédicats dont tous les symboles fonctionnels et relationnels appartiennent à cette signature.

Une *théorie* est définie par une signature et par un ensemble fini d'*axiomes*, qui sont des formules closes du langage correspondant. Ces axiomes, considérés comme des vérités premières de cette théorie, servent à préciser (à limiter) le sens fonctionnels et relationnels.

Un *théorème* est une formule qui peut être déduite des axiomes de la théorie par un système déductif correct et complet du calcul des prédicats. On désigne aussi parfois par *théorie* le plus petit ensemble de formules déductibles de ses axiomes.

Dans le calcul des prédicats (c'est-à-dire la théorie générale du premier ordre), il existe diverses méthodes de construction d'une démonstration de tout théorème. En revanche, il n'existe pas de méthode générale et totalement automatisable capable d'établir qu'une formule **n'est pas** un théorème. Pour chaque théorie particulière du premier ordre, on se pose donc la question de l'existence d'une telle méthode. Si elle existe, la théorie est dite *décidable*, et *semi-décidable* sinon.

Une méthode qui convient souvent consiste à tenter de construire un contre-exemple, c'est-à-dire un modèle de la théorie dans lequel la formule est fausse. Autrement dit, on cherche à déterminer si la négation de cette formule est satisfaisable. Cette approche est appelée *SMT*, pour *Satisfaisabilité Modulo Théorie* (*Satisfiability Modulo Theory* en anglais).

5.1.2 Plan du chapitre

Les parties suivantes présentent successivement des exemples de théories particulières, les plus utiles pour la preuve de programmes, pour l'égalité (partie 5.2), les entiers relatifs (partie 5.4) et la structure de données des tableaux d'entiers (partie 5.5), très présente en programmation. Ces deux dernières théories sont présentées avec la syntaxe WhyML, car elles sont disponibles dans l'outil Why3.

5.2 Théorie de l'égalité

Le calcul des prédicats n'attribue de statut particulier à aucun prédicat. En particulier, il ne distingue pas de prédicat d'égalité. Or, même en l'absence d'axiomes équationnels pour limiter la sémantique d'autres symboles, comme présenté dans la partie 5.2.3, un symbole binaire d'égalité doit satisfaire certains axiomes, pour capturer la notion d'égalité syntaxique, selon laquelle deux expressions sont égales si elles forment la même séquence de caractères.

Un tel système d'axiomes est appelé *théorie de l'égalité*. Cette partie présente l'axiomatisation la plus classique de l'égalité, puis discute brièvement sur sa correction, sa complétude et la manière dont les outils de preuve automatique raisonnent sur l'égalité.

5.2.1 Axiomatisation de l'égalité

Toute égalité est une *relation d'équivalence*, c'est-à-dire satisfait les trois axiomes suivants :

$$\begin{aligned}\forall x. x = x & \quad \text{(réflexivité)} \\ \forall x, y. x = y \Rightarrow y = x & \quad \text{(symétrie)} \\ \forall x, y, z. x = y \wedge y = z \Rightarrow x = z & \quad \text{(transitivité)}\end{aligned}$$

De plus, l'égalité doit vérifier une propriété, dite *de congruence*, par rapport à chaque symbole fonctionnel ou relationnel de la logique dans laquelle ce prédicat est défini. En logique du premier ordre, on ne peut pas quantifier sur les symboles fonctionnels et relationnels, donc on formalise cette propriété avec un axiome pour chaque symbole fonctionnel et un axiome pour chaque symbole relationnel, comme suit.

Pour chaque symbole fonctionnel f d'arité n non nulle, l'axiome de congruence est

$$\forall x_1, \dots, x_n, y_1, \dots, y_n. \quad x_1 = y_1 \wedge \dots \wedge x_n = y_n \Rightarrow f(x_1, \dots, x_n) = f(y_1, \dots, y_n). \quad \text{(congruence-fct)}$$

Autrement dit, pour des arguments égaux, les fonctions prennent des valeurs égales, ce qui est évidemment vrai pour toute fonction.

De même, pour chaque symbole relationnel p d'arité n non nulle, l'axiome de congruence est

$$\forall x_1, \dots, x_n, y_1, \dots, y_n. \quad x_1 = y_1 \wedge \dots \wedge x_n = y_n \Rightarrow (p(x_1, \dots, x_n) \Rightarrow p(y_1, \dots, y_n)). \quad \text{(congruence-prd)}$$

5.2.2 Discussion

On peut se convaincre facilement que la relation binaire qui est la diagonale du domaine d'interprétation satisfait tous ces axiomes, donc est une interprétation possible de ce prédicat binaire d'égalité. C'est même la plus petite telle relation binaire, au sens de l'inclusion des relations binaires, qui sont des ensembles de couples de valeurs.

Les axiomes de l'égalité peuvent aussi être vus comme des règles de raisonnement. On les présentera alors comme suit :

— L'égalité est réflexive :

$$\frac{}{e = e}$$

autrement dit toute formule de la forme $e = e$, où e est une expression, est un théorème.

— L'égalité est symétrique :

$$\frac{e = f}{f = e}$$

Si $e = f$ est un théorème alors $f = e$ est un théorème.

— L'égalité est transitive :

$$\frac{e = f, f = g}{e = g}$$

Si $e = f$ et $f = g$ sont deux théorèmes alors $e = g$ est un autre théorème.

Ce système de règles est correct pour raisonner sur l'égalité, et devient complet après ajout de règles de congruence. Trivialement, on voit bien qu'un prouveur qui utiliserait la règle de symétrie sans contrôle pourrait perdre beaucoup de temps à remplacer $e = f$ par $f = e$, puis à nouveau $f = e$ par $e = f$, et ainsi de suite sans produire de conséquence intéressante. C'est pourquoi les prouveurs automatiques ne sont en général pas basés (uniquement) sur ces règles, mais sur des mécanismes plus puissants, comme la réécriture, la normalisation ou la clôture de congruence, qui dépassent le cadre de ce cours.

5.2.3 Théories équationnelles

Il est fréquent que des expressions différentes syntaxiquement, c'est-à-dire différentes en tant que séquences de caractères, représentent la même donnée, autrement dit ont la même signification, la même "sémantique". C'est le cas par exemple pour les expressions $((5 + 2) * 3)$ et $((3 * 2) + (5 * 3))$, qui prennent la même valeur dans l'interprétation arithmétique usuelle de la notation $(... + ...)$ comme étant une addition et de la notation $(... * ...)$ comme étant une multiplication, dans l'ensemble des entiers naturels qui interprètent les notations $0, 1, 2, 3, \dots$.

On formalise cette situation en généralisant tous les cas possibles d'égalité sémantiques par des axiomes dits *équationnels*, qui sont des égalités générales entre deux expressions avec des variables. Par exemple, l'égalité générale

$$x * y = y * x$$

où x et y sont des variables formalise une propriété arithmétique de la multiplication, appelée *commutativité*, qui rend par exemple égales les valeurs des expressions $(2 * 3)$ et $(3 * 2)$. Toutes les autres propriétés arithmétiques de l'addition et de la multiplication peuvent être exprimées par de telles formules.

Lorsque la propriété

$$\forall x, y. x * y = y * x$$

est admise, elle est appelée *axiome de commutativité* (de la multiplication, car il peut y avoir d'autres symboles binaires commutatifs).

Une théorie est dite *équationnelle* si tous ses axiomes sont des égalités générales, comme cet exemple d'axiome de commutativité.

Lorsqu'on ajoute ainsi de tels axiomes équationnels, le symbole $=$ ne représente plus l'égalité syntaxique, mais une égalité sémantique, qui impose des contraintes aux fonctions et relations qui interprètent les symboles fonctionnels et relationnels.

Plus on ajoute d'axiomes, plus le raisonnement est complexe et difficile, voire impossible, à automatiser. Identifier des "petites" théories et adapter le raisonnement à leur contenu sont deux activités importantes pour maximiser l'automatisation du raisonnement et son efficacité pratique. La suite de ce chapitre présente des exemples de telles théories utiles en preuve de programmes, parfois directement telles qu'elles sont (bien) définies dans la plateforme Why3, qui est conçue pour faciliter la vérification déductive et maximiser son automatisation.

5.2.4 L'égalité dans Why3

Pour tout type défini dans le langage WhyML de Why3, qu'il soit prédéfini ou défini par l'utilisateur, un prédicat binaire d'égalité `=` est fourni, sans que l'utilisateur ait à le définir. Il correspond à l'égalité syntaxique, grâce à une formalisation correcte et complète par des règles de déduction spécifiques. On parle dans ce cas de “logique avec égalité”.

5.3 Théorie des booléens de Why3

Nous avons vu que le langage WhyML inclut un type `bool` pour les booléens, qui est un type énuméré avec deux habitants, qui sont les deux constantes `True` et `False`. Ce langage fournit aussi des symboles fonctionnels et relationnels sur ce type.

Comme tous les types WhyML, le type `bool` admet comme symbole relationnel un prédicat binaire d'égalité `=`, en notation infixée.

Le langage WhyML propose le connecteur logique `||`, appelé *disjonction asymétrique* et le connecteur logique `&&`, appelé *conjonction asymétrique*, qui ont la même signification logique que les connecteurs logiques $\vee$ et $\wedge$ et s'écrivent comme ces derniers en position infixée, mais qui sont différents sur deux points : Premièrement, les connecteurs logiques `||` et `&&` peuvent aussi être considérés comme des symboles fonctionnels binaires sur le type `bool`. Deuxièmement, dans un calcul, le deuxième argument de `||` (resp. `&&`) n'est pas évalué si son premier argument est évalué à `True` (resp. `False`), car ceci suffit à déterminer la valeur de vérité de l'expression complète.

De plus, le connecteur logique `not` de la négation peut aussi être considéré comme un symbole fonctionnel unaire sur le type `bool`.

5.4 Type des entiers relatifs

Le type $\mathbb{Z}$ des entiers relatifs est représenté en WhyML par un type prédéfini nommé `int`. Toutes les séquences (finies) non vides de caractères 0 à 9 (resp. précédées d'un signe `-`) peuvent être utilisées pour représenter un entier positif (resp. négatif) de ce type, comme illustré par quelques exemples dans le listing 5.1. Certains de ces buts utilisent le prédicat d'égalité (`=`), qui est prédéfini.

Cependant, cette mini-théorie ne contient aucun symbole fonctionnel et aucun autre symbole relationnel que l'égalité. Heureusement, la bibliothèque standard de Why3 contient un fichier, nommé `int.mlw`, qui définit et nous fournit plusieurs théories utiles sur les entiers. Sa documentation est disponible à l'URL <http://www.why3.org/stdlib/int.html>.

Dans le langage WhyML, chaque théorie est définie dans un *module* qui commence par `module nomDeLaTheorie` et se termine par le mot-clé `end`.

Dans le fichier `int.mlw`, la théorie principale s'appelle `Int`. Elle est définie par les lignes de code entre `module Int` et le mot-clé `end` le plus proche. Cette théorie définit, entre autres le symbole unaire `-` pour l'opposé d'un entier, les symboles fonctionnels binaires `+`, `-` et `*` pour l'addition, la soustraction et la multiplication, respectivement, et les relations d'ordre strictes et

```

val constant c : int
val function f int int : int
val function g int : int

val predicate r int int

goal q1: forall x:int. x = x
goal q2: r (f (g c) (-3)) (g 0) → r (f (g c) (-3)) (g 0)
goal q3: 000032 = 032 = 32 ∧ -0 = 0 ∧ -045 = -45

```

Listing 5.1 – Exemples de syntaxe WhyML sur les entiers relatifs.

larges $<$, $>$, $\leq$ et $\geq$, notées $<$, $>$, $\leq$ et $\geq$ dans nos listings. Cette théorie contient aussi, entre autres, tous les axiomes d'un anneau commutatif.

Toute utilisation de ces symboles et axiomes doit être précédée par la déclaration suivante, qui importe ce module :

```
use int.Int
```

Le listing 5.2 montre des exemples de buts qu'on peut écrire et démontrer automatiquement à propos des entiers.

```

use int.Int

lemma i1: 2 + 3 - 1 = 4
lemma i2: 17 - 3 = 17 + (-3)
lemma q2: forall x:int. x < x+1
lemma q3: exists x:int. x ≤ c

```

Listing 5.2 – Exemples de lemmes sur les entiers de Why3, avec la théorie **Int**.

D'autres modules du fichier `int.mlw` de la bibliothèque standard de Why3 étendent cette théorie des entiers. Par exemple, le module **EuclideanDivision** axiomatise les fonctions **div** et **mod**, respectivement pour le quotient et le reste de la division euclidienne, comme illustré par le listing 5.3.

```

use int.EuclideanDivision

lemma i3: 47 - ((div 47 3) * 3) = mod 47 3

```

Listing 5.3 – Exemple d'utilisation du quotient et du reste euclidiens en WhyML.

Ces théories sur les entiers seront souvent utiles lorsqu'on utilisera le type `array int` pour les tableaux d'entiers, présenté dans la partie 5.5.

5.5 Théorie des tableaux d'entiers

La déclaration `use array.Array` importe une théorie des tableaux polymorphes, que nous utiliserons principalement pour définir des tableaux d'entiers, de type `array int`. Tout tableau `a` a une longueur fixe, notée `a.length`. Un tableau `a` de type `array int` associe un entier relatif `a[i]` à chaque index `i` entre 0 et `a.length-1` inclus. L'expression `make n v` construit un tableau de longueur `n` dont tous les éléments sont égaux à `v`. L'instruction `a[i] ← v` permet de donner la valeur `v` à l'élément d'index `i` du tableau `a`.

Le listing 5.4 montre quelques exemples de syntaxe de propriétés d'un tableau d'entiers. Comme documenté dans la page

<http://www.why3.org/stdlib/array.html>,

d'autres prédicats sont disponibles dans la bibliothèque standard de Why3, par exemple dans les modules nommés `array.ArrayEq`, `array.ArrayExchange`, `array.ArrayPermut` et `array.IntArraySorted`.

```
use array.Array
use array.IntArraySorted

predicate cte (a:array int) (c:int) =
  forall i:int. 0 ≤ i < a.length → a[i] = c

predicate is_inj (a:array int) =
  forall i j:int. 0 ≤ i < j < a.length → a[i] ≠ a[j]

lemma cte_sorted: forall a:array int, c:int. cte a c → sorted a
```

Listing 5.4 – Exemples de formules WhyML sur les tableaux d'entiers.

5.6 Conclusion de ce chapitre

Après avoir présenté la notion théorique de théorie, définie par une signature et des axiomes, ce cours montre comment utiliser des théories prédéfinies dans l'outil Why3. Il suffit de s'inspirer des exemples de théories présentés pour écrire ses propres théories.

Chapitre 6

Logique de Hoare

Ce chapitre définit un mini-langage de programmes impératifs et une logique pour raisonner sur la correction de ces programmes, lorsqu'ils sont spécifiés par une précondition et une postcondition.

Historiquement, la logique de Hoare, conçue en 1969 [Hoa69], est la première présentation formelle de la vérification déductive. Nous l'étudions non seulement pour son intérêt historique, mais aussi parce qu'elle permet de mieux comprendre la démarche de raisonnement sur les programmes, et parce que ses améliorations ultérieures sont implémentées dans les outils de preuve de programmes.

L'objectif de ce chapitre est de s'appropriier la logique de Hoare, pour effectuer formellement des vérifications (appelées “preuves”) de programmes. La logique de Hoare étant un cas particulier de système formel déductif, composé d'un ensemble fini de règles de déduction, les notions générales de système formel et de preuve formelle au sein d'un tel système sont préalablement exposées.

Plan du chapitre

La partie 6.1 présente la syntaxe du mini-langage de programmation impérative considéré. La partie 6.2 décrit les restrictions de ce langage par rapport à des langages de programmation connus. La partie 6.3 présente la sémantique du langage de programmation et la partie 6.4 donne quelques exemples de programmes écrits dans ce mini-langage. Les parties 6.5 et 6.6 présentent respectivement la notion de système formel et de preuve dans un tel système. La partie 6.7 discute deux manières de présenter les preuves. La partie 6.8 décrit le système formel appelé logique de Hoare. La partie 6.9 montre un exemple de preuve dans cette logique. La partie 6.10 donne quelques informations sur ses propriétés.

6.1 Syntaxe du langage de programmation

Ce langage est appelé langage des *tantque programmes*.

Soit $\langle \text{Var} \rangle$ un ensemble de noms de variables. Les types de ces variables sont `int` pour les entiers relatifs et `bool` pour les booléens. Pour simplifier nous ne prenons pas en compte des données de type caractère ou chaîne de caractères. Les seules structures de données prises en compte sont les tableaux, dont l'utilisation est légèrement restreinte dans les expressions.

Soit $\langle \text{Expr} \rangle$ l'ensemble des expressions qu'on peut former avec ces types de variables, les parenthèses et les opérateurs suivants munis des priorités habituelles : $+$, $-$, $*$, div et mod sur les entiers, `non`, `et` et `ou` sur les booléens, $<$, $>$, $=$, $\neq$, $\geq$, $\leq$ comme opérateurs de comparaison entre valeurs de même type (entier ou booléen).

Définition 1 (Instructions des tantque programmes) Une instruction est un élément de l'ensemble $\langle \text{Instruction} \rangle$ composé des quatre sortes d'instructions suivantes :

- $\langle \text{Var} \rangle := \langle \text{Expr} \rangle$, appelée affectation,
- `if` $\langle \text{Expr} \rangle$ `then` $\langle \text{Séquence d'Instructions} \rangle$ `fi`, appelée conditionnelle simple,
- `if` $\langle \text{Expr} \rangle$ `then` $\langle \text{Séquence d'Instructions} \rangle$ `else` $\langle \text{Séquence d'Instructions} \rangle$ `fi`, appelée conditionnelle double,
- `while` $\langle \text{Expr} \rangle$ `do` $\langle \text{Séquence d'Instructions} \rangle$ `od`, appelée itération, répétition ou “boucle”.

Une séquence d'instructions (appartenant à l'ensemble $\langle \text{Séquence d'Instructions} \rangle$) est de la forme $A_1; A_2; \dots; A_n$ où $A_1, A_2, \dots, A_n$ sont des instructions (appartenant à l'ensemble $\langle \text{Instruction} \rangle$).

Définition 2 (tantque programmes) Un tantque programme est une séquence d'instructions.

Les programmes peuvent être commentés par des textes compris entre “(” et “)” ou entre “/*” et “*/”.

Exemple 1 (Tantque programmes) Voir les premiers exemples de programmes complets dans la partie 6.4 de ce chapitre.

La syntaxe des programmes peut être décrite de manière synthétique par la grammaire formelle suivante :

$c, c' ::=$	$ x := e$	affectation
	$ c; c'$	séquence
	$ \text{if } b \text{ then } c \text{ fi}$	conditionnelle simple
	$ \text{if } b \text{ then } c \text{ else } c' \text{ fi}$	conditionnelle double
	$ \text{while } b \text{ do } c \text{ od}$	répétition

Dans cette grammaire, x est une variable, e est une expression (du même type que x) et b est une expression booléenne.

Ces programmes sont aussi appelés *commandes* ou *instructions*.

Les mots clés “fi” et “od” permettent d'éviter les confusions d'interprétation dans des commandes telles que

`if` b `then` $c; c'$, `if` b `then` c_1 `else` $c_2; c'$ `et` `while` b `do` $c_1; c'$

dans lesquelles on ne sait pas si la commande c' est exécutée après la commande conditionnelle ou de répétition, ou bien si elle fait partie de cette commande.

Ce langage des “tantque programmes” ajoute les instructions conditionnelles au langage de l'article historique de Hoare [Hoa69].

6.2 Restrictions du langage de programmation

Ce mini-langage est un sous-ensemble du langage impératif C et de la partie impérative du langage WhyML. Les principales restrictions sont :

- l'absence de données de type pointeur (donc pas de prise en compte de la notion d'objet),
- l'utilisation restreinte des tableaux (pas d'utilisation de plusieurs indices pouvant être égaux),
- l'absence d'appel de procédures.

Ces deux premières constructions invalident dans certains cas des règles de la logique de Hoare, à cause du problème de *synonymie* des variables, rendu possible par ces constructions. Sous certaines conditions, on peut étendre la théorie pour traiter ces constructions. C'est ce que font les environnements de preuve de programmes existants, comme Why3, Frama-C ou KeY.

Définition 3 (Variables synonymes) *Deux variables sont synonymes si elles désignent le même emplacement mémoire.*

Exemple 2 (Variables synonymes, les pointeurs) *Par exemple, l'affectation $p := q$ où p et q sont des pointeurs crée une synonymie entre la valeur pointée par p (notée p^* en C) et celle pointée par q . Les variables p^* et q^* sont synonymes.*

Le problème de la synonymie est le suivant : si une affectation (par exemple : $p^ := 4$ en C) a lieu alors non seulement celle-ci modifie la valeur désignée par p^* , mais également celle désignée par la variable synonyme q^* . Cette modification de q^* est totalement invisible dans l'affectation $p^* := 4$ car q^* n'apparaît pas dans l'instruction d'affectation $p^* := 4$. On dit que l'affectation $p^* := 4$ a un effet de bord sur q^* .*

Idée clé : Les variables synonymes invalident la règle d'affectation de la partie 6.8.

Les variables synonymes mettent en défaut le système de vérification car ce dernier utilise la substitution qui ne prend en compte que les modifications qui apparaissent explicitement dans le texte du *tantque* programme.

Exemple 3 (autres variables synonymes) *Les expressions de tableaux de la forme $t[i]$ et $t[j]$ permettent également de réaliser des synonymies si $i = j$. C'est pourquoi, dans les programmes avec des tableaux, nous n'autorisons plusieurs expressions de la forme $t[i]$ ou $t[j]$ qu'à condition que i soit différent de j .*

Exemple 4 (variables synonymes, substitution et invalidation de la règle d'affectation) *L'expression $\{p^* = 4 \wedge q^* = 2\}$ n'est pas vraie après exécution du programme*

$$p := q; q^* := 2; p^* := 4$$

alors qu'on peut démontrer que toutes les valeurs symboliques qui décorent ce programme ainsi :

$$\{\text{vrai}\} p := q; \{\text{vrai}\} q^* := 2; \{q^* = 2\} p^* := 4 \{p^* = 4 \wedge q^* = 2\}$$

sont vraies par substitution. L'erreur vient du fait que l'instruction $q^* := 2$ n'engendre une substitution que sur q^* alors que cette instruction modifie également la variable synonyme p^* . Il en est de même pour le programme

$$i := 4; j := 4; t[i] := 5; t[j] := 4$$

qui permet d'établir par substitution

$$\{\text{vrai}\} i := 4; j := 4; t[i] := 5; t[j] := 4 \{t[j] = 4 \wedge t[i] = 5\}.$$

La substitution étant établie à partir de la syntaxe, elle ne répercute pas l'effet de bord de l'instruction sur les variables synonymes.

6.3 Sémantique du langage de programmation

Le langage des *tantque* programmes possède la sémantique habituelle des langages de programmation impératifs :

- $x := e$ signifie que la valeur de l'expression e est affectée à la variable x ,
- $c; c'$ signifie que la commande c est exécutée, puis la commande c' ,
- **if b then c fi** signifie que si l'expression booléenne b a pour valeur de vérité vrai, la commande c est exécutée. Sinon, aucune instruction n'est exécutée et l'exécution se poursuit par l'instruction qui suit la commande **if b then c fi**.
- **if b then c else c' fi** signifie que si l'expression booléenne b a pour valeur de vérité vrai, la commande c est exécutée, et sinon la commande c' est exécutée.
- **while b do c od** signifie que la commande c (appelée *corps de boucle*) est exécutée tant que l'expression booléenne b a pour valeur de vérité vrai. Lorsque b a pour valeur de vérité faux, l'exécution se poursuit par l'instruction qui suit le **while**.

6.4 Exemples de programmes

6.4.1 Factorielle

On veut calculer dans une variable f la factorielle, notée $n!$, d'un nombre entier n positif ou nul donné. Si $n = 0$ alors $f = 1$. Sinon $f = 1 * 2 * 3 * \dots * n$. Cette spécification est décrite dans la figure 6.1. La figure 6.2 décrit un programme qui calcule f égal à la factorielle de n selon sa définition donnée dans la figure 6.1. Noter que ce programme utilise la donnée n pour laquelle on fait l'hypothèse qu'elle satisfait la précondition posée dans la figure 6.1.

6.4.2 Recherche dichotomique

Ce problème consiste à calculer la position $posx$ où insérer un élément x dans une séquence ordonnée de n entiers, représentée par les éléments d'un tableau t entre les indices 1 et n , $t[1..n]$

Données : n : entier

Résultat : f : entier, égal à la factorielle de n .

Précondition : $n \geq 0$

Postcondition : $f = \prod_{j=1}^n j$

FIGURE 6.1 – Spécification de la factorielle.

```
 $i := 0; f := 1; \quad \quad \quad /* f = 0! */$   
while  $i \neq n$  do  
   $i := i + 1;$   
   $f := f * i \quad \quad \quad /* f = i! */$   
od  
 $/* i = n \text{ et } f = i!, \text{ donc } f = n! */$ 
```

FIGURE 6.2 – Programme de calcul de la factorielle.

étant trié dans l'ordre croissant. La spécification de ce problème est décrite dans la figure 6.3. La figure 6.4 décrit un programme de calcul de $posx$ par recherche dichotomique de x dans $t[1..n]$.

Constante : $MAX = 100$, nombre maximum d'éléments dans le tableau.

Données :

t : tableau de n entiers classés en ordre croissant,

x : entier dont on recherche la position d'insertion dans la séquence $t[1..n]$.

Résultat : $posx$: entier compris entre 1 et $n + 1$, position d'insertion de x dans t .

Précondition : t est un tableau éventuellement vide trié en ordre croissant. Ceci est exprimé formellement ainsi :

$$n \geq 0 \wedge n \leq MAX \wedge \forall i. \forall j. ((i \in [1..n] \wedge j \in [1..n] \wedge i < j) \Rightarrow t[i] \leq t[j])$$

Postcondition : tous les éléments de t de position comprise entre 1 et $posx - 1$ sont inférieurs ou égaux à x , ceux de position comprise entre $posx$ et n sont supérieurs à x . Ceci s'exprime formellement ainsi :

$$\forall k. (k \in [1..posx - 1] \Rightarrow t[k] \leq x) \wedge \forall k. (k \in [posx..n] \Rightarrow t[k] > x)$$

FIGURE 6.3 – Spécification de la recherche dichotomique.

Le programme de la figure 6.4 calcule le bon résultat car à chaque étape de l'itération il établit une certaine proposition I . Les variables sont :

m : milieu de l'intervalle $min..max$.

min : indice tel que $t[1..min]$ est composé d'éléments inférieurs ou égaux à x

max : indice tel que $t[max + 1..n]$ est composé d'éléments supérieurs à x .

Si ce programme termine, la proposition I étant vraie, on en déduit facilement que le résultat est $min + 1$ ou $max + 1$. Par contre pour justifier que le programme s'arrête toujours, c'est plus difficile. En effet, pour qu'il s'arrête, il faut justifier qu'à chaque étape, soit min augmente,

```

min := 0; max := n;
/* I est t[j] ≤ x pour j ∈ [1..min] et t[j] > x pour j ∈ [max + 1..n] */
while min ≠ max do
  /* min ≠ max et I */
  m := (min + max + 1) div 2;
  if t[m] ≤ x then min := m else max := m - 1 fi
  /* I */
od
/* min = max et I, donc la position d'insertion et min + 1 ou max + 1 qui sont l'indice du
premier élément de t supérieur à x */
posx := min + 1

```

FIGURE 6.4 – Programme de recherche dichotomique.

soit max diminue de telle sorte que $min = max$ devienne vrai, c'est-à-dire que l'intervalle de recherche $min + 1..max$ devienne vide. Or le programme affecte min par m et max par $m - 1$. Autrement dit, étant donné le calcul de m , est-on sûr que m est toujours plus grand que min ou que $m - 1$ est toujours plus petit que max ?

Exercice 6.1 (arrêt de programme). Justifier que le programme de la figure 6.4 termine toujours.

6.4.3 Tri à bulle

Ce problème consiste à classer par ordre croissant une séquence de n entiers, représentée par les éléments d'un tableau tr entre 1 et n , à partir d'une séquence représentée par le tableau td , composé des mêmes éléments dans un ordre quelconque. Soit MAX le nombre maximum d'éléments des tableaux td et tr .

La spécification est dans la figure 6.5 et le programme dans la figure 6.6. Dans ce programme, i est un indice tel que $t[1..i]$ est trié et j est un indice de parcours de t .

Constante : $MAX = 100$, nombre maximum d'éléments dans le tableau.

Données : td : tableau de n entiers classés dans un ordre quelconque, n : nombre de valeurs dans td .

Résultat : tr : tableau de n entiers classés en ordre croissant.

Précondition : td est un tableau éventuellement vide. Formellement : $0 \leq n \leq max$.

Postcondition : tr est un tableau trié par ordre croissant et contient les éléments de td . Ceci s'exprime formellement ainsi :

$$\forall k. \forall u. k \in [1..n] \wedge u \in [1..n] \wedge k < u \Rightarrow tr[k] \leq tr[u] \wedge ran(tr) = ran(td)$$

FIGURE 6.5 – Spécification du tri à bulle.

```

i := 0; tr := td;           /* copie des éléments du tableau */
/* L'invariant I est tr[1..i] trié et tr[j] ≥ tr[i] pour j ∈ [i + 1..n] */
while i < n - 1 do
  /* i < n - 1 et I */
  /* placement du minimum de tr[i + 1..n] en position i + 1 */
  j := n;
  /* I' est tr[j] est le minimum de tr[j..n] et tr[j] ≥ tr[i] */
  while j - 1 > i do
    /* j > i et I' */
    if tr[j] < tr[j - 1] then           /* échanger tr[j] et tr[j - 1] */
      aux := tr[j]; tr[j] := tr[j - 1]; tr[j - 1] := aux
    fi;
    j := j - 1;
  /* I' */
  od;
  /* j - 1 = i et I' donc tr[1..i + 1] trié. Fin du placement du minimum */
  i := i + 1
od
/* i = n - 1 et I, donc tr[1..n] trié */

```

FIGURE 6.6 – Programme du tri à bulle.

6.5 Notion de système formel

Définition 4 (système formel, règle de déduction, axiome) *Un système formel est un couple $\langle L, R \rangle$ où L est un langage définissant un ensemble de formules et R est un ensemble de règles de déduction de formules de L à partir d'autres formules de L .*

La règle de déduction qui permet de déduire la formule C à partir des formules $P_1, \dots, P_n$ est notée

$$\frac{P_1, \dots, P_n}{C}$$

et se lit “à partir de P_1 et ... et P_n , on déduit C ”. Dans cette règle, les formules $P_1, \dots, P_n$ sont appelées prémisses et la formule C est appelée conclusion.

Un axiome est une règle de déduction sans prémisses ($n = 0$).

Exemple 5 (système formel) *Le système formel LP est le couple $\langle L, R \rangle$ où L est le langage des formules propositionnelles et R est donné dans la figure 6.7. Les 10 premières règles sont des axiomes. La dernière règle s'appelle “modus ponens”, abrégé en m.p..*

6.6 Notions de preuve et de théorème

Définition 5 (preuve) *Une preuve ou démonstration (formelle) de la formule f_k dans le système formel $\langle L, R \rangle$ est une séquence $f_1, f_2, \dots, f_k$ de formules du langage L telle que, pour tout i*

$$\begin{array}{c}
\overline{P \Rightarrow (Q \Rightarrow P)} \text{ (Ax. 1)} \quad \overline{(P \Rightarrow Q) \Rightarrow ((P \Rightarrow (Q \Rightarrow R)) \Rightarrow (P \Rightarrow R))} \text{ (Ax. 2)} \\
\overline{P \Rightarrow (Q \Rightarrow P \wedge Q)} \text{ (Ax. 3)} \quad \overline{P \wedge Q \Rightarrow P} \text{ (Ax. 4)} \quad \overline{P \wedge Q \Rightarrow Q} \text{ (Ax. 5)} \\
\overline{P \Rightarrow P \vee Q} \text{ (Ax. 6)} \quad \overline{Q \Rightarrow P \vee Q} \text{ (Ax. 7)} \\
\overline{(P \Rightarrow R) \Rightarrow ((Q \Rightarrow R) \Rightarrow (P \vee Q \Rightarrow R))} \text{ (Ax. 8)} \\
\overline{\neg \neg P \Rightarrow P} \text{ (Ax. 9)} \quad \overline{(P \Rightarrow Q) \Rightarrow ((P \Rightarrow \neg Q) \Rightarrow \neg P)} \text{ (Ax. 10)} \quad \frac{P, P \Rightarrow Q}{Q} \text{ (m.p.)}
\end{array}$$

FIGURE 6.7 – Règles du système LP

appartenant à $[1..k]$, f_i est déduit à partir de certaines formules parmi $f_1, \dots, f_{i-1}$ en utilisant une règle de R .

Idée clé (preuve) : La notion de preuve est formelle, au sens où le mécanisme est totalement syntaxique. Une formule f est déduite, soit à partir des axiomes, qui sont considérés comme les vérités de base, soit en appliquant des règles uniquement à partir de formules précédentes déjà prouvées.

Définition 6 (théorème) *Toute formule f qui apparaît dans une preuve est un théorème.*

Sur une ligne, la règle de déduction

$$\frac{P_1, \dots, P_n}{C}$$

s'écrit $P_1, \dots, P_n \vdash C$. La règle se lit : “ C se déduit de P_1 et ... et P_n ” ou “Si P_1 et ... et P_n , alors C ”.

Le symbole de la déduction $\vdash$ sert aussi à énoncer des théorèmes : $H_1, \dots, H_n \vdash C$ veut dire que C est un théorème sous les hypothèses $H_1, \dots, H_n$. La notation $\vdash E$ veut dire que E est un théorème sans hypothèses. En logique de Hoare, on ne démontre que des théorèmes sans hypothèses.

6.7 Comment présenter les preuves ?

- En général, la présentation d'une preuve (formelle) d'un théorème par arbre est un arbre dont
 - la racine est la conclusion du théorème,
 - chaque nœud interne (non feuille) est une déduction qui instancie une règle du système déductif, et

— chaque feuille est une hypothèse du théorème.

Un *axiome* étant une règle de déduction sans prémisse, son utilisation dans un arbre de preuve ne produit pas de feuille.

Exemple 6 (arbre de preuve dans le système LP) Une présentation par arbres d'une démonstration formelle du théorème $A, \neg A \vdash B$ dans le système LP est :

$$\begin{array}{c}
 \frac{\neg A \quad \overline{\neg A \Rightarrow (\neg B \Rightarrow \neg A)} \text{ Ax. 1}}{\neg B \Rightarrow \neg A} \text{ m.p.} \quad (1) \\
 \hline
 \frac{\neg \neg B \quad \overline{\neg \neg B \Rightarrow B} \text{ Ax. 9}}{B} \text{ m.p.}
 \end{array}$$

où (1) est le sous-arbre

$$\begin{array}{c}
 \frac{A \quad \overline{A \Rightarrow (\neg B \Rightarrow A)} \text{ Ax. 1}}{\neg B \Rightarrow A} \text{ m.p.} \quad \overline{(\neg B \Rightarrow A) \Rightarrow ((\neg B \Rightarrow \neg A) \Rightarrow \neg \neg B)} \text{ Ax. 10} \\
 \hline
 (\neg B \Rightarrow \neg A) \Rightarrow \neg \neg B \text{ m.p.}
 \end{array}$$

Dans cet exemple, la numérotation (1) a permis de couper l'arbre de preuve en deux parties, afin de ne pas dépasser la largeur de la feuille.

On peut présenter la même preuve soit sous la forme d'un arbre, soit sous la forme d'un tableau.

Les arbres de preuve ne sont pas toujours facilement représentables sur une feuille de papier, car la longueur des formules et le nombre des prémisses rendent souvent l'arbre plus large que la feuille de papier. C'est pourquoi nous utiliserons également la présentation par tableau, linéaire, tout en raisonnant à partir du but comme nous le faisons en construisant un arbre de preuve.

Une preuve linéaire, dans un tableau, est une séquence de formules munies d'une référence et d'une justification.

Exemple 7 (preuve par tableau dans le système LP) Une présentation dans un tableau d'une démonstration du théorème $A, \neg A \vdash B$ dans le système LP est :

Numéro	Justification	Formule
1	Hypothèse 1	A
2	Axiome 1 ($A/P, \neg B/Q$)	$A \Rightarrow (\neg B \Rightarrow A)$
3	m.p. sur 1 , 2	$\neg B \Rightarrow A$
4	Hypothèse 2	$\neg A$
5	Axiome 1 ($(\neg A)/P, (\neg B)/Q$)	$\neg A \Rightarrow (\neg B \Rightarrow \neg A)$
6	m.p. sur 4 , 5	$\neg B \Rightarrow \neg A$
7	Axiome 10 ($(\neg B)/P, A/Q$)	$(\neg B \Rightarrow A) \Rightarrow ((\neg B \Rightarrow \neg A) \Rightarrow \neg \neg B)$
8	m.p. sur 3 , 7	$(\neg B \Rightarrow \neg A) \Rightarrow \neg \neg B$
9	m.p. sur 6 , 8	$\neg \neg B$
10	Axiome 9	$\neg \neg B \Rightarrow B$
11	m.p. sur 9 , 10	B

Dans une telle preuve par tableau, la conclusion en général la dernière formule dans le tableau, tout en bas. Mais il est tout à fait correct et souvent plus pratique d'inverser l'ordre des formules dans le tableau. Ceci permet de construire le tableau dans le même ordre que l'arbre, en commençant par la conclusion de la preuve, qui est alors la première formule du tableau. Plus généralement, l'ordre des formules dans le tableau n'a aucune importance, puisque toute la structure déductive est codée grâce aux numéros (première colonne) et aux justifications (deuxième colonne).

6.8 Logique de Hoare

La logique de Hoare axiomatise la correction partielle d'un programme par rapport à une précondition et une postcondition, c'est-à-dire sa correction sous l'hypothèse que ce programme termine. Cette logique proposée par Hoare [Hoa69] est parfois appelée "logique de Floyd-Hoare" car elle doit beaucoup au travail antérieur de Floyd [Flo67].

La condition de correction partielle du programme c par rapport à la précondition P et à la postcondition Q est notée $\{P\} c \{Q\}$. Cette notation est appelée "triplet de Hoare". Le triplet de Hoare

$$\{P\} c \{Q\}$$

formalise la proposition suivante :

"L'exécution du programme c à partir d'un état qui satisfait la précondition P soit ne termine pas, soit termine dans un état qui satisfait la postcondition Q ."

Tout triplet de Hoare pour lequel cette proposition est vraie est appelé "triplet de Hoare valide". Parmi toutes les propositions syntaxiquement correctes, la logique de Hoare caractérise celles qui sont vraies, donc les triplets de Hoare valides, par un système d'inférence donné dans la définition 7 et expliqué dans la suite de cette partie.

Définition 7 (logique de Hoare) La logique de Hoare est le système formel $\langle L, R \rangle$ où

- L est l'ensemble des formules du premier ordre et des formules de la forme $\{P\} c \{Q\}$, appelées triplets de Hoare, où P et Q sont des formules de la logique du premier ordre et c est un programme du mini-langage de la partie 6.1, et
- R est l'ensemble de règles de déduction de la figure 6.8.

$$\begin{array}{c}
\frac{}{\{P(e/x)\} x := e \{P\}} \text{ (assignment)} \\
\\
\frac{\{P\} c \{Q\} \quad \{Q\} c' \{R\}}{\{P\} c; c' \{R\}} \text{ (sequence)} \\
\\
\frac{\{b \wedge P\} c \{Q\} \quad (\neg b \wedge P) \Rightarrow Q}{\{P\} \text{ if } b \text{ then } c \text{ fi } \{Q\}} \text{ (if-then)} \\
\\
\frac{\{b \wedge P\} c_1 \{Q\} \quad \{\neg b \wedge P\} c_2 \{Q\}}{\{P\} \text{ if } b \text{ then } c_1 \text{ else } c_2 \text{ fi } \{Q\}} \text{ (if-then-else)} \\
\\
\frac{\{b \wedge P\} c \{P\}}{\{P\} \text{ while } b \text{ do } c \text{ od } \{\neg b \wedge P\}} \text{ (while)} \\
\\
\frac{P \Rightarrow P' \quad \{P'\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}} \text{ (consequence)}
\end{array}$$

FIGURE 6.8 – Règles de la logique de Hoare.

Noter qu'il y a exactement une règle pour chaque sorte d'instructions dans le langage de programmation, et une règle complémentaire nommée (*consequence*). Ces règles permettent de démontrer des théorèmes sur des programmes composés de plusieurs instructions, à partir de théorèmes sur des programmes plus simples, comme illustré plus loin par des exemples. La suite de cette partie commente ces règles une par une.

6.8.1 Règle pour l'affectation

La règle

$$\frac{}{\{P(e/x)\} x := e \{P\}} \text{ (assignment)}$$

pour l'instruction d'affectation n'a pas de prémisses, c'est un axiome. Dans cette règle (e/x) formalise la substitution (remplacement uniforme) de x par e . Ainsi, $P(e/x)$ désigne la formule obtenue en remplaçant par e chaque occurrence libre de la variable x dans P .

On peut justifier la correction de cette règle comme suit : L'affectation $x := e$ ne modifie l'état qu'en donnant la valeur de l'expression e à la variable x . Par conséquent, la formule P (portant sur x) est vraie après exécution de la commande $x := e$ si la formule $P(e/x)$ est vraie avant cette affectation.

Exemple 8 (substitution dans une formule) Par exemple, la formule

$$(x < y \wedge (\forall x. x \in [1..n] \Rightarrow t[x] = 3)) (x + 1/x)$$

est égale à la formule

$$x + 1 < y \wedge (\forall x. x \in [1..n] \Rightarrow t[x] = 3)$$

qui est obtenue en substituant la seule occurrence libre de x qui est dans $x < y$ par $x + 1$. Les autres occurrences sont liées par le quantificateur universel et ne sont pas substituées car le prédicat aurait pu être noté ainsi, par renommage de la variable liée, de telle sorte qu'il n'y ait plus d'ambiguïté :

$$x < y \wedge (\forall z. z \in [1..n] \Rightarrow t[z] = 3).$$

6.8.2 Règle pour la composition séquentielle

Voici un exemple d'application de la règle (*sequence*) : à partir des triplets

$$\{n \geq 0^2 \wedge n < (n + 1)^2\} \text{ } rmin := 0 \{n \geq rmin^2 \wedge n < (n + 1)^2\}$$

et

$$\{n \geq rmin^2 \wedge n < (n + 1)^2\} \text{ } rmax := n + 1 \{n \geq rmin^2 \wedge n < rmax^2\}$$

qui se démontrent avec la règle (*assignment*) pour l'affectation, on peut démontrer le théorème

$$\{n \geq 0^2 \wedge n < (n + 1)^2\} \text{ } rmin := 0; \text{ } rmax := n + 1 \{n \geq rmin^2 \wedge n < rmax^2\}$$

en appliquant la règle (*sequence*). Noter que le prédicat après l'instruction $rmin := 0$ doit être le même que le prédicat avant l'instruction $rmax := n + 1$. Cette contrainte est indiquée dans la règle par le prédicat Q satisfait après c et avant c' .

6.8.3 Règles pour l'instruction conditionnelle

Dans les règles (*if-then*) et (*if-then-else*), on suppose que l'expression booléenne b est aussi une formule (portant sur les variables d'état). Sinon, si le langage des formules n'inclut pas le langage des expressions booléennes, il suffit de remplacer dans ces règles b par la formule $b = true$ et $\neg b$ par la formule $b = false$. Cette remarque vaut aussi pour b dans la règle (*while*).

6.8.4 Règle pour l'instruction itérative

Dans la règle

$$\frac{\{b \wedge P\} c \{P\}}{\{P\} \text{ while } b \text{ do } c \text{ od } \{\neg b \wedge P\}} \quad (\text{while})$$

la formule P est un *invariant de boucle*. La prémissse $\{b \wedge P\} c \{P\}$ se lit “une itération de la boucle préserve l’invariant” ou “le corps de boucle préserve l’invariant”. La règle complète se lit “Si l’invariant est satisfait au début de la boucle (précondition P) et si une itération quelconque de la boucle préserve l’invariant (prémisse $\{b \wedge P\} c \{P\}$), alors (si la boucle termine) l’état après la boucle satisfait la négation de la condition de boucle (postcondition $\neg b$) et l’invariant (postcondition P)”.

Ainsi, **après la boucle, l’invariant est encore vrai** (et la condition de boucle est fausse).

Il faut **deviner** cet invariant P , et ce n’est pas toujours facile. Il n’existe pas d’algorithme général pour calculer P . Trouver un “bon” invariant fait partie des compétences à acquérir dans le cadre de ce module.

6.8.5 Règle de conséquence

Intuitivement, la règle

$$\frac{P \Rightarrow P' \quad \{P'\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}} \quad (\text{consequence})$$

permet de “recoller entre eux des morceaux de preuve”. La partie suivante montre le cas le plus fréquent dans lequel cette règle est indispensable pour continuer la démonstration d’un triplet de Hoare.

6.8.6 Notion d’invariant de boucle

Un invariant (de boucle) sert à raisonner sur une boucle dont le nombre d’itérations est soit inconnu soit connu mais trop grand pour qu’on puisse remplacer la boucle par une suite d’instructions conditionnelles `if ... then ... else ... fi`. Un invariant de boucle est une formule qui doit être vraie avant la boucle et préservée par chaque itération de la boucle, ce qui implique que cet invariant est encore vrai après la boucle, ce qui permet d’en démontrer une postcondition.

Lorsqu’il s’agit de démontrer un triplet de Hoare de la forme $\{P\} \text{ while } b \text{ do } c \text{ od } \{Q\}$, il est rare que sa postcondition Q soit de la forme $\neg b \wedge P$. C’est pourtant nécessaire pour pouvoir appliquer la règle (*while*), puisque la conclusion de cette règle est $\{P\} \text{ while } b \text{ do } c \text{ od } \{\neg b \wedge P\}$. Si ce n’est pas le cas, on applique la règle (*consequence*) comme suit, en introduisant une proposition I qui permet de produire une prémisse (la deuxième) sur laquelle la règle (*while*) peut s’appliquer.

$$\frac{P \Rightarrow I \quad \{I\} \text{ while } b \text{ do } c \text{ od } \{\neg b \wedge I\} \quad \neg b \wedge I \Rightarrow Q}{\{P\} \text{ while } b \text{ do } c \text{ od } \{Q\}} \quad (\text{consequence})$$

Ensuite, l'application de la règle (*while*) à cette deuxième prémisse donne la déduction suivante :

$$\frac{\{b \wedge I\} c \{I\}}{\{I\} \text{ while } b \text{ do } c \text{ od } \{\neg b \wedge I\}} \text{ (while)}$$

Globalement, les trois prémisses qui restent à démontrer formalisent que la proposition I est un invariant de cette boucle :

1. La prémisse $P \Rightarrow I$ se lit “L’invariant est vrai avant la boucle”.
2. La prémisse $\{b \wedge I\} c \{I\}$ se lit approximativement “L’invariant est préservé par chaque itération de la boucle”. Plus précisément, ce triplet dit qu’on peut utiliser l’hypothèse que la condition de boucle b est vraie pour démontrer cette préservation.
3. La prémisse $\neg b \wedge I \Rightarrow Q$ se lit “La négation de la condition de boucle et l’invariant impliquent la postcondition initiale”. Après exécution de la boucle (si elle termine), la condition de boucle b est fausse. C’est une hypothèse qu’on peut aussi utiliser pour démontrer la postcondition Q .

6.9 Exemple de preuve en logique de Hoare

Les arbres suivants présentent une preuve formelle d’un triplet de Hoare pour le programme de calcul de la factorielle et sa spécification, présentés dans la partie 6.4.1.

$$\frac{\overline{\{n \geq 0\} i := 0 \{n \geq 0 \wedge i = 0\}} \text{ (assignment)}}{\{n \geq 0\} i := 0; f := 1; \text{ while } i < n \text{ do } i := i + 1; f := f * i \text{ od } \{f = n!\} \text{ (sequence)}} \quad (1)$$

où (1) est

$$\frac{\overline{\{n \geq 0 \wedge i = 0\} f := 1 \{n \geq 0 \wedge i = 0 \wedge f = 1\}} \text{ (assignment)}}{\{n \geq 0 \wedge i = 0\} f := 1; \text{ while } i < n \text{ do } i := i + 1; f := f * i \text{ od } \{f = n!\} \text{ (sequence)}} \quad (2)$$

où (2) est

$$\frac{\begin{array}{c} n \geq 0 \wedge i = 0 \wedge f = 1 \Rightarrow I, \\ \{I\} \text{ while } i < n \text{ do } i := i + 1; f := f * i \text{ od } \{\neg(i < n) \wedge I\}, \\ \neg(i < n) \wedge I \Rightarrow f = n! \end{array}}{\{n \geq 0 \wedge i = 0 \wedge f = 1\} \text{ while } i < n \text{ do } i := i + 1; f := f * i \text{ od } \{f = n!\} \text{ (consequence)}}$$

Pour l’invariant I on propose la formule $i \leq n \wedge f = i!$. Ensuite, on traite séparément les 3 prémisses de (2).

6.9.1 Invariant vrai avant la boucle

$$n \geq 0 \wedge i = 0 \wedge f = 1 \Rightarrow i \leq n \wedge f = i!$$

car

$$n \geq 0 \wedge i = 0 \Rightarrow i \leq n$$

et

$$1 = 0!$$

6.9.2 Une itération préserve l'invariant

$$\frac{\begin{array}{l} \{i < n \wedge f = i!\} i := i + 1 \{i \leq n \wedge f = (i - 1)!\}, \\ \{i \leq n \wedge f = (i - 1)!\} f := f * i \{i \leq n \wedge f = i!\} \end{array}}{\begin{array}{l} \{i < n \wedge i \leq n \wedge f = i!\} i := i + 1; f := f * i \{i \leq n \wedge f = i!\} \quad (sequence) \\ \{i \leq n \wedge f = i!\} \text{ while } i < n \text{ do } i := i + 1; f := f * i \text{ od } \{i \not< n \wedge i \leq n \wedge f = i!\} \quad (while) \end{array}}$$

Exercice 6.2. Justifier la correction les deux feuilles de cet arbre.

6.9.3 Négation de la condition de boucle et invariant impliquent la post-condition

$$\neg(i < n) \wedge i \leq n \wedge r = i! \Rightarrow f = n!$$

car

$$\neg(i < n) \wedge i \leq n \Rightarrow i = n.$$

6.10 Propriétés de la logique de Hoare

Un système formel est une modélisation mathématique d'un problème réel de décision. Dans notre cas, la logique de Hoare permet de décider si un programme c satisfait ou pas une spécification composée d'une précondition P et d'une postcondition Q , en prouvant que la formule $\{P\} c \{Q\}$ est un théorème de cette logique.

Cette modélisation doit donc être conforme au problème réel qu'elle modélise. Dans le cas de la logique de Hoare, le problème réel est celui de l'exécution d'un programme c sur une machine. La conformité a été démontrée par Hoare. Elle recouvre deux propriétés :

- la *correction*, qui signifie que si l’on démontre le théorème $\{P\} c \{Q\}$, alors l’exécution du programme c à partir de n’importe quelles données qui satisfont le prédicat P aboutit, si elle termine, à des résultats qui satisfont le prédicat Q ;
- la *complétude*, qui signifie que si le programme c aboutit à des résultats qui satisfont le prédicat Q à partir de n’importe quelles données qui satisfont le prédicat P , alors il existe une preuve du théorème $\{P\} c \{Q\}$ selon les règles de la logique de Hoare.

Ces deux propriétés garantissent que la vérification en utilisant la logique de Hoare correspond à l’interprétation du monde réel des machines, bien qu’elle soit un processus purement mécanique, qui fait complètement abstraction du sens des formules manipulées.

Néanmoins, ces démonstrations ne peuvent pas être totalement automatisées, car certaines étapes exigent de faire un choix dans un ensemble infini. C’est principalement le cas pour les instructions *while*, pour lesquelles il faut en général “deviner” un invariant, qui est une formule sur les variables du programme. Ceci requiert de choisir une formule dans l’ensemble souvent infini ou très vaste de toutes les formules possibles. Même s’il existe des outils de “synthèse d’invariants” dans certains cas, le problème général dépasse les limites théoriques ou pratiques de calculabilité des ordinateurs. Ainsi, les qualités humaines de perspicacité et d’accumulation d’expérience sont utiles pour trouver un bon invariant de boucle. De plus, une bonne compréhension des règles de la logique de Hoare et du concept d’invariant donne un point de vue plus précis et rigoureux sur les activités de conception d’algorithmes et de construction de programmes.

6.11 Résumé

Nous avons défini un langage de programmation simplifié permettant d’exprimer des programmes de calcul de valeurs et de collections de valeurs de type booléen et entier. Pour représenter les collections, nous utilisons des tableaux à condition de ne jamais avoir des éléments de tableau synonymes.

Pour raisonner sur ces programmes, nous avons préalablement défini un langage d’expressions logiques, dans le chapitre 3. Ces expressions nous permettent de spécifier ces programmes avec une précondition et une postcondition, pour former un triplet de Hoare, qui est une formule à démontrer.

Ainsi, vérifier qu’un programme c satisfait une spécification formée de la précondition P et de la postcondition Q consiste à prouver que la formule $\{P\} c \{Q\}$ est un théorème de la logique de Hoare. Nous avons vu que les axiomes et les règles de la logique de Hoare constituent un système formel correct et complet pour démontrer formellement qu’un triplet de Hoare est valide.

Notons qu’il s’agit ici de preuves de correction partielle. Pour prouver la correction totale, il faut aussi prouver la terminaison. Ce sujet sera abordé dans un prochain chapitre.

Chapitre 7

Spécifications et programmes WhyML

WhyML est le langage de spécification et de programmation de Why3. Nous en présentons un fragment proche du mini-langage du chapitre 6, suffisant pour programmer et vérifier de nombreux algorithmes. Pour simplifier, nous désignons encore ce fragment par “WhyML”.

Plan du chapitre

La partie 7.1 définit les expressions du langage WhyML. La partie 7.2 explique les notions de variable mutable et de référence pour modifier l’état de la mémoire lors de l’exécution d’un programme WhyML. La partie 7.3 présente les annotations qui permettent de spécifier formellement des programmes WhyML. Des exemples complets de programmes WhyML annotés sont donnés dans la partie 7.4.

7.1 Expressions

Le langage WhyML ne distingue pas des “expressions” et des “instructions”. Il n’a que des expressions, qui ont un type, parmi les types `unit`, `bool`, `int` et `array int` présentés dans le chapitre 4.

Les expressions vues dans le chapitre 3, dites “sans effet de bord”, ou “pures”, car leur évaluation ne modifie pas l’état de la mémoire du programme, sont appelées *termes*.

Les expressions du langage WhyML sont définies par la grammaire suivante :

e, e'	$::=$	t	(expression pure)
		begin e end	(bloc)
		$x := e$	(affectation)
		let $v = e$ in e'	(déclaration locale)
		if b then e else e'	(conditionnelle)
		while b do e done	(répétition)

où t est un terme, x est une variable *mutable* (voir partie 7.2) définie avant, v est un nouveau nom de variable introduit par la déclaration locale, et b est une expression de type `bool`. L’expression

begin *e* **end** permet de placer l'expression *e* dans un bloc, comme les accolades ouvrantes et fermantes (*{* et *}*) en C. Dans l'affectation, l'expression *e* doit avoir le même type que la variable *x*. Dans la conditionnelle, les expressions *e* et *e'* doivent avoir le même type, qui est le type de l'expression conditionnelle.

La déclaration locale **let** *v* = *e* **in** *e'* permet à la fois d'introduire une variable locale *v* et de lui donner une valeur, obtenue par évaluation de l'expression *e*. L'expression *e'* est appelée la *portée* de la variable *v*. C'est l'expression dans laquelle la variable *v* peut être utilisée.

La notation *e*; *e'* pour la notion de séquence est disponible comme raccourci de la déclaration locale **let** *v* = *e* **in** *e'*, lorsque *e* est de type **unit** et *v* n'est pas dans *e'*.

La conditionnelle simple

if *b* **then** *e*

est disponible comme raccourci et cas particulier de la conditionnelle

if *b* **then** *e* **else** *()*

lorsque *e* est de type **unit** (comme la constante *()*).

7.2 Variables mutables et références

WhyML est un langage fonctionnel (proche de ML¹) étendu avec des instructions impératives dont les effets de bord sont limités à la modification de variables dites *mutables*, ou appelées *références*, car on peut modifier leur valeur. Une référence est l'analogue d'un pointeur en C.

Les variables de type **array int** sont mutables. En revanche, les variables de type **unit**, **bool** ou **int** ne sont pas mutables. Pour disposer d'une variable modifiable de l'un de ces types, on utilise une référence. Dans ce but, on importe d'abord la théorie **ref.Ref** de la bibliothèque standard de Why3, avec la déclaration

use **ref.Ref**

Cette théorie définit le type **ref 'a** des références sur une donnée de type **'a**. Par exemple, **ref int** est le type des références sur une donnée de type **int**.

Si *e* est une expression de type **int**, alors l'expression

let **r** = **ref** *e* **in** ...

déclare une variable mutable **r** dont la valeur initiale est la valeur de l'expression *e*. Dans la portée de cette déclaration locale, l'expression

r := *e'*

permet de modifier la valeur référencée par la référence **r**, en lui donnant la valeur de l'expression *e'*. Dans cette portée, l'expression **!r** désigne la valeur courante référencée par la référence **r**. L'affectation **:=** et l'opérateur préfixé de déréférencement **!** sont deux fonctions définies dans la théorie **ref.Ref**.

1. [https://fr.wikipedia.org/wiki/ML_\(langage\)](https://fr.wikipedia.org/wiki/ML_(langage)).

Après avoir bien compris que $\mathbf{r}$ n'est pas une donnée du même type que e , mais une **référence** (un pointeur) sur une telle donnée, et qu'il faut écrire $!\mathbf{r}$ pour lire cette donnée, on évitera d'utiliser la syntaxe

```
let  $\mathbf{r}$  = ref  $e$  in ...
```

et on utilisera plutôt la syntaxe

```
let ref  $\mathbf{r}$  =  $e$  in ...
```

où le mot-clé **ref** est placé avec le signe $=$, car cette syntaxe permet d'écrire $\mathbf{r}$ au lieu $!\mathbf{r}$ chaque fois qu'on veut lire la donnée pointée par la référence $\mathbf{r}$.

Un exemple de chacune de ces deux syntaxes de déclaration locale est visible dans l'exemple 7.4.1.

Si une fonction $\mathbf{f}$ doit modifier un entier, la fonction pourra être déclarée comme suit, avec un paramètre $\mathbf{r}$ de type **ref int**.

```
let  $\mathbf{f}$  ( $\mathbf{r}$  : ref int)
```

Le contrat et la définition de cette fonction peuvent alors désigner par $!\mathbf{r}$ l'entier référencé par la référence $\mathbf{r}$. La syntaxe

```
let  $\mathbf{f}$  (ref  $\mathbf{r}$  : int)
```

permet plus légèrement d'écrire $\mathbf{r}$ au lieu de $!\mathbf{r}$ pour désigner la valeur référencée, dans le contrat et la définition de cette fonction.

7.3 Annotations WhyML

La spécification de programmes WhyML suit le paradigme des *contrats* introduit dans le langage Eiffel [MNM87]. Un contrat décrit le comportement d'un programme par des *clauses* formelles (parfois appelées *contraintes formelles*), afin de réduire le nombre d'erreurs dans les programmes. Le contrat précise ce qui doit être vrai à un moment donné de l'exécution d'un programme.

On peut distinguer trois sortes de clauses :

- Les *préconditions* spécifient l'état du programme avant son exécution.
- Les *postconditions* spécifient l'état du programme après son exécution.
- Les *invariants* spécifient la préservation d'une partie de l'état du programme durant une partie de son exécution.

Cette partie présente les principales clauses de contrat du langage WhyML, appelées *annotations* (formelles) car elles sont ajoutées au programme mais sont ignorées lors de son exécution, comme les commentaires.

L'exemple 7.4.1 montre où ces annotations doivent être écrites, par rapport au reste du code WhyML, plus précisément par rapport à l'entête de la fonction ou du début de l'instruction **while** à laquelle elles se rapportent.

7.3.1 Préconditions

Dans le contrat d'une fonction, une annotation

```
requires { $R$ }
```

spécifie que la *précondition* R doit être vérifiée par les paramètres de la fonction lorsqu'elle est appelée.

7.3.2 Postconditions

Une *postcondition* d'un programme décrit les états attendus après exécution du programme. Une annotation WhyML de la forme

ensures $\{E\}$

affirme que la *postcondition* E est vraie en sortie de fonction. Dans cette postcondition la variable dédiée **result** désigne la donnée retournée par la fonction.

Les expressions présentes dans ces postconditions sont toutes évaluées **après** exécution de la fonction. Or, il est souvent utile de pouvoir représenter dans ces postconditions la valeur de certaines expressions **avant** exécution de la fonction. Un exemple simple est une fonction qui incrémente un entier. On veut pourvoir exprimer dans une postcondition que la nouvelle valeur de cet entier est égale à son ancienne valeur augmentée de 1.

Pour décrire dans une postcondition la valeur d'une expression e **avant** exécution de la fonction, il faut écrire $(\text{old } e)$, c'est-à-dire entourer cette expression e par $(\text{old}$ et $)$. Par exemple, le listing 7.1 montre une postcondition qui exprime que le contenu d'une partie d'un tableau n'est pas modifiée. Dans cet exemple, les parenthèses pour **old** sont omises car inutiles, mais elles peuvent être utiles dans d'autres cas.

```
use int.Int
use ref.Ref
use array.Array

let init (a : array int) (ref l : int) (u : int) : unit
  requires { 0 ≤ l ≤ u < length a }
  ensures { forall i. 0 ≤ i < old l → a[i] = old a[i] }
=
  while l < u do
    invariant { 0 ≤ old l ≤ l ≤ u < length a }
    invariant { forall i. 0 ≤ i < old l → a[i] = old a[i] }
    variant { u+1-l }
    a[l] ← 0;
    l := l + 1
  done
```

Listing 7.1 – Exemple de postcondition avec **old**.

7.3.3 Contrat de boucle

Pour automatiser la preuve d'un programme avec boucles, il est presque toujours nécessaire de spécifier un invariant et un variant pour chaque boucle. Ils constituent le contrat de la boucle, comme détaillé dans cette partie.

7.3.3.1 Invariant de boucle

Une annotation

invariant { I }

immédiatement après

while ... **do**

déclare que la formule I est un *invariant (inductif)* de cette boucle, c'est-à-dire une propriété qui doit être établie avant exécution de la boucle et préservée à chaque passage dans la boucle.

7.3.3.2 Variant de boucle

Si l'on ne souhaite pas démontrer la terminaison d'une fonction, il suffit de déclarer qu'elle peut ne pas terminer, en ajoutant le mot **diverges** après le type de la fonction, comme dans l'exemple de la factorielle dans la partie 7.4.1. Sinon, pour justifier la terminaison d'une boucle, on peut ajouter une annotation

variant { V }

après

while ... **do**

ou après les invariants de cette boucle. Cette clause définit un *variant de boucle* V . Cette expression V doit être de type entier, doit rester positive ou nulle pendant toute l'exécution de la boucle et doit décroître strictement entre deux itérations successives de la boucle.

Le listing 7.3 donne un premier exemple de variant.

7.3.3.3 Assertions

Une annotation

assert { A }

en un point d'un programme, appelée *assertion*, déclare que la propriété A est vraie lors de toute exécution qui atteint cette ligne du programme.

7.3.4 Vérification déductive avec Why3 en ligne

Après avoir défini un programme Why3 et annoté ce programme avec un contrat, la conformité entre contrat et programme peut être vérifiée à l'aide de l'interface en ligne à l'adresse <https://www.why3.org/try/>, en cliquant sur l'icône  qui représente une baguette magique.

7.4 Exemples

Cette partie présente quelques exemples de code WhyML annoté, dont la vérification est automatique avec l'interface en ligne <https://www.why3.org/try/>. Ce code WhyML est parfois commenté par du texte compris entre “(*)” et “(*)”.

7.4.1 Factorielle

Les listings 7.2 et 7.3 présentent une adaptation à WhyML du programme de calcul de la factorielle du chapitre 6. La postcondition utilise une fonction prédéfinie dans le module `int.Fact`. La fonction `main` permet de calculer une factorielle, en cliquant sur l'icône ► de l'interface en ligne <https://www.why3.org/try/> qui représente un triangle équilatéral dont une pointe est orientée vers la droite.

```
use int.Int
use ref.Ref
use int.Fact (* defines function "fact" *)

let factorial_partial (n : int) : int
  requires { n ≥ 0 }
  ensures { result = fact n }
  diverges
=
  let ref i = 0 in
  let f = ref 1 in
  while i < n do
    invariant { 0 ≤ i ≤ n ∧ !f = fact i }
    i := i+1;
    f := !f * i
  done;
  !f
```

Listing 7.2 – Factorielle en WhyML, correction partielle.

```
let factorial (n : int) : int
  requires { n ≥ 0 }
  ensures { result = fact n }
=
  let ref i = 0 in
  let f = ref 1 in
  while i < n do
    invariant { 0 ≤ i ≤ n ∧ !f = fact i }
    variant { n - i }
    i := i+1;
    f := !f * i
  done;
  !f

let main () : int
=
  factorial 10
```

Listing 7.3 – Factorielle en WhyML, correction totale et exemple d'utilisation.

7.4.2 Recherche linéaire dans un tableau

Le listing 7.4 présente un programme WhyML de recherche d'une valeur x dans un tableau d'entiers a .

```
use int.Int
use ref.Ref
use array.Array

let linear_search (a: array int) (x: int) : int
  ensures { (result = -1 && forall i. 0 ≤ i < length a → a[i] ≠ x) ||
    (0 ≤ result < length a && a[result] = x &&
      forall i. 0 ≤ i < result → a[i] ≠ x) }
=
  let i = ref 0 in
  while !i < length a do
    invariant { 0 ≤ !i ≤ length a && forall j. 0 ≤ j < !i → a[j] ≠ x }
    variant { length a - !i }
    if a[!i] = x then return !i;
    i := !i + 1
  done;
  -1

let main () : int
=
  let a = make 4 1 in
  a[1] ← 2;
  a[2] ← 3;
  a[3] ← 4;
  linear_search a 2
```

Listing 7.4 – Recherche linéaire en WhyML, avec un exemple d'utilisation.

Chapitre 8

Calculs inductifs

Le chapitre 4 nous a donné des moyens de définir des types de données. Le présent chapitre montre comment définir formellement des calculs sur ces types, comme des transformations syntaxiques.

Dans la partie 8.1 nous apprenons à définir des fonctions de transformation entre ces types, appelées « calculs inductifs ». Les parties 8.2 à 8.5 approfondissent la technique de définition de ces fonctions de manière récursive, par des égalités qui divisent la définition en cas disjoints.

8.1 Calculer sur des types inductifs

On veut pouvoir extraire une information de chaque habitant d'un type inductif, ou transformer ces données en données du même type ou d'un autre type. On dit alors qu'on effectue un « calcul (inductif) ». Un *calcul inductif* est une fonction f entre deux types inductifs T et T' . La fonction f a elle-même un type, noté $T \rightarrow T'$.

8.1.1 Exemple des fonctions booléennes

Les notations $+$: $\mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ et $.$: $\mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ déclarent un type pour les opérations « ou » et « et » de l'algèbre de Boole (voir définition dans l'annexe A).

Puisque le type $\mathbb{B}$ n'a que deux habitants (0 et 1), on peut définir complètement ces deux opérations par les égalités suivantes :

$$+(0, 0) = 0 \tag{8.1}$$

$$+(0, 1) = 1 \tag{8.2}$$

$$+(1, 0) = 1 \tag{8.3}$$

$$+(1, 1) = 1 \tag{8.4}$$

et

$$.(0, 0) = 0 \quad (8.5)$$

$$.(0, 1) = 0 \quad (8.6)$$

$$.(1, 0) = 0 \quad (8.7)$$

$$.(1, 1) = 1 \quad (8.8)$$

Ces égalités correspondent aux tables de vérité de la disjonction et de la conjonction.

8.1.2 Calcul du nombre d'éléments d'une liste

Cette partie et la suivante donne deux exemples de calculs définis sur le type des listes homogènes.

La fonction $length : liste(\alpha) \rightarrow \mathbb{N}$ permet de calculer le nombre d'éléments dans une liste. Elle est définie par les égalités suivantes :

$$length(Nil) = 0 \quad (8.9)$$

$$length(Cons(a, l)) = 1 + length(l) \quad (8.10)$$

REMARQUE 8.1. En toute rigueur, le type $\mathbb{N}$ étant défini par les constructeurs Z et S , nous ne devrions utiliser que ces constructeurs pour définir le résultat de la fonction $length$. Par abus, ici et ailleurs, lorsqu'il n'y a pas de risque de confusion, nous utilisons plutôt les notations usuelles en mathématique pour les entiers naturels : 0, 1 et la fonction binaire d'addition $+$, en notation *infixée* $\dots + \dots$ plutôt qu'en notation *préfixée* $+(\dots, \dots)$.

L'égalité (8.10) est particulière : elle définit la fonction $length$ à partir d'elle-même. On dit que la fonction $length$ est *récursive*. En général, ce genre d'auto-définition peut poser le problème de la non-terminaison du calcul. Ce problème ne se pose pas ici, car l'égalité (8.10) définit la longueur de la liste $Cons(a, l)$ à partir de la longueur de sa sous-liste l , qui contient un élément de moins.

Le listing 8.1 montre une implémentation de cette fonction en WhyML. Le mot-clé **rec** est indispensable pour signaler à Why3 que cette définition est récursive. Cette définition code exactement les égalités (8.9) et (8.10), mais génère l'avertissement **unused variable a**, car cette variable est inutile, puisqu'elle n'est pas utilisée dans le second membre de l'égalité (8.10). Dans ce cas, le langage WhyML permet de remplacer **Cons a l** par **Cons _ l**, qui n'introduit pas cette variable.

8.1.3 Juxtaposition de deux listes

La *concaténation* (ou *juxtaposition*) de deux listes l_1 et l_2 est la liste qui commence par les éléments de l_1 , dans le même ordre, et continue avec les éléments de l_2 , dans le même ordre. Par exemple, la concaténation des listes $[3; 0; 1]$ et $[1; 1; 4; 2]$ est la liste $[3; 0; 1; 1; 1; 4; 2]$.

```

use int.Int

type list 'a = Nil | Cons 'a (list 'a)

let rec function length (l : list 'a) : int
= match l with
| Nil → 0
| Cons a l → 1 + length l
end

```

Listing 8.1 – Calcul inductif de la longueur d’une liste en WhyML.

La fonction $append : liste(\alpha) \times liste(\alpha) \rightarrow liste(\alpha)$, telle que $append(l_1, l_2)$ soit la juxtaposition des listes l_1 et l_2 , est définie inductivement par les égalités suivantes :

$$append(Nil, l) = l \quad (8.11)$$

$$append(Cons(x, l), m) = Cons(x, append(l, m)) \quad (8.12)$$

Remarquez que ce calcul ne distingue que les deux cas de première liste vide ou pas. La seconde liste (l dans (8.11) et m dans (8.12)) est quelconque. Pour définir cette fonction, il est inutile de distinguer les cas où cette seconde liste est vide ou pas. Pour définir une fonction plus complexe, cette distinction peut être nécessaire.

Le listing 8.2 montre une implémentation de cette fonction en WhyML, pour le même type de liste défini dans le listing 8.1.

```

let rec function append (l m : list 'a) : list 'a
= match l with
| Nil → m
| Cons x l → Cons x (append l m)
end

```

Listing 8.2 – Concaténation de deux listes en WhyML.

Le type des listes et ses fonctions les plus usuelles sont définis dans le fichier `list.mlw` de la bibliothèque standard de Why3. La déclaration `use list.List` permet d’importer ce type. La déclaration `use list.Length` permet d’importer une définition de la fonction `length` et deux lemmes de propriétés de cette fonction. La déclaration `use list.Append` permet d’importer une définition d’une fonction `++` en notation infixée analogue à notre définition de la fonction `append`, ainsi que cinq lemmes de propriétés de cette fonction. Le listing 8.3 montre un exemple d’utilisation de ces modules.

8.1.4 Généralisation

Qu’est-ce qu’un calcul sur un type inductif ?

On peut dire (rapidement) qu'un calcul sur un type inductif T est une fonction f de type $T \rightarrow T'$ où T' est un type quelconque, ce qu'on écrit $f : T \rightarrow T'$.

Pour définir une telle fonction de calcul f , on considère un par un tous les constructeurs du type T .

Pour chaque constructeur C de type $T_1 \times \dots \times T_n \rightarrow T$ du type T , on définit la valeur de $f(C(e_1, \dots, e_n))$ par une égalité de la forme

$$f(C(e_1, \dots, e_n)) = g(f_1(e_1), \dots, f_n(e_n))$$

où g est une fonction (constructeur ou fonction de calcul) et où la fonction f_i ($1 \leq i \leq n$) peut être soit l'identité soit la fonction f appliquée à l'expression e_i , si cette expression est de type T . Quand au moins l'une des fonctions f_i ($1 \leq i \leq n$) est la fonction f qu'on est en train de définir, on dit que cette fonction f est *réursive*.

Pour le même constructeur, il est parfois nécessaire de distinguer plusieurs sous-cas pour la valeur de la fonction, ce qu'on fait en général en définissant plusieurs telles égalités, chacune s'appliquant sous une certaine condition. Il faut alors veiller à ce que ces conditions soient deux à deux disjointes et couvrent toutes les situations possibles. Si c'est ce cas, la définition de f est complète si on a traité ainsi tous les constructeurs du type T , car, par définition de ce type, on ne peut pas en définir des habitants autrement qu'à l'aide de ses constructeurs.

La structure de ces égalités qui définissent la fonction f garantit la terminaison de ce calcul.

REMARQUE 8.2. La définition d'un calcul peut être plus complexe. En particulier, g peut être une expression de calcul complexe, mais cette notion de calcul est suffisante dans beaucoup de cas qui nous intéressent.

REMARQUE 8.3. On peut considérer chaque constructeur comme un calcul qu'on ne définit que par son type (sans égalités).

8.1.5 Exemple : calcul du nombre de feuilles d'un arbre binaire

Détaillons à présent la définition d'un calcul inductif *nb-feuilles* tel que *nb-feuilles*(t) calcule le nombre de feuilles dans l'arbre binaire t .

La première étape consiste à donner le type de cette fonction :

La fonction *nb-feuilles* est de type $arbre(\alpha) \rightarrow \mathbb{N}$, ce qu'on écrit

$$nb-feuilles : arbre(\alpha) \rightarrow \mathbb{N},$$

car son unique paramètre est un arbre binaire dont les feuilles sont d'un type quelconque α , et son résultat est un entier naturel.

La deuxième étape consiste à définir la fonction par des égalités. Ces égalités s'appuient sur les constructeurs des habitants du type d'entrée, qui est ici le type inductif $arbre(\alpha)$:

1. Le premier constructeur est *Leaf* : $\alpha \rightarrow arbre(\alpha)$, qui construit une feuille.

L'égalité qui définit le nombre de feuilles dans ce cas est donc de la forme

$$nb-feuilles(Leaf(a)) = \dots$$

Il reste à trouver une expression convenable pour remplacer les pointillés dans cette égalité. Puisque l'expression $Leaf(a)$ représente un arbre réduit à **une** feuille, la bonne réponse est **1**. On écrit donc

$$nb\text{-}feuilles(Leaf(a)) = 1. \quad (8.13)$$

Notez que la réponse est bien une expression de type $\mathbb{N}$.

2. Le second constructeur est $Node : arbre(\alpha) \times arbre(\alpha) \rightarrow arbre(\alpha)$.

L'égalité qui définit le nombre de feuilles dans ce cas est donc de la forme

$$nb\text{-}feuilles(Node(t_1, t_2)) = \dots$$

Puisque l'expression $Node(t_1, t_2)$ construit un arbre à partir de deux arbres t_1 et t_2 , son nombre de feuilles est la somme des nombres de feuilles des arbres t_1 et t_2 , ce qui donne l'égalité

$$nb\text{-}feuilles(Node(t_1, t_2)) = nb\text{-}feuilles(t_1) + nb\text{-}feuilles(t_2). \quad (8.14)$$

Remarquez dans le membre de droite (à droite du signe $=$) les deux appels récursifs de la fonction. Ils sont corrects du point de vue du typage car le type du constructeur $Node$ exige que t_1 et t_2 soient de type $arbre(\alpha)$.

Continuons à présent cette vérification du typage. D'après le type $arbre(\alpha) \rightarrow \mathbb{N}$ de la fonction $nb\text{-}feuilles$, les expressions $nb\text{-}feuilles(t_1)$ et $nb\text{-}feuilles(t_2)$ sont des entiers naturels. Leur somme est aussi un entier naturel, ce qui est le type attendu pour l'expression $nb\text{-}feuilles(Node(t_1, t_2))$.

Attention : une telle vérification de typage ne permet pas de garantir qu'on a défini la bonne fonction. Par exemple, l'égalité fausse

$$nb\text{-}feuilles(Node(t_1, t_2)) = nb\text{-}feuilles(t_1)$$

est elle-aussi bien typée. Cette vérification de typage est cependant utile et recommandée, car elle permet de détecter de nombreuses erreurs.

En WhyML, cette fonction pourrait être déclarée comme un symbole fonctionnel, avec le mot-clé **function**, et les égalités qui la définissent pourraient être formalisées en WhyML par des axiomes. Mais on n'obtiendrait ainsi qu'une fonction WhyML dite *logique*, qui ne peut être utilisée que dans des formules. Or, avec le mot-clé **let**, le langage WhyML permet aussi de définir des fonctions qui sont des *programmes*, qu'on pourra exécuter pour obtenir leur résultat. Pour définir une telle fonction-programme constructeur par constructeur, on n'écrit pas des égalités, mais on utilise l'instruction de *filtrage (de motif)* (*pattern-matching* en anglais), comme dans l'exemple du listing 8.3.

Dans ce code, l'utilisation conjointe du mot-clé **let** et du mot-clé **function** définit la fonction **nbLeaf** à la fois comme un programme et comme une fonction logique, ce qui permettra de l'utiliser dans d'autres programmes et dans n'importe quelle formule WhyML.

En WhyML, le mot-clé **rec** dans cette définition est indispensable pour signaler à Why3 que la fonction s'appelle elle-même.

Enfin, on peut remarquer que la variable a dans l'égalité (8.13) ne sert à rien, puisqu'elle n'est pas utilisée dans le second membre de cette égalité. Dans ce cas, le langage WhyML permet de la remplacer par la syntaxe `_`, comme dans le listing 8.3, qui “économise” la déclaration (et la création à l'exécution) d'une variable inutile.

```
use int.Int

type bintree 'a = Leaf 'a | Node (bintree 'a) (bintree 'a)

let rec function nbLeaf (t : bintree 'a) : int
= match t with
| Leaf _ → 1
| Node t1 t2 → nbLeaf t1 + nbLeaf t2
end
```

Listing 8.3 – Calcul inductif du nombre de feuilles d'un arbre binaire en WhyML.

8.1.6 Composition des calculs

Certains calculs peuvent être définis en composant d'autres calculs ou constructeurs.

Par exemple, on aurait pu définir la fonction *nb-feuilles* par composition de la fonction *length* sur les listes et d'une fonction *ldf* qui calcule la liste des feuilles de tout arbre binaire, en écrivant l'égalité

$$nb\text{-}feuilles(t) = length(ldf(t)). \quad (8.15)$$

Exercice : Définir la fonction *ldf*.

Réponse :

$$ldf(Leaf(a)) = Cons(a, Nil) \quad (8.16)$$

$$ldf(Node(t_1, t_2)) = append(ldf(t_1), ldf(t_2)) \quad (8.17)$$

Le listing 8.4 montre comment implémenter cette fonction en WhyML.

Évidemment, il faut que la composition soit possible, ce qui peut se voir dans le type des fonctions que l'on compose.

Dans l'exemple, la fonction *length* de type $liste(\alpha) \rightarrow \mathbb{N}$ se compose avec la fonction *ldf* de type $arbre(\alpha) \rightarrow liste(\alpha)$ pour donner une fonction de type $arbre(\alpha) \rightarrow \mathbb{N}$.

En WhyML, la ligne suivante suffit pour définir une fonction **nbLeafCompos** par composition de deux fonctions définies avant.

```
let function nbLeafCompos t = length (listOfLeaves t)
```

```

use list.List
use list.Length
use list.Append

let rec function listOfLeaves (t : bintree 'a) : list 'a
= match t with
| Leaf a → Cons a Nil
| Node t1 t2 → listOfLeaves t1 ++ listOfLeaves t2
end

```

Listing 8.4 – Calcul inductif de la liste des feuilles d’un arbre binaire en WhyML.

Les parties 8.2 à 8.5 approfondissent la définition de fonctions qui ont pour arguments une, deux ou plusieurs listes, de manière récursive, par des égalités qui distinguent les deux cas de définition de la liste.

8.2 Calculs simples sur une liste

Rappelons que

$$Nil : liste(\alpha)$$

est le constructeur de la liste vide et que

$$Cons : \alpha \times liste(\alpha) \rightarrow liste(\alpha)$$

est l’autre constructeur, qui ajoute un élément devant ceux d’une autre liste. Rappelons aussi qu’il n’y a qu’une seule façon de construire une liste avec ces deux constructeurs.

Dans cette partie et les suivantes, nous donnons des exemples variés de définitions de calculs sur les listes qui s’appuient sur les deux cas de construction d’une liste.

8.2.1 Somme des éléments d’une liste d’entiers

Une fonction *somme* qui fait la somme des éléments d’une liste d’entiers a pour type *somme* : $liste(\mathbb{N}) \rightarrow \mathbb{N}$. Elle peut être définie inductivement par les égalités suivantes :

$$somme(Nil) = 0 \tag{8.18}$$

$$somme(Cons(a, l)) = a + somme(l) \tag{8.19}$$

Remarquez bien où se place l’appel récursif : Si l’on veut respecter le typage, c’est la seule place possible !

Exercice 8.1. Coder cette fonction *somme* par une fonction WhyML nommée **somme**, en utilisant le type **int** au lieu des entiers naturels.

Le calcul avec une telle fonction *réursive* peut être présenté par égalités successives, où chaque égalité n'applique qu'une seule des égalités qui définissent cette fonction.

Ainsi, on *déplie* le calcul de

$$\text{somme}(\text{Cons}(2, \text{Cons}(7, \text{Cons}(3, \text{Nil}))))$$

en écrivant

$$\begin{aligned} & \text{somme}(\text{Cons}(2, \text{Cons}(7, \text{Cons}(3, \text{Nil})))) \\ = & \hspace{10em} < \text{par (8.19)} > \\ & 2 + \text{somme}(\text{Cons}(7, \text{Cons}(3, \text{Nil}))) \\ = & \hspace{10em} < \text{par (8.19)} > \\ & 2 + (7 + \text{somme}(\text{Cons}(3, \text{Nil}))) \\ = & \hspace{10em} < \text{par (8.19)} > \\ & 2 + (7 + (3 + \text{somme}(\text{Nil}))) \\ = & \hspace{10em} < \text{par (8.18)} > \\ & 2 + (7 + (3 + 0)) \\ = & \hspace{10em} < \text{par calcul arithmétique} > \\ & 12 \end{aligned}$$

On peut aussi voir ce calcul comme une preuve équationnelle du théorème

$$\text{somme}(\text{Cons}(2, \text{Cons}(7, \text{Cons}(3, \text{Nil})))) = 12. \quad (8.20)$$

Il n'y a dans cette preuve qu'une possibilité à chaque pas, parce que la preuve est entièrement guidée par la structure de la liste, décrite par les constructeurs *Cons* et *Nil*, et que cette structure est unique.

En conséquence, on a bien un *calcul* unique (on dit aussi déterministe). C'est pourquoi cette description est un *programme* (on dit aussi une *implémentation*).

Exercice 8.2. Proposer un *but WhyML* permettant de vérifier ce calcul.

Exercice 8.3. Définir une fonction *réursive* pour calculer le produit des éléments d'une liste d'entiers.

Coder cette fonction par une fonction *WhyML* nommée **produit**, en utilisant à nouveau le type **int** au lieu des entiers naturels.

Exercice 8.4. Justifier informellement que le typage des égalités (8.18) et (8.19) est correct, en prenant pour hypothèses le type de *somme*, le type de $+$: $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ et celui de l'égalité des entiers $=$: $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{B}$.

8.2.2 Un argument supplémentaire influence le calcul

On veut définir une fonction *compte* telle que *compte*(*x*, *l*) soit le nombre d'occurrences de *x* dans la liste *l*.

Le type de x doit être le même que celui des éléments de la liste l , mais il peut être quelconque. On le désigne par la variable de type α (ces notions de typage sont développées dans l'annexe B). Par conséquent, le type de la fonction *compte* est $\alpha \times \text{liste}(\alpha) \rightarrow \mathbb{N}$. Autrement dit, on a l'hypothèse

$$\text{compte} : \alpha \times \text{liste}(\alpha) \rightarrow \mathbb{N}.$$

La fonction *compte* a deux arguments : une valeur de type α et une liste. On ne sait rien sur le type α de la valeur. Par conséquent, les différents cas de la définition de la fonction *compte* ne peuvent provenir que des différents cas de construction d'une liste (vide ou pas).

$$\text{compte}(a, \text{Nil}) = 0 \quad (8.21)$$

$$\text{compte}(a, \text{Cons}(a, l)) = 1 + \text{compte}(a, l) \quad (8.22)$$

$$\text{compte}(a, \text{Cons}(b, l)) = \text{compte}(a, l) \quad \text{si } a \neq b \quad (8.23)$$

La première égalité (8.21) déclare que la liste vide contient 0 occurrences de la valeur a . Si la liste n'est pas vide, il faut distinguer deux cas, selon que son premier élément est l'élément compté ou pas. Dans l'égalité (8.22) le premier élément de la liste est l'élément compté a . Pour compter le nombre d'occurrences de a dans la liste $\text{Cons}(a, l)$, dont le premier élément est a et le reste est l , on peut compter ce nombre dans la liste l et ajouter 1 pour le a au début de la liste. C'est ce que formalise cette égalité. L'égalité (8.23) traite le cas où le premier élément de la liste ne correspond pas à l'élément compté.

Cette définition est un exemple de définition inductive sur le type des listes avec trois égalités au lieu de deux, c'est-à-dire plus d'égalités que de règles de construction du type *liste*.

Exercice 8.5. Coder cette fonction *compte* par une fonction WhyML nommée **compte**, en utilisant le type *int* au lieu des entiers naturels.

Exercice 8.6. Déplier le calcul de $\text{compte}(4, \text{Cons}(5, \text{Cons}(4, \text{Cons}(7, \text{Nil}))))$.

Expliquer comment vérifier ce calcul avec Why3.

8.2.3 Le résultat est une liste

Le type d'une fonction *filtre-pair* qui doit retenir dans le résultat la liste de tous les éléments pairs d'une liste donnée est $\text{liste}(\mathbb{N}) \rightarrow \text{liste}(\mathbb{N})$.

Le résultat est une liste qu'il va falloir construire. La fonction *filtre-pair* peut être définie inductivement par les égalités suivantes :

$$\text{filtre-pair}(\text{Nil}) = \text{Nil} \quad (8.24)$$

$$\text{filtre-pair}(\text{Cons}(x, l)) = \text{Cons}(x, \text{filtre-pair}(l)) \quad \text{si } x \text{ est pair} \quad (8.25)$$

$$\text{filtre-pair}(\text{Cons}(x, l)) = \text{filtre-pair}(l) \quad \text{si } x \text{ n'est pas pair} \quad (8.26)$$

Exercice 8.7. Coder cette fonction par une fonction WhyML nommée **filtrePair**, en utilisant le type *int* au lieu des entiers naturels.

8.2.4 Choix d'une liste pour guider le calcul

Nous avons défini dans la partie 8.1.3 la fonction *append* qui juxtapose deux listes. Son type est

$$\text{liste}(\alpha) \times \text{liste}(\alpha) \rightarrow \text{liste}(\alpha).$$

Cette fonction a deux arguments de type liste. Une question se pose : quelle liste choisir pour décomposer le calcul en cas ? La première, la seconde, ou les deux ? Y-a-t-il un choix possible ? Si oui, quel est le meilleur choix ?

Il n'y a pas de réponse générale à ces questions. Dans la partie 8.1.3, nous avons choisi de décomposer par cas selon la première liste.

En général, le choix de la liste (ou des listes) guidant le mieux la définition des calculs est affaire d'intuition (on devine pourquoi un choix est judicieux), d'expérience (on a déjà spécifié beaucoup de fonctions ressemblantes) et d'expérimentation (sans intuition et sans expérience, on peut toujours essayer tous les choix possibles l'un après l'autre).

8.2.5 Construction plus complexe d'une liste résultat d'un calcul

La construction d'une liste peut ne pas suivre l'ordre donné par la liste en argument. Considérons par exemple le calcul de la liste inverse d'une liste donnée, par une fonction

$$\text{miroir} : \text{liste}(\alpha) \rightarrow \text{liste}(\alpha).$$

Cette fonction peut être définie par :

$$\text{miroir}(\text{Nil}) = \text{Nil} \quad (8.27)$$

$$\text{miroir}(\text{Cons}(x, l)) = \text{append}(\text{miroir}(l), \text{Cons}(x, \text{Nil})) \quad (8.28)$$

Une définition peut utiliser une fonction auxiliaire, comme c'est le cas ici avec *append*.

8.3 Suivre deux listes en même temps

Un calcul peut très bien suivre la structure de deux listes à la fois. Par exemple le calcul de la liste des couples d'éléments de même rang de deux listes données

$$\text{couples} : \text{liste}(\alpha) \times \text{liste}(\beta) \rightarrow \text{liste}(\alpha \times \beta)$$

est définie inductivement par

$$\text{couples}(\text{Nil}, \text{Nil}) = \text{Nil} \quad (8.29)$$

$$\text{couples}(\text{Cons}(x, l_1), \text{Cons}(y, l_2)) = \text{Cons}((x, y), \text{couples}(l_1, l_2)) \quad (8.30)$$

si les deux listes sont de même longueur.

8.4 Préconditions d'un calcul

Le calcul de *couples* n'est correct que si les deux listes sont de même longueur. Aussi, on doit associer la précondition

$$(length(l_1) = length(l_2))$$

à *couples*(l_1, l_2)).

Exercice 8.8. Définir inductivement une fonction *zip* qui prend en paramètres deux listes de longueurs quelconques et ne met en couples que les deux débuts de même longueur des deux listes.

Coder cette fonction par une fonction WhyML nommée **zip**.

Une précondition est une hypothèse du calcul, elle est donc supposée être vérifiée par les arguments de la fonction.

Par exemple la fonction *car* qui prend l'élément de tête d'une liste ne s'applique que sur des listes non vides. Sa précondition est ($l \neq Nil$).

8.5 Ajouter des arguments accumulateurs du résultat

Définissons le calcul *cumuler* : $liste(\mathbb{N}) \times \mathbb{N} \rightarrow \mathbb{N}$ qui ajoute à un entier la somme des éléments d'une liste.

$$cumuler(Nil, s) = s \tag{8.31}$$

$$cumuler(Cons(x, l), s) = cumuler(l, x + s) \tag{8.32}$$

Exercice 8.9. Coder cette fonction par une fonction WhyML nommée **cumuler**.

Déplier le calcul $cumuler(Cons(5, Cons(4, Nil)), 3)$. Remarquer que le calcul se porte sur l'argument que l'on appelle accumulateur du résultat.

8.6 Synthèse

Ce chapitre montre comment définir des calculs sur des types inductifs, même infinis, par un nombre fini d'égalités, grâce à des appels récursifs. Le type de ces calculs permet de vérifier que chaque égalité est bien typée. La méthode de vérification de type est généralisée et formalisée dans l'annexe B. Le chapitre 10 expliquera comment démontrer formellement des égalités comme (8.15), à partir des définitions inductives des fonctions qui la compose.

Chapitre 9

Preuves de propriétés par récurrence

Ce chapitre porte sur la démonstration de propriétés universelles des entiers naturels, c'est-à-dire des formules de la forme $(\forall n : \mathbb{N}. \dots)$. L'ensemble $\mathbb{N}$ des nombres naturels étant infini, il n'est pas possible de prouver ces propriétés en énumérant tous les entiers naturels. On utilise une technique de preuve qui est fondamentale en mathématiques et en informatique, qui s'appelle "la preuve par récurrence".

Les trois premières parties de ce chapitre expliquent le principe de la preuve par récurrence, le formalise et l'illustre avec des exemples détaillés. Enfin, la partie 9.4 explique comment obtenir un raisonnement par récurrence automatique avec Why3.

9.1 Principe de récurrence

Considérons une proposition $P(n)$ qui dépend d'un paramètre n qui est un entier naturel. Par exemple, $P(n)$ peut être la proposition « La somme des n premiers nombres impairs est égale à n^2 ».

Par exemple, $P(2)$ affirme que $1 + 3 = 2^2$ et $P(3)$ affirme que $1 + 3 + 5 = 3^2$.

Le principe de récurrence est une conséquence de la définition inductive des entiers naturels (définition 4.2). Puisque tout entier naturel est soit 0 soit le successeur d'un entier naturel, pour prouver $P(n)$ pour tout entier naturel, il suffit de prouver $P(0)$ (la propriété pour $n = 0$) et de déduire la propriété $P(n + 1)$ pour le successeur de n en admettant $P(n)$, c'est-à-dire que la propriété est vraie pour n , pour un entier naturel n quelconque. Ce mode de raisonnement est appelé *principe de récurrence (structurelle)* car il est calqué sur la structure inductive des entiers naturels.

DÉFINITION 9.1 (PRINCIPE DE RÉCURRENCE). *Pour toute propriété $P(n)$ d'un entier naturel n , le principe de récurrence structurelle est la règle de déduction*

$$\frac{P(0), \quad \forall n : \mathbb{N}. P(n) \Rightarrow P(n + 1)}{\forall n : \mathbb{N}. P(n)}.$$

Cette règle se lit comme ceci : Pour prouver que la propriété $P(n)$ est vraie pour tout entier naturel n (conclusion de la règle), il suffit de

- démontrer que la propriété est vraie pour $n = 0$ (c'est ce que demande la première prémisses $P(0)$), et, indépendamment,
- pour tout entier n , déduire la propriété $P(n + 1)$ de la propriété $P(n)$, cette dernière étant appelée « hypothèse de récurrence » dans ce contexte. Ceci est formalisée dans la deuxième prémisses de cette règle de déduction.

Intuitivement, la deuxième prémisses permet de déduire $P(1)$ de $P(0)$, puis $P(2)$ de $P(1)$, et ainsi de suite jusqu'à $P(n)$ pour un entier naturel n quelconque. Ce principe est admis sans autre justification.

EXEMPLE 9.1. Soit $P(n)$ la proposition « La somme des n premiers nombres impairs est égale à n^2 ». Le premier nombre impair est 1, le deuxième est 3, le troisième est 5, ..., le i^{e} nombre impair est $2 * i - 1$.

$P(0)$ est la proposition « La somme des 0 premiers nombres impairs est égale à 0^2 », ce qui est vrai car une somme vide doit être égale au neutre de la somme, qui est 0. Si cet argument ne vous convainc pas, vous pouvez démontrer $P(1)$ et utiliser le principe de récurrence à partir de 1 (voir plus loin le déplacement du cas de base).

Montrons à présent comment déduire $P(n + 1)$ de $P(n)$, pour un entier n quelconque. $P(n + 1)$ est la proposition « La somme des $n + 1$ premiers nombres impairs est égale à $(n + 1)^2$ ». Cette somme est égale à la somme des n premiers nombres impairs, augmentée du $(n + 1)^{\text{e}}$ nombre impair, qui est $2 * (n + 1) - 1 = 2 * n + 1$. Selon l'hypothèse de récurrence $P(n)$, la somme des n premiers nombres impairs vaut n^2 . Or on sait que $n^2 + (2 * n + 1) = (n + 1)^2$, ce qui démontre $P(n + 1)$.

9.1.1 Définitions

La preuve de $P(0)$ s'appelle *le cas de base* de la récurrence.

La preuve de $\forall n : \mathbb{N}. P(n) \Rightarrow P(n + 1)$ s'appelle *le pas de la récurrence*.

La partie $P(n)$ de ce pas de récurrence est appelée *hypothèse de récurrence*.

9.1.2 Comment présenter une preuve par récurrence ?

Quand on prouve $(\forall n : \mathbb{N}. P(n))$ par récurrence, on prouve le cas de base et le pas de la récurrence séparément.

Reprenons l'exemple précédent, en désignant la somme des n premiers nombres impairs par l'expression $(\sum i : 1 \leq i \leq n. 2 * i - 1)$. La formule $P(n)$ à démontrer pour tout entier naturel n est l'égalité

$$(\sum i : 1 \leq i \leq n. 2 * i - 1) = n^2.$$

Preuve. La preuve est par récurrence sur n .

— **Cas de base** $P(0)$:

$$\begin{aligned}
 & P(0) \\
 = & \quad \text{par définition} \\
 & (\sum i : 1 \leq i \leq 0.2 * i - 1) \\
 = & \quad \text{puisque le domaine de la somme est vide} \\
 & 0 \\
 = & \quad \text{selon l'arithmétique} \\
 & 0^2
 \end{aligned}$$

— **Pas de récurrence** : Soit n un entier naturel quelconque, **fixé**. Pour cet entier n , on admet l'hypothèse de récurrence $P(n)$, qui est ici

$$(\sum i : 1 \leq i \leq n.2 * i - 1) = n^2.$$

Il s'agit alors de démontrer $P(n + 1)$, qui est l'égalité

$$(\sum i : 1 \leq i \leq n + 1.2 * i - 1) = (n + 1)^2.$$

Cette démonstration transforme le membre gauche $(\sum i : 1 \leq i \leq n + 1.2 * i - 1)$ de cette égalité en son membre droit $(n + 1)^2$, par égalités successives, comme ceci :

$$\begin{aligned}
 & (\sum i : 1 \leq i \leq n + 1.2 * i - 1) \\
 = & \quad < \text{en séparant le dernier terme} > \\
 & (\sum i : 1 \leq i \leq n.2 * i - 1) + (2 * (n + 1) - 1) \\
 = & \quad < \text{par l'hypothèse de récurrence} > \\
 & n^2 + (2 * (n + 1) - 1) \\
 = & \quad < \text{par calcul arithmétique} > \\
 & (n + 1)^2.
 \end{aligned}$$

C'est ce qu'il fallait démontrer.

9.2 Déplacement du cas de base

On peut aussi démontrer par récurrence une propriété $P(n)$ non définie pour les valeurs de n inférieures à un certain entier n_0 . Par exemple, on peut considérer que la proposition « La somme des n premiers nombres impairs est égale à n^2 » n'est pas définie pour $n = 0$ et commencer la démonstration avec $n = 1$.

Exercice 9.2. Rédiger une démonstration par récurrence de la proposition « La somme des n premiers nombres impairs est égale à n^2 », pour tout entier naturel $n \geq 1$.

9.2.1 Exemple

Démontrons que $2 * n + 1 < 2^n$ pour $n \geq 3$.

Il s'agit de prouver $(\forall n : \mathbb{N}. n \geq 3 \Rightarrow P(n))$, avec $P(n) = (2 * n + 1 < 2^n)$.

Preuve. La preuve est par récurrence sur $n \geq 3$.

— **Cas de base** $P(3)$:

$$P(3) = (2 * 3 + 1 < 2^3) = (7 < 8)$$

qui est évidemment vrai.

— **Pas de récurrence** :

Pour $n \geq 3$ quelconque, on admet l'hypothèse de récurrence $P(n)$ et on démontre $P(n + 1)$, qui est

$$2 * (n + 1) + 1 < 2^{n+1}.$$

On part du second membre de cette comparaison et on procède par comparaisons et égalités successives :

$$\begin{aligned} & 2^{n+1} \\ = & & \text{par arithmétique} \\ & 2 * 2^n \\ > & & \text{par l'hypothèse de récurrence} \\ & 2 * (2 * n + 1) \\ = & & \text{par arithmétique} \\ & 4 * n + 2 \\ = & & \text{par arithmétique} \\ & (2 * (n + 1) + 1) + 2 * n - 1 \\ > & & \text{puisque } 2 * n - 1 > 0 \text{ pour } n \geq 3 \\ & 2 * (n + 1) + 1. \end{aligned}$$

9.3 Encore un exemple

Supposons l'existence d'une monnaie composée uniquement de pièces de 2 centimes et de 5 centimes. Montrons que tout total au dessus de 3 centimes peut être formé avec ces pièces. On désigne par *multi-ensemble* une collection quelconque de pièces de 2 centimes et de 5 centimes.

L'énoncé de $P(n)$ est le suivant : « Il existe un multi-ensemble composé de pièces de 2 centimes et de 5 centimes ayant pour somme n ». Il s'agit alors de prouver

$$(\forall n. n \geq 4 \Rightarrow P(n)).$$

9.3.1 Cas de base

Pour obtenir 4 centimes on prend 2 pièces de 2 centimes.

9.3.2 Pas de récurrence

L'hypothèse de récurrence $P(n)$ assure qu'il existe un multi-ensemble dont la somme est n . Il y a deux cas :

1. Le multi-ensemble contient au moins une pièce de 5 centimes. Alors, on peut remplacer une des pièces de 5 centimes par 3 pièces de 2 centimes pour obtenir un multi-ensemble dont la somme est $n + 1$.
2. Le multi-ensemble contient seulement des pièces de 2 centimes. Dans ce cas, il y a au moins deux pièces de 2 centimes puisque $4 \leq n$. On remplace alors ces deux pièces de 2 centimes par une de 5 centimes et le tour est joué.

9.4 Preuve par récurrence automatique avec Why3

Pour qu'une preuve par récurrence soit automatique avec Why3, il faut que Why3 connaisse le principe de récurrence (définition 9.1), et sa variante par déplacement du cas de base, et applique le bon principe automatiquement dans le bon contexte. Ceci ne fait pas partie des fonctionnalités de Why3. Ces principes de récurrence ne peuvent pas être formalisés directement par des axiomes en WhyML, car ces formules commenceraient par une quantification "pour tous les prédicats P ", qui est une quantification du second ordre, alors que la logique de Why3 est limitée à la quantification du premier ordre. Par ailleurs, il n'y a pas de type prédéfini pour les entiers naturels en Why3.

Il existe plusieurs manières de formaliser ces principes en Why3. Cette partie présente une seule de ces manières, qui est assez légère, qui fonctionne dans l'interface en ligne de Why3 et qui convient aussi pour les propriétés définies sur les entiers relatifs (type prédéfini `int`) au lieu des entiers naturels.

9.4.1 Notion de fonction-lemme

En WhyML, une fonction-lemme (*lemma function* en anglais) est une fonction-programme sans effets de bord, dont la terminaison peut être prouvée, dont le type de retour est `unit` et qui comporte le mot-clé `lemma` en plus dans son entête. Sous ces conditions, la condition de vérification de cette fonction est produite sous la forme d'un lemme ou d'un but WhyML, qu'on peut ensuite démontrer automatiquement ou interactivement.

De plus, si cette fonction est récursive, sa définition peut encoder un principe d'induction, comme illustré les modèles et exemples de code suivants.

9.4.2 Modèle de fonction-lemme récursive sur le type prédéfini `int`

Supposons que la propriété P de la définition 9.1 soit implémentée en WhyML par un prédicat `p` défini de la manière suivante, avec les pointillés remplacés par une formule WhyML correcte sur l'entier `n` :

```
let predicate p (n : int) = ...
```

Alors, le principe de récurrence de la définition 9.1 peut être codé par la fonction-lemme suivante :

```
let rec lemma pRec (n: int): unit
  requires { n ≥ 0 }
  ensures { p n }
  variant { n }
= if n ≤ 0 then () else pRec (n-1)
```

En effet, prouver le contrat de cette fonction, c'est bien prouver

```
forall n: int. n ≥ 0 → p n
```

Par ailleurs, le code de la fonction indique comment commencer le raisonnement pour démontrer le lemme. La branche `then` du `if` correspond au cas de base. Sa branche `else` correspond au pas d'induction, qui consiste à supposer la propriété vraie pour `n-1`. Ici, un variant est requis pour démontrer que la fonction termine.

9.4.3 Modèle de fonction-lemme récursive sur les entiers naturels

Considérons la définition suivante d'un type inductif d'entiers naturels en WhyML :

```
type nat = Z | S nat
```

Supposons que la propriété P de la définition 9.1 soit implémentée en WhyML par un prédicat `p` défini sur ce type, de la manière suivante :

```
let predicate p (n : nat) = ...
```

Alors, le principe de récurrence de la définition 9.1 peut être codé par la fonction-lemme suivante :

```
let rec lemma pRec (n: nat): unit
  ensures { p n }
= match n with
| Z → ()
| S m → pRec m
end
```

En effet, Why3 exprimera la condition de vérification de cette fonction par le but suivant :

```
goal pRec'vc: forall n: nat.
  match n with
  | Z → true
  | S m → p m
  end
→ p n
```

Les deux cas du filtrage `match n with` donne les deux formules

```
true → p Z
```

et

```
p m → p (S m)
```

qui correspondent bien aux deux prémisses du principe de récurrence.

9.4.4 Exemple sur les entiers relatifs

Le code WhyML suivant définit une fonction-lemme qui permet de démontrer automatiquement avec l'interface en ligne de Why3 la propriété démontrée informellement dans la partie 9.2.

```
use int.Int
use int.Power

let rec lemma powerOf2 (n:int): unit
  requires { n ≥ 3 }
  ensures { 2*n+1 < power 2 n }
  variant { n }
= if n ≤ 3 then () else powerOf2 (n-1)
```


Chapitre 10

Preuve inductive, vérification de programmes récurrents

Nous avons vu dans le chapitre 9 comment le principe de récurrence permet de démontrer des propriétés pour tous les entiers naturels. Plus généralement, ce chapitre montre comment démontrer des propriétés pour tous les habitants d'un type inductif.

La notion de type inductif a été introduite dans le chapitre 4. Le principe de raisonnement sur les types inductifs s'appelle l'*induction structurelle*. L'induction structurelle généralise à tout type inductif le principe de récurrence (structurelle) étudié dans le chapitre 9. Une démonstration qui applique l'induction structurelle est appelée *preuve par induction*, ou *preuve inductive*.

Le chapitre 8 a présenté la notion de calcul inductif, qui est une fonction récursive lorsque ce calcul s'applique à un type inductif défini à partir de lui-même, comme le type des entiers naturels avec son constructeur *S* ou le type des listes avec son constructeur *Cons*. Le présent chapitre montre comment démontrer par une preuve inductive certaines propriétés portant éventuellement sur plusieurs fonctions récursives. Le premier exemple de telle propriété est la propriété (8.15) introduite dans le chapitre 8, entre le nombre de feuilles d'un arbre et la longueur de la liste de ses feuilles. La partie 10.2.1 présente une première manière d'automatiser une telle preuve inductive avec Why3. Dans certains cas, ces propriétés peuvent être exprimées dans un contrat (précondition et postcondition) d'une de ces fonctions récursives, et ceci peut faciliter l'automatisation de leur vérification avec Why3. Cette deuxième manière d'automatiser une preuve inductive ne sera abordée qu'à travers des exemples et des exercices.

La partie 10.1 rappelle le principe de récurrence. La partie 10.2 énonce le principe d'induction sur les listes, sous la forme d'une règle d'inférence, et l'illustre avec un exemple détaillé. La partie 10.3 explique le raisonnement par égalités successives utilisé dans de nombreuses démonstrations inductives. La partie 10.4 donne un exemple de preuve inductive sur les arbres binaires.

10.1 Rappel du principe de récurrence

Le principe de récurrence vu dans le chapitre 9 est le principe d'induction naturelle du type inductif des entiers naturels défini dans le chapitre 4. Ce principe sert à prouver des théorèmes sur les entiers naturels. Sous forme de règle d'inférence, il s'écrit :

$$\frac{P(0), \quad \forall n : \mathbb{N}. P(n) \Rightarrow P(n+1)}{\forall n : \mathbb{N}. P(n)}$$

10.2 Induction sur les listes

Le type inductif $liste(\alpha)$ des listes homogènes est défini dans le chapitre 4.

Sous forme d'une règle d'inférence, le principe d'induction sur les listes est :

$$\frac{P(Nil), \quad \forall l : liste(\alpha). \forall x : \alpha. P(l) \Rightarrow P(Cons(x, l))}{\forall l : liste(\alpha). P(l)}$$

Cette règle signifie que, pour prouver une propriété $P(l)$ pour toute liste l , il suffit de prouver cette propriété pour la liste vide, et, séparément, de prouver, sous l'hypothèse que la propriété P est vraie pour une liste l , qu'elle est encore vraie pour toute liste de la forme $Cons(x, l)$.

La première condition est formalisée par la première prémisse $P(Nil)$ et elle est appelée « cas de base ». La seconde condition est formalisée par la seconde prémisse $P(l) \Rightarrow P(Cons(x, l))$ et elle est appelée « pas d'induction ».

10.2.1 Modèle de fonction-lemme récursive sur les listes

Comme expliqué dans le chapitre 9 sur la récurrence, une preuve par induction structurelle peut être automatisée avec l'interface en ligne de Why3 si le lemme à démontrer est codé en WhyML sous la forme d'une fonction-lemme.

Pour le type des listes WhyML défini dans `list.List`, le principe précédent d'induction structurelle sur les listes peut être codé par la fonction-lemme suivante :

```
let rec lemma pInduction (l: list 'a): unit
  ensures { p l }
  = match l with
  | Nil → ()
  | Cons _ m → pInduction m
  end
```

en supposant que la propriété P de ce principe d'induction est implémentée en WhyML par un prédicat `p` défini sur ce type, de la manière suivante :

```
let predicate p (l : list 'a) = ...
```

10.2.2 Exemple

Une preuve par induction sur les listes fonctionne comme une preuve par récurrence.

Par exemple, prouvons formellement, inductivement, que pour toute liste de listes d'entiers l , on a l'égalité :

$$\text{somme-liste}(\text{liste-sommes}(l)) = \text{somme-liste}(\text{aplatir}(l)).$$

Les fonctions *somme-liste*, *liste-sommes* et *aplatir* sont définies comme suit ¹ :

$$\text{somme-liste}(\text{Nil}) = 0 \quad (10.1)$$

$$\text{somme-liste}(\text{Cons}(x, l)) = x + \text{somme-liste}(l) \quad (10.2)$$

$$\text{liste-sommes}(\text{Nil}) = \text{Nil} \quad (10.3)$$

$$\text{liste-sommes}(\text{Cons}(l, r)) = \text{Cons}(\text{somme-liste}(l), \text{liste-sommes}(r)) \quad (10.4)$$

$$\text{aplatir}(\text{Nil}) = \text{Nil} \quad (10.5)$$

$$\text{aplatir}(\text{Cons}(l, r)) = \text{append}(l, \text{aplatir}(r)) \quad (10.6)$$

La propriété $\text{somme-liste}(\text{liste-sommes}(l)) = \text{somme-liste}(\text{aplatir}(l))$ est notée $P(l)$. La preuve s'effectue par induction sur la liste l dont parle cette propriété $P(l)$.

10.2.2.1 Cas de base

Le cas de base est $P(\text{Nil})$. Il s'agit donc de prouver

$$\text{somme-liste}(\text{liste-sommes}(\text{Nil})) = \text{somme-liste}(\text{aplatir}(\text{Nil})).$$

On a d'une part ²

$$\begin{aligned} & \text{somme-liste}(\text{liste-sommes}(\text{Nil})) \\ = & \quad < \text{règle (10.3)} > \\ & \text{somme-liste}(\text{Nil}) \\ = & \quad < \text{règle (10.1)} > \\ & 0 \end{aligned}$$

et d'autre part

$$\begin{aligned} & \text{somme-liste}(\text{aplatir}(\text{Nil})) \\ = & \quad < \text{règle (10.5)} > \\ & \text{somme-liste}(\text{Nil}) \\ = & \quad < \text{règle (10.1)} > \\ & 0, \end{aligned}$$

ce qui établit ce cas de base.

1. C'est un exercice de la feuille « Exercices sur les calculs inductifs ».

2. Chaque égalité est justifiée par un texte entre $<$ et $>$.

10.2.2.2 Pas d'induction

Pour le pas d'induction, on suppose que, pour une liste l quelconque fixée,

$$\text{somme-liste}(\text{liste-sommes}(l)) = \text{somme-liste}(\text{aplatir}(l)).$$

C'est l'affirmation $P(l)$ de la règle d'induction, appelée « hypothèse d'induction ». On cherche à s'en servir pour prouver $P(\text{Cons}(x, l))$, qui est ici

$$\text{somme-liste}(\text{liste-sommes}(\text{Cons}(x, l))) = \text{somme-liste}(\text{aplatir}(\text{Cons}(x, l))).$$

On simplifie au maximum le membre de gauche de cette égalité, étape par étape, comme suit :

$$\begin{aligned} & \text{somme-liste}(\text{liste-sommes}(\text{Cons}(x, l))) \\ = & \quad < \text{règle (10.4)} > \\ & \text{somme-liste}(\text{Cons}(\text{somme-liste}(x), \text{liste-sommes}(l))) \\ = & \quad < \text{règle (10.2)} > \\ & \text{somme-liste}(x) + \text{somme-liste}(\text{liste-sommes}(l)) \\ = & \quad < \text{hypothèse d'induction} > \\ & \text{somme-liste}(x) + \text{somme-liste}(\text{aplatir}(l)) \end{aligned}$$

La troisième égalité applique l'hypothèse d'induction à la liste d'entiers l .

On simplifie au maximum le membre de droite de l'égalité $P(\text{Cons}(x, l))$, étape par étape, comme suit :

$$\begin{aligned} & \text{somme-liste}(\text{aplatir}(\text{Cons}(x, l))) \\ = & \quad < \text{règle (10.6)} > \\ & \text{somme-liste}(\text{append}(x, \text{aplatir}(l))) \\ = & \quad < \text{lemme 1} > \\ & \text{somme-liste}(x) + \text{somme-liste}(\text{aplatir}(l)) \end{aligned}$$

La deuxième égalité utilise le lemme suivant, dont la preuve est laissée en exercice.

Lemme 1 : $\text{somme-liste}(\text{append}(l_1, l_2)) = \text{somme-liste}(l_1) + \text{somme-liste}(l_2)$.

Puisque les deux membres de l'égalité $P(\text{Cons}(x, l))$ sont égaux au même terme, ils sont égaux entre eux, ce qui prouve l'égalité $P(\text{Cons}(x, l))$.

10.2.2.3 Code WhyML correspondant

Le code WhyML des fonctions n'est pas détaillé ici. Le code suivant permet de prouver automatiquement cette propriété dans l'interface en ligne de Why3.

```
let rec lemma pInduction (l: list (list int)): unit
  ensures { sommeListe (listeSommes l) = sommeListe (aplatir l) }
= match l with
| Nil → ()
| Cons _ m → pInduction m
end
```

10.3 Raisonnement équationnel inductif

Dans le cadre des types et des calculs inductifs, la preuve inductive d'un théorème exprimé sous forme d'égalité peut souvent se faire par réduction de chaque membre de l'égalité et comparaison syntaxique des formes réduites.

Cette partie illustre cette stratégie par un exemple et formalise la présentation des déductions équationnelles, non seulement pour en faciliter la lecture par un humain, mais aussi pour comprendre qu'une machine peut automatiser une grande partie du processus.

10.3.1 Exemple de preuve inductive

10.3.1.1 Théorème à démontrer

On veut prouver le théorème

$$\text{length}(\text{append}(l, m)) = \text{length}(l) + \text{length}(m) \quad (10.7)$$

pour toutes les listes (génériques) l et m de type $\text{liste}(\alpha)$.

L'induction peut porter sur l , sur m , ou sur ces deux variables. En général, mieux vaut ne mener qu'une seule induction à la fois. Si nécessaire, l'autre induction peut être ajoutée à l'intérieur de la première.

10.3.1.2 Axiomes

Les calculs inductifs length et append ont été définis dans le chapitre 8. Nous rappelons ici les égalités qui définissent ces fonctions :

$$\text{length}(\text{Nil}) = 0 \quad (10.8)$$

$$\text{length}(\text{Cons}(x_2, l_4)) = 1 + \text{length}(l_4) \quad (10.9)$$

$$\text{append}(\text{Nil}, l_1) = l_1 \quad (10.10)$$

$$\text{append}((\text{Cons}(x_1, l_2), l_3)) = \text{Cons}(x_1, \text{append}(l_2, l_3)) \quad (10.11)$$

Chaque égalité dans ces définitions des calculs length et append est un axiome utilisable dans la preuve du théorème. Les variables de ces axiomes sont volontairement choisies distinctes entre axiomes, afin de mieux mettre en évidence le mécanisme de substitution.

10.3.1.3 Choix du schéma inductif

On choisit de prouver (10.7) par induction sur l , sa première variable. Autrement dit, on considère (10.7) comme une propriété $P(l)$ dans laquelle m est une constante.

10.3.1.4 Cas de base

Il s'agit de prouver $P(\text{Nil})$.

La réduction du membre gauche de cette égalité donne, étape par étape :

$$\begin{aligned} & \text{length}(\text{append}(\text{Nil}, m)) \\ = & \quad < \text{r\`egle (10.10)} > \\ & \text{length}(m) \end{aligned}$$

La réduction du membre droit (où $+$ est un symbole fonctionnel d'arité 2 en notation infixée) donne :

$$\begin{aligned} & \text{length}(\text{Nil}) + \text{length}(m) \\ = & \quad < \text{r\`egle (10.8)} > \\ & 0 + \text{length}(m) \\ = & \quad < \text{axiome } 0 + x = x \text{ de l'arithmétique} > \\ & \text{length}(m) \end{aligned}$$

Les deux formes réduites étant syntaxiquement égales, $P(\text{Nil})$ est prouvé.

10.3.1.5 Pas d'induction

Il s'agit de prouver $P(\text{Cons}(x, l))$ sous l'hypothèse $P(l)$, qui s'ajoute aux axiomes mais peut être utilisée dans les deux sens.

La réduction du membre gauche donne, étape par étape :

$$\begin{aligned} & \text{length}(\text{append}(\text{Cons}(x, l), m)) \\ = & \quad < \text{r\`egle (10.11)} > \\ & \text{length}(\text{Cons}(x, \text{append}(l, m))) \\ = & \quad < \text{r\`egle (10.9)} > \\ & 1 + \text{length}(\text{append}(l, m)) \\ = & \quad < \text{hypoth\`ese d'induction } P(l) > \\ & 1 + \text{length}(l) + \text{length}(m) \end{aligned}$$

La réduction du membre droit (où $+$ est un symbole fonctionnel d'arité 2 en notation infixée) donne :

$$\begin{aligned} & \text{length}(\text{Cons}(x, l)) + \text{length}(m) \\ = & \quad < \text{r\`egle (10.9)} > \\ & 1 + \text{length}(l) + \text{length}(m) \end{aligned}$$

Les deux membres réduits étant syntaxiquement égaux, le pas d'induction est prouvé, ce qui achève la démonstration.

10.3.2 Formalisation du raisonnement équationnel

Les raisonnements équationnels utilisés dans la partie précédente sont présentés étape par étape. Une étape consiste **uniquement** à appliquer une instance d'**une seule** égalité (axiome ou hypothèse d'induction) en **une seule** position d'un terme, pour obtenir un autre terme.

À chaque étape, on cite la règle utilisée.

Si la règle vient d'une égalité utilisable dans les deux sens, on doit préciser le sens d'utilisation de l'égalité.

10.4 Exemple de preuve inductive sur les arbres binaires

On veut démontrer que la proposition

$$Prop(t) =_{\text{def}} nb\text{-feuilles}(t) = length(ldf(t))$$

est vraie pour tout arbre binaire t . Les calculs utilisés ici ont été définis dans le chapitre 8. La preuve est inductive sur la structure de l'arbre t .

On doit vérifier la propriété pour chaque construction possible de t . Pour un constructeur récursif comme $Node : arbre(\alpha) \times arbre(\alpha) \rightarrow arbre$, le principe d'induction nous autorise à supposer que la propriété est vraie pour les deux sous-arbres t_1 et t_2 , lorsqu'on cherche à la démontrer pour l'arbre $Node(t_1, t_2)$. Plus précisément, dans cet exemple sur les arbres binaires, on prend pour *hypothèses d'induction* que la propriété $Prop$ est vraie pour les composants t_1 et t_2 , i.e. les deux hypothèses d'induction sont $Prop(t_1)$ et $Prop(t_2)$, et on prouve que la propriété est vraie pour l'arbre $Node(t_1, t_2)$.

Une telle preuve peut se rédiger comme suit :

- Premier cas (cas de base) : Soit $t = Leaf(a)$ un arbre réduit à une feuille étiquetée par $a : \alpha$.

$$Prop(Leaf(a)) =_{\text{def}} nb\text{-feuilles}(Leaf(a)) = length(ldf(Leaf(a)))$$

Il suffit ici de déplier les définitions des fonctions par simple remplacement d'un égal par un égal :

$$\begin{aligned} & nb\text{-feuilles}(Leaf(a)) \\ = & \\ & 1 \\ \text{et} & \\ & length(ldf(Leaf(a))) \\ = & \\ & length(Cons(a, Nil)) \\ = & \\ & 1 + length(Nil) \\ = & \\ & 1 + 0 \\ = & \\ & 1 \end{aligned}$$

donc la propriété est vraie pour tous les arbres réduits à une feuille.

Exercice 10.1. Ajouter une explication pour chaque égalité dans cette démonstration.

- Deuxième cas (second constructeur, pas d'induction). Soit $t = Node(t_1, t_2)$. On doit prouver

$$Prop(Node(t_1, t_2)) =_{\text{def}} nb\text{-feuilles}(Node(t_1, t_2)) = length(ldf(Node(t_1, t_2)))$$

sous les deux hypothèses d'induction

$$Prop(t_1) =_{\text{def}} nb\text{-feuilles}(t_1) = length(ldf(t_1))$$

et

$$Prop(t_2) =_{\text{def}} nb\text{-feuilles}(t_2) = length(ldf(t_2)).$$

Pour aller plus vite, on dépie les calculs simultanément dans les deux membres de $Prop(Node(t_1, t_2))$, qui devient

$$nb\text{-feuilles}(t_1) + nb\text{-feuilles}(t_2) = length(append(ldf(t_1), ldf(t_2)))$$

Ici, on voudrait pouvoir appliquer les hypothèses d'induction pour conclure, mais on ne le peut qu'en utilisant un lemme (qu'il faudrait aussi prouver). Ce lemme doit nous permettre de composer *length* avec *append*. Il faut l'inventer. On propose le lemme suivant :

$$length(append(l_1, l_2)) = length(l_1) + length(l_2) \quad (10.12)$$

ce qui permet d'obtenir

$$nb\text{-feuilles}(t_1) + nb\text{-feuilles}(t_2) = length(ldf(t_1)) + length(ldf(t_2))$$

qui permet d'appliquer les hypothèses d'induction dans le membre de gauche, pour obtenir

$$length(ldf(t_1)) + length(ldf(t_2)) = length(ldf(t_1)) + length(ldf(t_2))$$

qui termine cette démonstration.

Exercice 10.2. *Présenter ce raisonnement sous forme d'égalités successives justifiées.*

Chapitre 11

Références bibliographiques

- [BFM⁺] F. Bobot, J.-C. Filliâtre, C. Marché, G. Melquiond, and A. Paskevich. Why3 - where programs meet provers. <https://www.why3.org>.
- [Dad06] F. Dadeau. *Évaluation symbolique à contraintes pour la validation – Application à Java/JML*. Thèse de doctorat, Université de Franche-Comté, 2006.
- [Flo67] Robert W. Floyd. Assigning meanings to programs. In *Symposia in Applied Mathematics*, volume 19, pages 19–32, 1967.
- [Hoa69] C. A. R. Hoare. An axiomatic basis for computer programming. *Commun. ACM*, 12(10) :576–580, 1969.
- [Jul10] J. Julliand. *Cours et exercices corrigés d’algorithmique, vérifier, tester et concevoir des programmes en les modélisant*. 2010.
- [MNM87] B. Meyer, J.-M. Nerson, and M. Matsuo. Eiffel : Object-oriented design for software engineering. In *ESEC’87 (European Software Engineering Conference)*, volume 289 of *LNCS*, pages 221–229. Springer, 1987.
- [MP92] Zohar Manna and Amir Pnueli. *The temporal logic of reactive and concurrent systems*. Springer-Verlag New York, Inc., 1992.
- [MP95] Zohar Manna and Amir Pnueli. *Temporal verification of reactive systems : safety*. Springer-Verlag New York, Inc., 1995.
- [Pet15] G. Petiot. *Contribution à la vérification de programmes C par combinaison de tests et de preuves*. Thèse de doctorat, Université de Franche-Comté, 2015.

Solutions des exercices du cours

Chapitre 12

Solutions des exercices du cours 2 sur la logique des propositions

Exercice 2.11

Réponses : Les formules 1., 3., 4., 5., 7. et 8. sont des tautologies.

Pour la formule 8, on a :

$$\begin{aligned} & \text{expr-bool}((p \Rightarrow q) \Rightarrow ((q \Rightarrow r) \Rightarrow (p \Rightarrow r))) \\ = & \quad \text{par la définition (2.5)} \\ & \overline{\overline{p} + q} + (\overline{\overline{q} + r} + (\overline{p} + r)) \\ = & \quad \text{par De Morgan et double négation} \\ & (\overline{\overline{p}} \cdot \overline{q}) + (q \cdot \overline{r}) + \overline{p} + r \\ = & \quad \text{par associativité et commutativité} \\ & \overline{p} + \overline{\overline{p}} \cdot \overline{q} + r + (\overline{r} \cdot q) \\ = & \quad \text{avec la 3ème règle de redondance} \\ & \overline{p} + \overline{q} + r + q \\ = & \quad \text{par associativité et commutativité} \\ & \overline{p} + r + \overline{q} + q \\ & \quad \text{par la règle de partition} \\ & \overline{p} + r + 1 \\ = & \quad \text{par la règle d'absorption} \\ & 1. \end{aligned}$$

En guise d'exercice, procéder de même (en calculant l'expression booléenne associée) pour les autres formules.

Chapitre 13

Solutions des exercices du cours 3 sur la logique des prédicats

Exercice 3.2

Description de l'ensemble des symboles fonctionnels employés dans chacune des expressions suivantes :

1. $*(*(+(2, 3), 5), +(4, Moins(3)), 2))$

On a

$$F =_{\text{def}} \{*, +, 2, 3, 4, 5, Moins\}$$

avec $F_0 =_{\text{def}} \{2, 3, 4, 5\}$, $F_1 =_{\text{def}} \{Moins\}$, $F_2 =_{\text{def}} \{+, *\}$ et $F_i =_{\text{def}} \emptyset$ pour $i \geq 3$.

2. $f(g(a, h(c)), g(f(a, b, c), h(a)), a)$

$$F =_{\text{def}} \{f, g, h, a, b, c\}$$

avec $F_0 =_{\text{def}} \{a, b, c\}$, $F_1 =_{\text{def}} \{h\}$, $F_2 =_{\text{def}} \{g\}$, $F_3 =_{\text{def}} \{f\}$ et $F_i =_{\text{def}} \emptyset$ pour $i \geq 4$.

Exercice 3.3

Justification que les expressions suivantes appartiennent à T_F , où F est l'un des ensembles de symboles fonctionnels trouvés dans l'exercice précédent.

La règle d'inférence utilisée dans chacune des déductions suivantes n'est pas précisée, puisque c'est toujours la règle d'application de l'annexe B.

Pour l'expression $*(*(+(2, 3), 5), +(4, Moins(3)), 2))$, soit $F = F_0 \cup F_1 \cup F_2$ avec $F_0 = \{2, 3, 4, 5\}$, $F_1 = \{Moins\}$ et $F_2 = \{+, *\}$.

1. 2, 3, 4 et 5 sont des termes de T_F car ce sont des constantes de F (dans F_0).
2. De (1.) et $+$ $\in F_2$ on déduit $+(2, 3) : T_F$.
3. De (2.), (1.) et $*$ $\in F_2$ on déduit $*(+(2, 3), 5) : T_F$.

4. De (1.) et $Moins \in F_1$ on déduit $Moins(3) : T_F$.
5. De (1.), (4.) et $* \in F_2$ on déduit $*(4, Moins(3)) : T_F$.
6. De (5.), (1.) et $+ \in F_2$ on déduit $+(*(4, Moins(3)), 2) : T_F$.
7. De (3.), (6.) et $* \in F_2$ on déduit $*(+(2, 3), 5), +(*(4, Moins(3)), 2)) : T_F$.

Pour l'expression $f(g(a, h(c)), g(f(a, b, c), h(a)), a)$, soit $F = F_0 \cup F_1 \cup F_2$ avec $F_0 = \{a, b, c\}$, $F_1 = \{h\}$, $F_2 = \{g\}$ et $F_3 = \{f\}$.

1. De $a \in F_0$ on déduit $a : T_F$.
2. De $b \in F_0$ on déduit $b : T_F$.
3. De $c \in F_0$ on déduit $c : T_F$.
4. De (3.) et $h \in F_1$ on déduit $h(c) : T_F$.
5. De (1.) et $h \in F_1$ on déduit $h(a) : T_F$.
6. De (1.), (4.) et $g \in F_2$ on déduit $g(a, h(c)) : T_F$.
7. De (1.), (2.), (3.) et $f \in F_3$ on déduit $f(a, b, c) : T_F$.
8. De (7.), (5.) et $g \in F_2$ on déduit $g(f(a, b, c), h(a)) : T_F$.
9. De (6.), (8.), (1.) et $f \in F_3$ on déduit $f(g(a, h(c)), g(f(a, b, c), h(a)), a) : T_F$.

Exercice 3.4

Les figures 13.1 et 13.2 sont des représentations arborescentes des deux termes de l'exercice précédent.

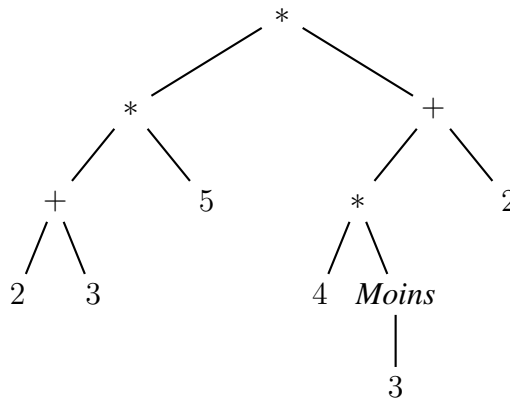


FIGURE 13.1 – Représentation arborescente du terme $*(*(+(2, 3), 5), +(*(4, Moins(3)), 2))$.

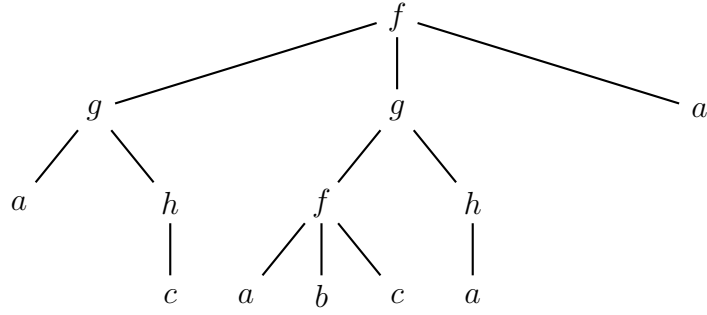


FIGURE 13.2 – Représentation arborescente du terme $f(g(a, h(c)), g(f(a, b, c), h(a)), a)$.

Questions

1. Tout terme a-t-il une représentation arborescente ?

Oui, car en supposant que $t_1, \dots, t_n$ ont une représentation arborescente, il suffit d'enraciner avec f pour obtenir une représentation arborescente de $f(t_1, \dots, t_n)$. De plus, pour un terme ouvert, on peut dire qu'une variable est une arborescence.

2. Toute arborescence est-elle un terme ?

Non. Par exemple $+(2, 3, +(2, 3))$ est une arborescence mais elle ne représente pas un terme car on ne peut pas attribuer une arité unique à $+$.

Exercice 3.5

Avec $X = \{x, y, z\}$, décrire un ensemble des symboles fonctionnels F tel que les expressions suivantes soient dans $T_F(X)$:

1. $(x + x) * (x * (4 * (Moins(x)) + 2))$

Avec $X = \{x, y, z\}$ (ou $X = \{x\}$), on a

$$F =_{\text{def}} \{+, *, Moins, 2, 4\}$$

avec $F_0 =_{\text{def}} \{2, 4\}$, $F_1 =_{\text{def}} \{Moins\}$, $F_2 =_{\text{def}} \{+, *\}$ et $F_i =_{\text{def}} \emptyset$ pour $i \geq 3$.

2. $f(g(a, h(x)), y, g(f(a, z, z), h(y)))$

Avec $X =_{\text{def}} \{x, y, z\}$, on a

$$F =_{\text{def}} \{f, g, h, a\}$$

avec $F_0 =_{\text{def}} \{a\}$, $F_1 =_{\text{def}} \{h\}$, $F_2 =_{\text{def}} \{g\}$, $F_3 =_{\text{def}} \{f\}$ et $F_i =_{\text{def}} \emptyset$ pour $i \geq 4$.

Exercice 3.6

1. Peut-on représenter un terme ouvert de façon arborescente ?

Oui, comme pour les termes clos.

2. Quelle est la position des variables ?

Les variables sont des feuilles particulières. Les autres feuilles sont les constantes.

Exercice 3.12

Soit $\varphi = (\forall y. (\exists z. R(z)) \vee (\forall z. S(y, z) \wedge \neg S(h(x, z), a)))$.

On considère l'interprétation $\mathcal{I}$ définie par $\mathcal{U}_{\mathcal{I}} = \mathbb{Z}$, $a_{\mathcal{I}} = -1$, $h_{\mathcal{I}}(x, y) = x + y$, $R_{\mathcal{I}} = \{\dots \leq 0\}$ et $S_{\mathcal{I}} = \{\dots > \dots\}$.

1. Traduire en français ce que signifie φ dans cette structure.

Dans cette structure, cette formule s'écrit

$$(\forall y. (\exists z. z \leq 0) \vee (\forall z. y > z \wedge \neg(x + z > -1)))$$

et se lit “Pour tout (entier) y , il existe un (entier) z négatif ou nul, ou, pour tout (entier) z , y est strictement supérieur à z et la somme de x et z n'est pas strictement supérieure à -1 ”.

2. Pour quelles valeurs de x la formule φ est-elle vraie dans cette structure ?

La formule φ est vraie quelle que soit la valeur de x . En effet, la sous-formule $(\exists z. z \leq 0)$ est vraie dans les entiers relatifs (il suffit de choisir 0 ou -1 pour z), donc la sous-formule

$$(\exists z. z \leq 0) \vee (\forall z. y > z \wedge \neg(x + z > -1))$$

est vraie quelle que soit la valeur de vérité de la seconde sous-formule dans cette disjonction, de même avec $\forall y$ avant cette formule.

Chapitre 14

Solutions des exercices du cours 4 sur les types inductifs

Exercice 4.1

Énoncé : Donner un exemple d'élément appartenant à chacun de ces types.

$Cons(1, Nil)$ est une liste d'entiers.

$Cons(Node(5, 2), Cons(Node(1, 2), Nil))$ est une liste d'arbres binaires (à feuilles de type entier).

$Cons(Cons(5, Nil), Nil)$ est une liste de listes d'entiers.

Exercice 4.2

Énoncé : Quelles sont les différences majeures entre une liste et un ensemble ?

Dans une liste, les éléments sont ordonnés et on peut trouver plusieurs fois le même élément. Dans un ensemble, les éléments ne sont pas ordonnés et sont en un seul exemplaire.

Exercice 4.3

Cet exercice porte sur le code reproduit dans le listing 4.1.

1. Utiliser l'interface en ligne de Why3 pour vérifier que ce code est syntaxiquement correct et bien typé.

Après avoir copié ce code dans l'interface en ligne de Why3, je tape Alt+R ou je clique sur l'icône  qui représente une baguette magique. J'obtiens l'icône  ou  dans l'onglet "Task list", et aucun message d'erreur, ce qui signifie que ce code est syntaxiquement correct et bien typé.

2. Sans introduire d'autres déclarations, proposer un autre lemme qui serait syntaxiquement correct mais qui serait incorrect du point de vue des types.

Le lemme suivant est syntaxiquement correct mais incorrect du point de vue des types.

```
lemma 16: p True False
```

3. Reproduire le message de l'interface en ligne de Why3 signalant cette erreur de typage.

Cette erreur de typage est signalée par le message suivant :

This term has type bool, but is expected to have type unit

Chapitre 15

Solutions des exercices du cours 6 sur la logique de Hoare

Exercice 6.1

Chaque itération de la boucle du programme de recherche dichotomique fait strictement décroître la différence $max - min$, tout en maintenant cette différence positive ou nulle. Puisque ce nombre entier ne peut prendre qu'un nombre fini de valeurs différentes entre sa valeur initiale et 0, le nombre d'itérations est fini, donc le programme de recherche dichotomique termine.

Exercice 6.2

La correction de la première feuille

$$\{i < n \wedge f = i!\} \ i := i + 1 \ \{i \leq n \wedge f = (i - 1)!\}$$

de cet arbre est justifiée dans la solution de l'exercice 1 de la feuille d'exercices sur la logique de Hoare.

La deuxième feuille

$$\{i \leq n \wedge f = (i - 1)!\} \ f := f * i \ \{i \leq n \wedge f = i!\}$$

de cet arbre résulte d'une application de la règle (*assignment*), qui justifie le triplet

$$\{i \leq n \wedge f * i = i!\} \ f := f * i \ \{i \leq n \wedge f = i!\},$$

et de l'équivalence mathématique entre les égalités $f = (i - 1)!$ et $f * i = i!$.

Chapitre 16

Solutions des exercices du cours 8 sur les calculs inductifs

Exercice 8.1

Le code WhyML suivant correspond à la définition de la fonction *somme*, avec le type *int* au lieu des entiers naturels.

```
let rec function somme (l : list int) : int
= match l with
| Nil → 0
| Cons a l → a + somme l
end
```

Exercice 8.2

Le but WhyML suivant permet de vérifier l'égalité 8.20 :

```
goal unfoldSomme: somme (Cons 2 (Cons 7 (Cons 3 Nil))) = 12
```

Exercice 8.3

$$\begin{aligned} \text{produit}(\text{Nil}) &= 1 \\ \text{produit}(\text{Cons}(a, l)) &= a * \text{produit}(l) \end{aligned}$$

Fonction *produit* en WhyML, avec le type `int` au lieu des entiers naturels :

```
let rec function produit (l : list int) : int
= match l with
| Nil → 1
| Cons a l → a * produit l
end
```

Exercice 8.4

Le type de la fonction *somme* est $liste(\mathbb{N}) \rightarrow \mathbb{N}$.

Vérification du typage de (8.18) :

Puisque la constante *Nil* est de type $liste(\alpha)$ pour tout type α , elle est de type $liste(\mathbb{N})$. Puisque la fonction *somme* est de type $liste(\mathbb{N}) \rightarrow \mathbb{N}$, il en résulte que l'expression $somme(Nil)$ est de type $\mathbb{N}$. Or, $0 : \mathbb{N}$, donc l'égalité (8.18) est bien typée, puisque ses deux membres ont le même type.

Vérification du typage de (8.19) :

Montrons que les deux membres de l'égalité (8.19) sont de type $\mathbb{N}$.

D'après les types de la fonction *somme* et du constructeur de listes *Cons*, le membre de gauche $somme(Cons(a, l))$ est bien typé et de type $\mathbb{N}$ si a est de type $\mathbb{N}$ et si l est de type $liste(\mathbb{N})$.

Sous ces mêmes hypothèses, le membre de droite $a + somme(l)$ est bien typé et aussi de type $\mathbb{N}$, puisque l'addition $+$ est de type $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$.

Exercice 8.5

Fonction *compte* en WhyML :

```
let rec function compteInt (a : int) (l : list int) : int
= match l with
| Nil → 0
| Cons b m → if a = b then 1 else 0 + compteInt a m
end
```

Dans cet exemple, on n'utilise pas le filtrage pour séparer les cas (8.22) et (8.23), mais l'instruction `if then else`, dont la valeur est ici l'entier 1 ou 0 selon le cas.

Exercice 8.6

On déplie le calcul :

$$\begin{aligned}
& \text{compte}(4, \text{Cons}(5, \text{Cons}(4, \text{Cons}(7, \text{Nil})))) \\
= & \\
& \text{compte}(4, \text{Cons}(4, \text{Cons}(7, \text{Nil}))) \\
= & \\
& 1 + \text{compte}(4, \text{Cons}(7, \text{Nil})) \\
= & \\
& 1 + \text{compte}(4, \text{Nil}) \\
= & \\
& 1 + 0 \\
= & \\
& 1
\end{aligned}$$

L'interface en ligne de Why3 prouve automatiquement le but suivant, ce qui vérifie que ce calcul est correct :

```
goal exempleCompte : compteInt 4 (Cons 5 (Cons 4 (Cons 7 Nil))) = 1
```

Exercice 8.7

La fonction *filtre-pair* peut être codée par la fonction WhyML **filtrePair** suivante, en utilisant le type **int** au lieu des entiers naturels et la fonction **mod** définie dans le module **int.EuclideanDivision** de la bibliothèque standard de Why3.

```
use int.EuclideanDivision

let rec function filtrePair (l : list int) : list int
= match l with
| Nil → Nil
| Cons x m →
    let k = filtrePair m in
    if mod x 2 = 0 then Cons x k else k
end
```

Exercice 8.8

$$\begin{aligned}
\text{zip}(\text{Nil}, \text{Nil}) &= \text{Nil} \\
\text{zip}(\text{Cons}(x, l), \text{Nil}) &= \text{Nil} \\
\text{zip}(\text{Nil}, \text{Cons}(y, l)) &= \text{Nil} \\
\text{zip}(\text{Cons}(x, l_1), \text{Cons}(y, l_2)) &= \text{Cons}((x, y), \text{zip}(l_1, l_2))
\end{aligned}$$

En WhyML, on peut implémenter cette fonction comme suit :

```

let rec function zip (l : list 'a) (m : list 'b) : list ('a,'b)
= match l with
| Nil → Nil
| Cons x l →
    match m with
    | Nil → Nil
    | Cons y m → Cons (x,y) (zip l m)
    end
end

```

Exercice 8.9

En WhyML, on peut implémenter cette fonction comme suit :

```

let rec function cumuler (l : list int) (s : int) : int
= match l with
| Nil → s
| Cons x l → cumuler l (x+s)
end

```

Dépliement du calcul de $\text{cumuler}(\text{Cons}(5, \text{Cons}(4, \text{Nil})), 3)$:

$$\begin{aligned}
& \text{cumuler}(\text{Cons}(5, \text{Cons}(4, \text{Nil})), 3) \\
= & \\
& \text{cumuler}(\text{Cons}(4, \text{Nil}), 5 + 3) \\
= & \\
& \text{cumuler}(\text{Nil}, 4 + (5 + 3)) \\
= & \\
& 4 + (5 + 3) \\
= & \\
& 12.
\end{aligned}$$

Chapitre 17

Solutions des exercices du cours 9 sur la preuve par récurrence

Exercice 9.2

Démontrons pour tout $n \geq 1$ la propriété $P(n)$, qui est
“La somme des n premiers nombres impairs est égale à n^2 ”,
par récurrence sur $n \geq 1$.

— **Cas de base** $P(1)$: le premier nombre impair est 1, qui est bien égal à 1^2 .

— **Pas de récurrence** :

Pour $n \geq 1$ quelconque, on admet l’hypothèse de récurrence $P(n)$ et on démontre $P(n + 1)$, qui est “La somme des $(n + 1)$ premiers nombres impairs est égale à $(n + 1)^2$ ”.

Cette somme est égale à la somme des n premiers nombres impairs augmentée par le $(n + 1)^{\text{e}}$ nombre impair, qui est $2n + 1$. Selon l’hypothèse de récurrence, la somme des n premiers nombres impairs est égale à n^2 , donc la somme des $(n + 1)$ premiers nombres impairs est égale à $n^2 + (2n + 1) = (n + 1)^2$.

□

Chapitre 18

Solutions des exercices du cours 10 sur les preuves inductives

Exercice 10.1

On reprend la démonstration du cours en ajoutant une explication pour chaque égalité dans cette démonstration.

$$\begin{aligned} & nb\text{-feuilles}(Leaf(a)) \\ = & \quad \text{par (8.13)} \\ & 1 \\ \text{et} & \\ & length(lf(Leaf(a))) \\ = & \quad \text{par (8.16)} \\ & length(Cons(a, Nil)) \\ = & \quad \text{par (8.10) avec } (Nil/l) \\ & 1 + length(Nil) \\ = & \quad \text{par (8.9)} \\ & 1 + 0 \\ = & \quad \text{par arithmétique} \\ & 0 \end{aligned}$$

Exercice 10.2

Présentation du raisonnement du cours sous la forme d'égalités successives justifiées.

On doit prouver

$$nb\text{-feuilles}(Node(t_1, t_2)) = length(lf(Node(t_1, t_2))) \quad (18.1)$$

sous les deux hypothèses d'induction suivantes, venant des prémisses de la règle d'inférence :

$$nb\text{-}feuilles(t_1) = length(ldf(t_1)) \quad (18.2)$$

$$nb\text{-}feuilles(t_2) = length(ldf(t_2)) \quad (18.3)$$

On peut aussi utiliser le lemme suivant :

$$length(append(l_1, l_2)) = length(l_1) + length(l_2). \quad (18.4)$$

On simplifie séparément le membre gauche et le membre droit de l'égalité (18.1) :

$$\begin{aligned} & nb\text{-}feuilles(Node(t_1, t_2)) \\ = & \quad \quad \quad \text{par (8.14)} \\ & nb\text{-}feuilles(t_1) + nb\text{-}feuilles(t_2) \\ = & \quad \quad \quad \text{par (18.2)} \\ & length(ldf(t_1)) + nb\text{-}feuilles(t_2) \\ = & \quad \quad \quad \text{par (18.3)} \\ & length(ldf(t_1)) + length(ldf(t_2)) \end{aligned}$$

et

$$\begin{aligned} & length(ldf(Node(t_1, t_2))) \\ = & \quad \quad \quad \text{par (8.17)} \\ & length(append(ldf(t_1), ldf(t_2))) \\ = & \quad \quad \quad \text{par le lemme (18.4) avec } (ldf(t_1)/l_1, ldf(t_2)/l_2) \\ & length(ldf(t_1)) + length(ldf(t_2)) \end{aligned}$$

Annexes

Annexe A

Algèbre de Boole

A.1 Propriétés générales

A.1.1 Définition

DÉFINITION A.1 (ALGÈBRE DE BOOLE). On appelle algèbre de Boole la structure algébrique définie par un ensemble (non vide) $\mathcal{A}$ et trois opérations :

- la somme booléenne (binaire) : $+$,
- le produit booléen (binaire) : $\cdot$,
- la négation booléenne (unaire) : $\bar{}$ (ex. $\bar{a}$).

Ces opérations doivent de plus posséder les propriétés données dans le tableau A.1, pour que l'on puisse dire que le quadruplet $(\mathcal{A}, +, \cdot, \bar{})$ est une algèbre de Boole.

Le tableau A.1 met en parallèle les mêmes propriétés, écrites en utilisant

- soit les notations générales d'une algèbre de Boole,
- soit les notations ensemblistes, définies lorsque $\mathcal{A}$ est l'ensemble $\mathcal{P}(E)$ des parties d'un ensemble E : c'est une algèbre de Boole particulière, bien connue, et qui possède des notations spécifiques.

REMARQUE A.1. Les signes opératoires utilisés sont les mêmes que ceux de l'addition et de la multiplication des réels. Cependant, ces opérations n'ont évidemment pas les mêmes propriétés, et ne portent pas sur les mêmes éléments.

A.1.2 Règles de calcul dans une algèbre de Boole

A.1.2.1 Particularités du calcul booléen

1. Les priorités habituelles sont respectées pour la somme et le produit booléen.
2. Les éléments neutres sont notés 0 et 1, par analogie avec les entiers de même symbole (ne pas oublier que ces calculs ne se déroulent pas dans $\mathbb{R}$...)

Propriété	$\mathcal{P}(E)$	$\mathcal{A}$
idempotence	$A \cup A = A$ $A \cap A = A$	$a + a = a$ $a \cdot a = a$
commutativité	$A \cup B = B \cup A$ $A \cap B = B \cap A$	$a + b = b + a$ $a \cdot b = b \cdot a$
associativité	$A \cup (B \cup C) = (A \cup B) \cup C$ $A \cap (B \cap C) = (A \cap B) \cap C$	$a + (b + c) = (a + b) + c$ $a \cdot (b \cdot c) = (a \cdot b) \cdot c$
éléments neutres	$A \cup \emptyset = A$ $A \cap E = A$	$a + 0 = a$ $a \cdot 1 = a$
absorption	$A \cup E = E$ $A \cap \emptyset = \emptyset$	$a + 1 = 1$ $a \cdot 0 = 0$
distributivités	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	$a \cdot (b + c) = a \cdot b + a \cdot c$ $a + b \cdot c = (a + b) \cdot (a + c)$
involution	$E \setminus (E \setminus A) = A$	$\overline{\overline{a}} = a$
complémentation	$E \setminus \emptyset = E$ $E \setminus E = \emptyset$	$\overline{0} = 1$ $\overline{1} = 0$
partition	$A \cup (E \setminus A) = E$ $A \cap (E \setminus A) = \emptyset$	$a + \overline{a} = 1$ $a \cdot \overline{a} = 0$
« Lois de De Morgan »	$E \setminus (A \cup B) = (E \setminus A) \cap (E \setminus B)$ $E \setminus (A \cap B) = (E \setminus A) \cup (E \setminus B)$	$\overline{a + b} = \overline{a} \cdot \overline{b}$ $\overline{a \cdot b} = \overline{a} + \overline{b}$

TABLE A.1 – Propriétés d’une algèbre de Boole

3. L'absence d'éléments symétriques pour la somme et pour le produit interdit les simplifications que l'on a l'habitude de pratiquer « sans y réfléchir » :

— $a + b = a + c$ ne donne pas $b = c$,

— $ab = ac$ n'entraîne pas $b = c$.

En particulier, ne jamais perdre de vue que

— $a + b = 0$ n'est réalisable en algèbre de Boole que si $a = b = 0$

— $a.b = 1$ n'est réalisable en algèbre de Boole que si $a = b = 1$ ($A \cap B = E \Leftrightarrow A = E$ et $B = E$)

— $a.b = 0$ peut être réalisé avec $a \neq 0$ et $b \neq 0$ (par exemple, avec $b = \bar{a}$, mais ce n'est pas la seule solution...). On parle de « diviseurs de zéro ». (Ainsi, $A \cap B = \emptyset$ est possible sans avoir obligatoirement $A = \emptyset$ et $B = \emptyset$).

4. Il y a deux distributivités. Celle de la somme (booléenne) sur le produit (booléen) n'est pas habituelle. Par exemple, on a $(a + b)(a + c)(a + d)(a + e)(a + f) = a + bcdef$!

A.1.2.2 Règles de « redondance »

Dans une expression booléenne, une sous-expression est dite « redondante » lorsqu'on peut la supprimer sans changer la « valeur » de l'expression :

1. Dans une somme booléenne, tout terme absorbe ses multiples.

Autrement dit : $a + a \cdot b = a$.

PREUVE En effet, $a + a \cdot b = a \cdot (\bar{b} + b) + a \cdot b = a \cdot \bar{b} + a \cdot b + a \cdot b = a \cdot \bar{b} + a \cdot b$ (par idempotence) $= a \cdot (\bar{b} + b) = a$. ■

2. Dans un produit booléen, tout facteur absorbe tout autre facteur qui le contient en tant que terme.

Autrement dit : $a \cdot (a + b) = a$.

PREUVE En effet, $a \cdot (a + b) = a \cdot a + a \cdot b = a + a \cdot b = a$. ■

3. Enfin, la troisième règle de redondance s'exprime par :

$$a + \bar{a} \cdot b = a + b$$

PREUVE $a + \bar{a} \cdot b = (a + \bar{a}) \cdot (a + b) = 1 \cdot (a + b) = a + b$. ■

EXEMPLE A.1. $ab + \bar{a}c + \bar{b}c = ab + (\bar{a} + \bar{b}) \cdot c = ab + \overline{ab} \cdot c = ab + c$

REMARQUE A.2. L'application de cette troisième règle peut être combinée avec celle des autres, comme par exemple dans le calcul suivant :

$$a \cdot b + \bar{a} \cdot c + b \cdot c = a \cdot b + \bar{a} \cdot c$$

PREUVE $a \cdot b + \bar{a} \cdot c + b \cdot c = a \cdot b + \bar{a} \cdot c + (a + \bar{a}) \cdot b \cdot c = a \cdot b + \bar{a} \cdot c + a \cdot b \cdot c + \bar{a} \cdot b \cdot c$; $a \cdot b$ absorbe $a \cdot b \cdot c$ et $\bar{a} \cdot c$ absorbe $\bar{a} \cdot b \cdot c$, d'où le résultat. ■

A.2 Fonctions booléennes

A.2.1 Définitions

Soit $\mathcal{A}$ une algèbre de Boole.

DÉFINITION A.2 (FONCTION BOOLÉENNE). *On appelle fonction booléenne de n variables toute application de $\mathcal{A}^n$ dans $\mathcal{A}$ dont l'expression ne contient que :*

- *les symboles des opérations booléennes,*
- *des symboles de variables, de constantes,*
- *d'éventuelles parenthèses.*

EXEMPLE A.2. $f(a, b, c) = a \cdot \bar{b} + c$.

REMARQUE A.3. Si a est une variable booléenne, elle peut intervenir dans l'expression d'une fonction booléenne sous la forme a ou sous la forme $\bar{a}$, qui sont appelées les deux aspects de cette variable : affirmé et nié.

DÉFINITION A.3 (FONCTION BOOLÉENNE NULLE). *On appelle fonction booléenne nulle (à n variables) la fonction booléenne qui, à chaque valeur des variables, associe la valeur 0.*

Son expression est $f(x_1, x_2, \dots, x_n) = 0$.

DÉFINITION A.4 (FONCTION RÉFÉRENTIEL). *On appelle fonction référentiel (à n variables) la fonction booléenne qui, à chaque valeur des variables, associe la valeur 1.*

Son expression est $f(x_1, x_2, \dots, x_n) = 1$.

Annexe B

Vérification et inférence de types

Cette annexe montre comment le typage permet à l'informaticien d'éviter certaines erreurs de spécification, de programmation ou de logique. Il lui suffit de connaître deux algorithmes fondamentaux sur les types :

1. La vérification des erreurs de typage, qui détecte si une expression e est correctement typée selon un type donné T . Ceci revient à prouver que la formule $e : T$ est un théorème, selon un système déductif formel explicitant les règles de typage.
2. L'inférence de type, qui trouve un type correct pour une expression e , en fonction du type de ses ingrédients, qui sont ses variables et ses symboles fonctionnels et relationnels si cette expression est une formule de logique du premier ordre. Ce problème n'a pas toujours de solution algorithmique. Nous verrons ici qu'il en a une pour les types génériques. Il y a souvent plusieurs solutions, mais dans ce systèmes de types, on peut trouver par déduction automatique la plus générale des solutions.

B.1 Introduction

B.1.1 Pourquoi des types ?

On comprend aisément que l'attribution d'un type particulier à chaque élément d'une expression en clarifie le sens.

Dans ce but, de nombreux langages de programmation disposent d'un système de types (dits *de base*, *élémentaires* ou *prédéfinis*) et de règles de construction de nouveaux types par l'utilisateur. Ces règles *de typage* permettent d'exprimer quels opérateurs et quelles fonctions sont applicables à quelles expressions, éliminant ainsi certaines erreurs de programmation.

B.1.2 Importance des types pour la cohérence des logiques

L'importance des types a été soulignée par le paradoxe de Russel dans la théorie des ensembles. Ce paradoxe est le même que le paradoxe du barbier qui s'énonce ainsi : "Dans le village, le barbier rase tout homme qui ne se rase pas lui-même. Qui rase le barbier?". On peut

aussi bien répondre par “le barbier” que par “certainement pas le barbier”, ce qui rend équivalent une expression “le barbier rase le barbier” avec sa négation “le barbier ne rase pas le barbier”.

Cela crée donc un paradoxe, autrement dit une incohérence.

Dans la théorie des ensembles, ce paradoxe s’énonce par : “Soit S l’ensemble des ensembles qui n’ont pas eux-mêmes pour élément. Est-ce que $S \in S$?”. La paradoxe est qu’on a $S \in S \Leftrightarrow S \notin S$.

Un moyen d’éviter le paradoxe est d’introduire un type pour l’*ensemble* des valeurs d’un certain type, par exemple le type $set(\mathbb{N})$ pour les ensembles d’entiers naturels. Avec ce type *ensemble*, ce paradoxe ne peut pas exister car, pour avoir $S \in S$, il faudrait que S soit tout à la fois de type $set(\mathbb{N})$ et de type $set(set(\mathbb{N}))$, ce qui est considéré comme impossible.

B.1.3 Importance des types pour les spécifications

Connaître le type d’une expression, c’est connaître l’ensemble des valeurs qu’elle est susceptible de prendre quand elle est évaluée. C’est la moindre des choses quand on est en train de spécifier un programme.

Exercice B.1. *Quel est le type des expressions suivantes ?*

1. $(x + 2) * y$

2. $append(Cons(a, l), ldf(t))$

Cons est un constructeur de listes, append est la fonction de concaténation de deux listes et ldf est la fonction qui retourne la liste des feuilles d’un arbre.

C’est facile si vous connaissez le type des ingrédients : fonctions, variables et constructions utilisées. Avoir une discipline de type, c’est se forcer à plus de précision et de rigueur dans l’expression et donc éviter des erreurs, voire des paradoxes.

B.1.4 Importance des types pour la programmation

Il y a toujours un typage implicite ou explicite dans les expressions et les constructions d’un programme : on manipule des ensembles de valeurs, on structure des données. Cependant les types contiennent des informations sur les valeurs qui ne sont pas toujours précisées ni exigées dans le programme.

À tout langage de programmation est attaché un certain nombre de regroupements des types de base que l’on peut utiliser.

Exercice B.2. *Quels sont les types possibles en C ?*

Le langage C qui demande de préciser le type des variables et vérifie un typage cohérent à la compilation du programme est dit à *typage fort*. Un langage qui n’exige rien de la sorte est dit à *typage faible*.

Il est important de se poser la question et d’écrire (éventuellement en commentaire) quels sont les types des fonctions, des constantes et des domaines des variables utilisés, pour :

- une documentation précise des programmes,
 - une première approche de la vérification de la correction de ces programmes.
- Il a été dit que l'exécution des programmes bien typés “*never goes wrong*”.

B.1.5 Origine des types génériques

Vous connaissez tous le type tableau dans un langage de programmation. Il peut y avoir des tableaux d'entiers, des tableaux de réels, des tableaux de booléens, mais aussi des tableaux de tableaux d'entiers, etc. Pour regrouper tous ces types sous le même concept de tableau, nous dirons que le type “tableau” est *générique* (on dit aussi *paramétrique* ou *polymorphe*). Dans cette annexe, nous utiliserons les types génériques des listes et des arbres présentés dans le chapitre 4.

B.1.6 Le système Hindley-Milner

Le système Hindley-Milner est un système de types avec inférence automatique des types, défini par :

- Un ensemble de types de base (int, bool, etc.)
- Des types fonctionnels, de la forme $T_1 \rightarrow T_2$
- Des types polymorphes, de la forme $\forall \alpha. T$
- Des règles d'inférence pour déterminer les types des expressions.

Par exemple, le type de la fonction identité est $\forall \alpha. \alpha \rightarrow \alpha$, où α est une variable de type, car on peut définir cette fonction sur tout type. En pratique, dans toute la suite, on n'écrit jamais les $\forall \alpha$ pour les types avec variables de type, qui sont faciles à distinguer car elles sont toujours désignées par une lettre de l'alphabet grec.

B.2 Définitions et notations

La notation $e : T$ est usuellement utilisée pour exprimer que l'expression e est de type T . Même si cette notation ressemble à la syntaxe de la déclaration de variables dans votre langage de programmation favori, ce n'est pas une déclaration, mais une affirmation logique (vraie ou fausse).

On peut donner diverses interprétations pour $e : T$, mais la plus simple est largement suffisante pour tout ce module. Elle consiste à considérer un type comme un ensemble (parfois infini) de données de même nature. On peut interpréter $e : T$ par “ e appartient à T ”, ce qui signifie que toutes les valeurs possibles de l'expression e appartiennent à l'ensemble T . Cependant, on distinguera soigneusement le symbole “:” du symbole “ $\in$ ” qui exprime l'appartenance à un ensemble, ne serait-ce que pour pouvoir associer un type à ce symbole en écrivant $\in : T$.

Pour désigner un type générique, on emploie une variable substituable par un type quelconque, appelée *variable de type*. Pour distinguer ces variables de type des variables substituables par des termes ou par des expressions, notées $x, y, z, \dots$, nous désignerons les variables de types par des lettres grecques α, β, γ , etc, éventuellement avec des indices, comme dans α_3

ou β_1 . Ainsi, le type générique $liste(\alpha)$ signifie que la variable α peut être substituée par n'importe quel type (générique ou pas). Par exemple, avec la substitution $[\alpha := \mathbb{N}]$, on obtient le type $liste(\mathbb{N})$ des listes d'entiers naturels.

On utilisera aussi le type générique $arbre(\alpha)$ des arbres binaires à feuilles de type quelconque.

B.2.1 Les constructeurs des types génériques

Un constructeur d'un type T est une fonction qui explique comment construire certains éléments du type T . Lorsque le type T est générique, ses constructeurs ont un type qui dépend de la variable de type, comme dans les trois exemples suivants.

B.2.2 Type des listes

Le type générique $liste(\alpha)$ dispose des constructeurs suivants :

- $Nil : liste(\alpha)$, qui construit une liste vide
- $Cons : \alpha \times liste(\alpha) \rightarrow liste(\alpha)$, tel que $Cons(x, l)$ désigne la liste obtenue en ajoutant x (de type α) au début de la liste l .

On donne à ces constructeurs une propriété de *liberté* qui inscrit le type $liste$ comme un cas particulier de type inductif. Les types inductifs, dont le type $liste$, ont été introduits dans le chapitre 4.

La liberté des constructeurs de listes peut s'exprimer par les relations suivantes

$$Cons(x_1, l_1) = Cons(x_2, l_2) \Rightarrow (x_1 = x_2) \wedge (l_1 = l_2)$$

$$Cons(x, l) \neq Nil$$

qui signifient que toute liste admet une unique expression composée de Nil et de $Cons$.

B.2.3 Type des arbres binaires

Les constructeurs $Node$ et $Leaf$ du type $arbre(\alpha)$ des arbres binaires à feuilles de type α ont pour type :

$$Leaf : \alpha \rightarrow arbre(\alpha)$$

$$Node : arbre(\alpha) \times arbre(\alpha) \rightarrow arbre(\alpha)$$

Il ne faut surtout pas confondre les constructeurs des éléments du type, comme $Node$ et $Leaf$, avec le type générique $arbre(\alpha)$ lui-même.

B.2.4 Type des ensembles

Si α est un type, le type des ensembles dont tous les éléments sont de type α est noté $set(\alpha)$.

Remarque : Ce type est aussi parfois noté $\{\alpha\}$, mais cette seconde notation est en conflit avec le constructeur de singletons : Usuellement, $\{x\}$ désigne l'ensemble dont le seul élément est x .

Exercice B.3. On désigne par $\mathcal{P}(E)$ l'ensemble des parties (sous-ensembles) de l'ensemble E . Le symbole $\subseteq$ traduit l'inclusion large d'un ensemble dans un autre, qui inclut le cas où les deux ensembles sont égaux.

1. Proposer un type pour $\mathcal{P}$.
2. Expliquer pourquoi l'expression $x \subseteq \mathcal{P}(x)$ est douteuse.
3. Expliquer pourquoi l'expression $x \in \mathcal{P}(x)$ peut être correcte.

B.2.5 Les fonctions à type générique

On a vu qu'une fonction peut avoir un type générique, tel que $\alpha \rightarrow \beta$ ou $\alpha \times \text{liste}(\alpha) \rightarrow \alpha$.

Quel est le type le plus général possible qu'on peut donner à une fonction donnée ? A priori, c'est α , mais ce n'est pas très informatif. En fait, le type *le plus général* que l'on cherche à écrire doit être *le plus informatif* possible, donc le plus instancié possible, dans l'ensemble des plus généraux.

Par exemple, la fonction *miroir* d'une liste a le type le plus général $\text{liste}(\alpha) \rightarrow \text{liste}(\alpha)$. C'est une instance de α , de $\alpha \rightarrow \beta$, de $\alpha \rightarrow \alpha$, de $\text{liste}(\alpha) \rightarrow \beta$, de $\beta \rightarrow \text{liste}(\alpha)$, de $\text{liste}(\alpha) \rightarrow \text{liste}(\beta)$. Tous ces types seraient plus généraux que $\text{liste}(\alpha) \rightarrow \text{liste}(\alpha)$. Mais c'est $\text{liste}(\alpha) \rightarrow \text{liste}(\alpha)$ qui est la bonne réponse car si on l'instancie un peu plus, comme par exemple $\text{liste}(\mathbb{N}) \rightarrow \text{liste}(\mathbb{N})$, on a trop perdu en généralité.

La fonction identité a son type le plus général qui se note $\alpha \rightarrow \alpha$, car cette fonction existe dans n'importe quel type. De même, on peut donner à l'égalité le type le plus général suivant $\alpha \times \alpha \rightarrow \mathbb{B}$ ¹.

Quelqu'un (Ph. Wadler) a lancé le défi suivant : “Donnez moi le type le plus général d'une fonction générique et je vous dirai ce qu'elle est !” Autrement dit, pas besoin de programme dans ces cas-là. Le type **est** le programme.

C'est le cas pour la fonction identité qui est la seule fonction qui peut avoir $\alpha \rightarrow \alpha$ comme type le plus général. Une fonction d'un type aussi général ne peut que ne rien faire.

Sans aller jusqu'aux types qui sont des programmes, nous verrons plus loin comment un bon typage limite considérablement les erreurs de programmation.

B.2.6 Expressions de type

Les expressions de type sont formées avec :

- des types de base tels que $\mathbb{N}$, $\mathbb{B}$ (souvent noté $\{\text{vrai}, \text{faux}\}$), $\mathbb{Z}$, $\mathbb{R}$ ou d'autres symboles d'ensembles connus ;
- des produits cartésiens ($\times$) d'ensembles, par exemple $A \times B$, ensemble des couples formés d'un élément de A et d'un élément de B ;
- le constructeur *set* tel que $\text{set}(A)$ désigne le type des ensembles dont tous les éléments sont de type A , c'est-à-dire l'ensemble des parties de A , que l'on note aussi $\mathcal{P}(A)$ ou 2^A , notation anglo-saxonne ;

1. En toute rigueur, ceci est le type de l'égalité *décidable*, mais ceci est ignoré dans ce cours.

- le constructeur des ensembles de fonctions, par exemple $A \rightarrow B$ est l'ensemble des fonctions de A dans B ;
- d'autres constructeurs de types, comme par exemple $liste(\dots)$;

Dans les expressions de type, les *variables de type* (substituables par des expressions de type) sont notées par des lettres grecques : $\alpha, \beta, \gamma, \dots$

B.3 Règles de typage

Cette partie suppose connues les notions générales de système formel, de règle de déduction et de démonstration formelle présentées dans la partie 6.5. Les *règles de typage* sont des règles de déduction particulières, qui définissent la relation de typage $e : t$ entre une expression e et une expression de type t , car leurs prémisses et leur conclusion sont de la forme $e : t$, qui se lit “ e est de type t ”.

On utilise souvent la règle d'*application* suivante, aussi appelée *élimination d'une fonction*. Elle définit sous quelles conditions sur les types on peut appliquer une fonction f d'arité n à n expressions, désignées par $e_1, \dots, e_n$ dans cette règle.

$$\frac{f : \alpha_1 \times \dots \times \alpha_n \rightarrow \beta, \quad e_1 : \alpha_1, \quad \dots, \quad e_n : \alpha_n}{f(e_1, \dots, e_n) : \beta} \quad (\text{application})$$

Cette règle impose que le type α_i de chacun des arguments e_i ($1 \leq i \leq n$) de la fonction f soit le type déclaré pour cet argument dans le type de la fonction f . Si c'est le cas, la règle dit qu'on peut en déduire que le type de l'application $f(e_1, \dots, e_n)$ de cette fonction à ces arguments est le type β déclaré comme type de retour de cette fonction f (à droite de la flèche ($\rightarrow$) dans la première prémisses de cette règle.

La règle suivante, appelée *abstraction* ou *introduction d'une fonction*, permet de déduire le type d'une expression construite avec le constructeur λ des fonctions.

$$\frac{x_1 : \alpha_1, \dots, x_n : \alpha_n \vdash e : \beta}{(\lambda x_1, \dots, x_n. e) : \alpha_1 \times \dots \times \alpha_n \rightarrow \beta}$$

Exercice B.4. *Quel est le type des expressions suivantes ?*

1. $\lambda l. \text{append}(\text{Cons}(a, l), \text{ldf}(t))$, où λ est le constructeur de fonction.
2. $\lambda x, y. x \in f(f(y))$

Les parties suivantes montrent comment utiliser ces règles soit pour démontrer qu'une expression est bien typée (vérification de type), soit pour deviner le type le plus général possible pour une expression donnée (inférence de type).

B.4 Exemples

Le type inductif $pf(ap)$ des formules propositionnelles sur le type ap des propositions atomiques est défini dans la partie 4.2.4.3. L'exemple suivant montre comment démontrer qu'une expression donnée est de type $pf(ap)$.

EXEMPLE B.5. On suppose ici que le type ap est le type énuméré qui ne contient que deux constantes, notées p et q . Autrement dit, on a les deux hypothèses $p : ap$ et $q : ap$.

Une preuve formelle que l'expression $Et(Atom(q), Non(Atom(p)))$ est une formule propositionnelle bien formée, de type $pf(ap)$, peut être présentée de manière arborescente, comme suit.

$$\begin{array}{c}
 \frac{\frac{Atom : ap \rightarrow pf(ap), \quad q : ap}{(4.6), \quad Atom(q) : pf(ap)}, \quad \frac{Non : pf(ap) \rightarrow pf(ap), \quad \frac{Atom : ap \rightarrow pf(ap), \quad p : ap}{Atom(p) : pf(ap)}}{Non(Atom(p)) : pf(ap)}}{Et(Atom(q), Non(Atom(p))) : pf(ap)}
 \end{array}$$

Pour gagner de la place, l'hypothèse $Et : pf(ap) \times pf(ap) \rightarrow pf(ap)$ est remplacée ici par son numéro (4.6) dans la définition du type $pf(ap)$ de la partie 4.2.4.3.

Chaque étape d'une telle preuve s'appelle une *dédution*. Chaque déduction est une *instance* (un cas particulier) d'une règle de typage. Le nom de la règle instanciée ici n'est pas indiqué, car c'est toujours la règle (*application*). Dans cette présentation arborescente, chaque déduction est représentée par une fraction. Vous pouvez vérifier que chaque fraction dans cette preuve est un cas particulier de la règle (*application*), en identifiant ce que sont f , n , e_1 , etc dans ce cas particulier.

On remarque que cet *arbre de preuve* (de typage) a la même forme qu'un arbre qui représenterait l'expression elle-même, et que cette preuve est la seule qui respecte les règles de typage, cette solution est unique.

Exercice B.6. Utiliser les règles de typage et les types déclarés pour les constructeurs de listes et de booléens pour prouver formellement que l'expression

$$Cons(0, Cons(1, Cons(1, Nil)))$$

est de type $liste(\mathbb{B})$. Présenter cette preuve comme un arbre, comme dans l'exemple précédent.

B.5 Vérification du type d'une expression

La *vérification de type* consiste à vérifier qu'une expression e donnée a un type donné t . Une manière de le faire est de (dé)montrer la formule $e : t$ en n'utilisant que les règles de typage, considérées comme correctes. Il s'agit donc de démontrer un théorème de la forme

$$H_1, \dots, H_n \vdash e : t$$

dont la conclusion est cette formule $e : t$ et dont les hypothèses sont les types admis pour certaines constantes, certains constructeurs de types et certaines fonctions de calcul sur ces types.

B.5.1 Premier exemple

Par exemple, montrons que

$$\begin{array}{l}
 \text{append} : \text{liste}(\alpha) \times \text{liste}(\alpha) \rightarrow \text{liste}(\alpha), \\
 \text{Cons} : \alpha \times \text{liste}(\alpha) \rightarrow \text{liste}(\alpha), \\
 \text{ldf} : \text{arbre}(\alpha) \rightarrow \text{liste}(\alpha), \\
 a : A, \\
 l : \text{liste}(A), \\
 t : \text{arbre}(A)
 \end{array}
 \vdash \text{append}(\text{Cons}(a, l), \text{ldf}(t)) : \text{liste}(A)$$

ce qui prouverait que l'expression $\text{append}(\text{Cons}(a, l), \text{ldf}(t))$ est correctement typée et a bien le type $\text{liste}(A)$.

Par manque de place en largeur, on ne présente pas la preuve comme un arbre, mais on présente chacune de ces étapes comme une déduction séparée. Toutes les déductions suivantes utilisent la règle (*application*).

1.

$$\frac{\text{ldf} : \text{arbre}(A) \rightarrow \text{liste}(A), \quad t : \text{arbre}(A)}{\text{ldf}(t) : \text{liste}(A)}$$

2.

$$\frac{\text{Cons} : A \times \text{liste}(A) \rightarrow \text{liste}(A), \quad a : A, \quad l : \text{liste}(A)}{\text{Cons}(a, l) : \text{liste}(A)}$$

3.

$$\frac{
 \begin{array}{l}
 \text{append} : \text{liste}(A) \times \text{liste}(A) \rightarrow \text{liste}(A), \\
 \text{Cons}(a, l) : \text{liste}(A), \\
 \text{ldf}(t) : \text{liste}(A)
 \end{array}
 }{\text{append}(\text{Cons}(a, l), \text{ldf}(t)) : \text{liste}(A)}$$

Dans cette troisième étape, les trois hypothèses ne sont pas présentées horizontalement mais verticalement, car la ligne n'est pas assez large.

Il est facile de constater et de se convaincre que pour toute expression e effectivement de type t , il n'existe qu'une seule telle preuve de typage, qu'on peut obtenir en considérant le type du symbole fonctionnel le plus externe (le plus à gauche) dans l'expression, et que cette vérification s'arrête sur les feuilles de (l'arbre de) l'expression, qui sont ses constantes. Ainsi, cette vérification est un algorithme automatisable, de faible complexité, car son nombre d'étapes est égal au nombre de symboles dans l'expression.

B.6 Inférence du type d'une expression

L'*inférence de type* consiste à trouver le type le plus général possible, s'il existe, pour une expression e donnée, ou sinon expliquer pourquoi l'expression n'est pas typable.

Dans cette partie, nous présentons à travers quelques exemples un algorithme d'inférence de type. Il fonctionne comme l'algorithme précédent de vérification de type, avec seulement les

différences suivantes. D'abord, on démarre l'algorithme en essayant de vérifier que le type de l'expression e est une variable de type α_1 (ou lieu de t donné en vérification de type). Ensuite, au cours de l'algorithme, on trouve des conditions nécessaires sur cette variable de type α_1 et d'autres variables de type α_2 qui apparaissent progressivement, pour qu'on puisse continuer la vérification. Enfin, on détermine si cet ensemble de conditions sur ces variables de type a une solution. Si oui, sa solution la plus générale donne un type à l'expression e (et à toutes ses sous-expressions). Sinon, la première étape où une contradiction apparaît indique où se situe l'erreur de typage.

Dans la plupart des exemples suivants, les hypothèses sont nommées $(H_1), \dots$, afin d'écrire des déductions plus courtes, comme dans l'exemple suivant, dans lequel les hypothèses sont nommées de (H_1) à (H_7) .

B.6.1 Premier exemple d'inférence de type

Avec les hypothèses

$$\begin{aligned} (H_1) \quad & \text{foo} : \alpha \times \text{liste}(\alpha) \rightarrow \text{liste}(\alpha), \\ (H_2) \quad & \text{Cons} : \alpha \times \text{liste}(\alpha) \rightarrow \text{liste}(\alpha), \\ (H_3) \quad & \text{append} : \text{liste}(\alpha) \times \text{liste}(\alpha) \rightarrow \text{liste}(\alpha), \\ (H_4) \quad & a : \alpha, \\ (H_5) \quad & x : \alpha, \\ (H_6) \quad & l : \text{liste}(\alpha), \\ (H_7) \quad & \text{Nil} : \text{liste}(\alpha) \end{aligned}$$

montrons que l'expression

$$\text{Cons}(\text{append}(\text{Cons}(a, \text{Nil}), l), \text{foo}(x, l))$$

est typable et trouvons son type le plus général.

On suppose que l'expression $\text{Cons}(\text{append}(\text{Cons}(a, \text{Nil}), l), \text{foo}(x, l))$ est de type α_1 et on cherche une preuve de cette affirmation. L'expression sera correctement typée si les conditions nécessaires à son bon typage ne sont pas contradictoires.

1. La seule règle applicable pour démontrer

$$\text{Cons}(\text{append}(\text{Cons}(a, \text{Nil}), l), \text{foo}(x, l)) : \alpha_1$$

est la règle (*application*). D'après le type du constructeur *Cons*, donné par l'hypothèse (H_2) , on ne peut appliquer cette règle que s'il existe un type α_2 tel que $\alpha_1 = \text{liste}(\alpha_2)$. Cette égalité $\alpha_1 = \text{liste}(\alpha_2)$ est la première condition nécessaire de bon typage. On peut donc écrire la déduction suivante, qui est une instance de la règle (*application*).

$$\frac{\begin{array}{l} (H_2) [\alpha := \alpha_2], \\ \text{append}(\text{Cons}(a, \text{Nil}), l) : \alpha_2, \\ \text{foo}(x, l) : \text{liste}(\alpha_2) \end{array}}{\text{Cons}(\text{append}(\text{Cons}(a, \text{Nil}), l), \text{foo}(x, l)) : \text{liste}(\alpha_2)} \quad (\text{application})$$

Dans cette déduction, la notation $(H_2) [\alpha := \alpha_2]$ désigne la déclaration de type

$$Cons : \alpha_2 \times liste(\alpha_2) \rightarrow liste(\alpha_2)$$

obtenue en remplaçant α par α_2 dans l'hypothèse (H_2) .

2. De même, d'après (H_3) , une condition nécessaire pour $append(Cons(a, Nil), l) : \alpha_2$ est $\alpha_2 = liste(\alpha_3)$. On peut alors faire la déduction suivante :

$$\frac{(H_3)[\alpha := \alpha_3], \quad Cons(a, Nil) : liste(\alpha_3), \quad l : liste(\alpha_3)}{append(Cons(a, Nil), l) : liste(\alpha_3)} \quad (application)$$

La prémisse $l : liste(\alpha_3)$ est l'instance $(H_6) [\alpha := \alpha_3]$ de l'hypothèse (H_3) qui déclare que l est une liste quelconque (générique).

3. La déduction

$$\frac{Cons : \alpha_3 \times liste(\alpha_3) \rightarrow liste(\alpha_3), \quad a : \alpha_3, \quad Nil : liste(\alpha_3)}{Cons(a, Nil) : liste(\alpha_3)} \quad (application)$$

justifie la deuxième prémisse de la déduction précédente. Sa prémisse $a : \alpha_3$ est $(H_4) [\alpha := \alpha_3]$ et sa prémisse $Nil : liste(\alpha_3)$ est $(H_7) [\alpha := \alpha_3]$, ce qui achève cette branche de la preuve.

4. Pour la dernière branche de cette preuve, on peut instancier la règle *(application)* pour obtenir la déduction suivante :

$$\frac{(H_1) [\alpha := \alpha_2], \quad x : \alpha_2, \quad l : liste(\alpha_2)}{foo(x, l) : liste(\alpha_2)} \quad (application)$$

Sa prémisse $x : \alpha_2$ est $(H_5) [\alpha := \alpha_2]$. Sa prémisse $l : liste(\alpha_2)$ est l'instance $(H_6) [\alpha := \alpha_2]$ de l'hypothèse (H_3) . Ceci complète l'arbre de preuve, si l'ensemble de conditions n'est pas contradictoire.

Les conditions $\alpha_1 = liste(\alpha_2)$ et $\alpha_2 = liste(\alpha_3)$ ne sont pas contradictoires. Elles impliquent l'égalité $\alpha_1 = liste(liste(\alpha_3))$. On a donc construit une preuve complète que le type $liste(liste(\alpha_3))$ est correct pour l'expression

$$Cons(append(Cons(a, Nil), l), foo(x, l))$$

et c'est son type le plus général possible.

B.6.2 Exemple d'inférence de type avec une erreur de typage

Voici un exemple de recherche de preuve qui échoue.

Avec les mêmes hypothèses que dans la partie B.6.1, sauf (H_6) , cherchons une preuve que l'expression

$$\lambda l. Cons(append(Cons(a, Nil), l), foo(x, l))$$

a un type. Pour déduire un type pour cette expression, on ne peut appliquer que la règle (*abstraction*), avec $n = 1$. Ce type sera $\alpha \rightarrow \beta$ si l'on peut démontrer

$$l : \alpha \vdash \text{Cons}(\text{append}(\text{Cons}(a, \text{Nil}), l), \text{foo}(x, l)) : \beta$$

c'est-à-dire que l'expression $\text{Cons}(\text{append}(\text{Cons}(a, \text{Nil}), l), \text{foo}(x, l))$ a le type β sous l'hypothèse que l est de type α .

Selon les étapes 1 et 4 de la partie B.6.1, on démontre que l'expression

$$\text{Cons}(\text{append}(\text{Cons}(a, \text{Nil}), l), \text{foo}(x, l))$$

a le type $\beta = \text{liste}(\text{liste}(\alpha_3))$ sous les hypothèses $l : \text{liste}(\alpha_3)$ et $l : \text{liste}(\alpha_2)$ avec $\alpha_2 = \text{liste}(\alpha_3)$.

Mais ici, on a seulement l'hypothèse $l : \alpha$ pour le type de l , donc on doit avoir $\alpha = \text{liste}(\alpha_3) = \text{liste}(\alpha_2)$, ce qui implique $\alpha_3 = \alpha_2$, qui contredit $\alpha_2 = \text{liste}(\alpha_3)$. Par conséquent, l'expression n'est pas typable.

B.6.3 Exemple sans hypothèses

Quand le type de certains symboles fonctionnels n'est pas donné, l'algorithme présenté d'inférence de type fonctionne encore, et il donne aussi le type le plus général possible pour ces symboles pour que l'expression soit typable. Cette partie présente un exemple extrême dans lequel aucune hypothèse ne fixe de type.

L'objectif de cet exemple est d'inférer le type de l'expression $\lambda x, y. g(x, f(f(y)))$.

La table B.1 présente sous forme de tableau la recherche d'une preuve que $\lambda x, y. g(x, f(f(y)))$ est de type α_1 .

La ligne (1) est la prémisse de la règle (*abstraction*) quand sa conclusion est la première ligne. Les lignes numérotées avec des lettres latines, comme (a), sont des égalités entre expressions de type. Ce sont des contraintes nécessaires sur les $\alpha_{\dots}$ pour qu'on puisse appliquer la règle. On les rassemblera en fin de recherche de preuve pour voir si elles ont une solution ou pas.

Les conditions (a), (b) et (c) sur les variables ne sont pas contradictoires. Une solution est

$$\alpha_1 = \alpha_5 \times \alpha_8 \rightarrow \alpha_4, \quad \alpha_2 = \alpha_5 \quad \text{et} \quad \alpha_3 = \alpha_8.$$

Les conditions (α), (β) et (γ) sur les symboles fonctionnels se généralisent en gardant, par exemple, les deux conditions (α) et (β). La condition (γ) peut être vue comme une instance (un cas particulier) de la condition (β).

Par conséquent, sous les hypothèses $g : \alpha_5 \times \alpha_6 \rightarrow \alpha_4$ et $f : \alpha_7 \rightarrow \alpha_6$, l'expression $(\lambda x, y. g(x, f(f(y))))$ a pour type $\alpha_5 \times \alpha_8 \rightarrow \alpha_4$.

Numéro	Hypothèses	Dédution élémentaire	Règle ou numéro
		$(\lambda x, y . g(x, f(f(y)))) : \alpha_1$ $\text{si } x : \alpha_2, y : \alpha_3 \vdash g(x, f(f(y))) : \alpha_4$ $\text{et } \alpha_1 = \alpha_2 \times \alpha_3 \rightarrow \alpha_4$	<i>(abstraction)</i> (1) (a)
(1)	$x : \alpha_2$ $y : \alpha_3$	$g(x, f(f(y))) : \alpha_4$ $\text{si } x : \alpha_5$ $\text{et si } f(f(y)) : \alpha_6$ $\text{et si } g : \alpha_5 \times \alpha_6 \rightarrow \alpha_4$	<i>(application)</i> (2) (3) (α)
(2)	$x : \alpha_2$ $y : \alpha_3$	$x : \alpha_5$ $\text{si } \alpha_2 = \alpha_5$	(b)
(3)	$x : \alpha_2$ $y : \alpha_3$	$f(f(y)) : \alpha_6$ $\text{si } f(y) : \alpha_7$ $\text{et si } f : \alpha_7 \rightarrow \alpha_6$	<i>(application)</i> (4) (β)
(4)	$x : \alpha_2$ $y : \alpha_3$	$f(y) : \alpha_7$ $\text{si } y : \alpha_8$ $\text{et si } f : \alpha_8 \rightarrow \alpha_7$	(5) (γ)
(5)	$x : \alpha_2, y : \alpha_3$	$y : \alpha_8$ $\text{si } \alpha_3 = \alpha_8$	(c)

TABLE B.1 – Application des règles d'inférence de type.

B.7 Synthèse

La connaissance du système formel présenté dans cette annexe ne fait pas partie du programme de ce module, mais on doit avoir compris les notions suivantes :

Nous pouvons distinguer des données et des programmes par leur type, vérifier automatiquement que ces données et ces programmes sont bien typés, et même deviner (le terme savant est “inférer”) le meilleur type possible pour une donnée ou un programme sans type, à partir d’hypothèses sur leurs ingrédients.

Cette vérification de type et cette inférence de type automatiques sont implémentées dans l’outil Why3, qui les exécute pour toute expression qu’on lui soumet. L’inférence de type autorise le spécifieur/programmeur à ne pas donner le type de chaque expression qu’il écrit, pourvu qu’il ait donné avant celui des éléments de base. La vérification de type permet de détecter rapidement toutes les erreurs de typage.

Annexe C

Solutions des exercices de l'annexe B sur la vérification et l'inférence de types

Exercice B.1

1. Le type de l'expression $(x+2)*y$ dépend de l'interprétation qu'on en fait : Si on interprète les variables dans un ensemble de nombres (les entiers par exemple) dans lequel le $+$ et le $*$ ont leur signification usuelle, alors cette expression est du même type que ses variables.
2. L'expression $append(Cons(a, l), ldf(t))$ est une liste de même type que $Cons(a, l)$ et que $ldf(t)$. Si a est de type α , alors $Cons(a, l)$ est de type $liste(\alpha)$. Donc il convient que t soit de type $arbre(\alpha)$ et il en résulte que l et l'expression complète sont de type $liste(\alpha)$.

Exercice B.2

Les types possibles en C sont short, int, char, float, double, ...

Exercice B.3

1. La fonction $\mathcal{P}$ peut être définie pour tout ensemble x par

$$\mathcal{P}(x) = \{y \mid y \subseteq x\}.$$

Ceci n'a de sens que si x est un ensemble, autrement dit est de type $set(\alpha)$, avec α un type quelconque.

Si x est de type $set(\alpha)$ et si $y \in \mathcal{P}(x)$, alors y est de type $set(\alpha)$. Le type de la fonction $\mathcal{P}$ est donc

$$set(\alpha) \rightarrow set(set(\alpha))$$

2. L'opérateur d'inclusion $\subseteq$ a pour type $set(\beta) \times set(\beta) \rightarrow \mathbb{B}$, donc l'expression $x \subseteq \mathcal{P}(x)$ serait correcte si $set(\alpha) = set(\beta)$ et si $set(set(\alpha)) = set(\beta)$, ce qui reviendrait à confondre α et $set(\alpha)$. Cette expression n'est donc pas correcte.

3. En revanche, l'opérateur d'appartenance $\in$ a pour type $\beta \times \text{set}(\beta) \rightarrow \mathbb{B}$ donc l'expression $x \in \mathcal{P}(x)$ est correcte en prenant $\beta = \text{set}(\alpha)$.

Exercice B.4

1. Compte tenu de la réponse à l'exercice B.1, l'expression $\lambda l. \text{append}(\text{Cons}(a, l), \text{ldf}(t))$ est de type $\text{liste}(\alpha) \rightarrow \text{liste}(\alpha)$.
2. Ici, nous ne proposons qu'une solution correcte, sans chercher à savoir si c'est la plus générale (la question du type le plus général de l'expression $f(f(y))$ peut être abordée en exercice). Pour que l'expression $f(f(y))$ ait un sens, remarquons qu'il suffit que f soit de type $\alpha \rightarrow \alpha$ lorsque y est de type α . On peut écrire $x \in f(f(y))$ s'il existe un type β tel que $\alpha = \text{set}(\beta)$. x est alors de type β . Par conséquent, un type possible pour $\lambda x, y. x \in f(f(y))$ est $\beta \times \text{set}(\beta) \rightarrow \{\text{vrai}, \text{faux}\}$.

Exercice B.6

L'arbre de preuve suivant utilise les règles d'inférence de type et les hypothèses suivantes pour prouver que l'expression $\text{Cons}(0, \text{Cons}(1, \text{Cons}(1, \text{Nil})))$ est de type $\text{liste}(\mathbb{B})$.

Hypothèses :

$$\begin{array}{ll} \text{Nil} : \text{liste}(\alpha) & (H_{\text{Nil}}) \\ \text{Cons} : \alpha \times \text{liste}(\alpha) \rightarrow \text{liste}(\alpha) & (H_{\text{Cons}}) \end{array}$$

Preuve :

$$\frac{(H_{\text{Cons}}), \quad 0 : \mathbb{B}, \quad \frac{(H_{\text{Cons}}), \quad 1 : \mathbb{B}, \quad \frac{(H_{\text{Cons}}), \quad 1 : \mathbb{B}, \quad \text{Nil} : \text{liste}(\mathbb{B})}{\text{Cons}(1, \text{Nil}) : \text{liste}(\mathbb{B})}}{\text{Cons}(1, \text{Cons}(1, \text{Nil})) : \text{liste}(\mathbb{B})}}{\text{Cons}(0, \text{Cons}(1, \text{Cons}(1, \text{Nil}))) : \text{liste}(\mathbb{B})}$$

La première déduction

$$\frac{\text{Nil} : \text{liste}(\alpha)}{\text{Nil} : \text{liste}(\mathbb{B})}$$

est un cas particulier d'une règle de substitution (qui remplace ici la variable α par $\mathbb{B}$), qui n'est pas formalisée dans le cadre de ce cours, mais correspond au théorème de substitution.